Emending Nonlinear Recurrence

Erik Garrison
erik.garrison@gmail.com
Poietic PBC / UTHSC, Memphis, TN, USA

Abstract

Conversation, perception, and feedback call for ongoing computation with predictable per-input cost. Emender explores a complementary path through efficient nonlinear recurrence within bounded memory: each input corrects the memory's current prediction and transforms the state carried forward. Width-axis multi-programming runs many small memory heads in parallel while preserving each head's sequential update. At 1.3 billion parameters, E97 trains through 150.8 billion Pile tokens on one eight-GPU workstation and reaches comparable training loss to a tightly parameter-matched GDN2 control. On the trained pair, modular-quadratic terminal accuracy is 99.96% versus 76.09% at trained length, and 76.07% versus 53.36% at eight times that length. This is a single-seed, common-cap comparison with unequal realized training over the tested lengths. On S_5 group-state tracking, GDN2 leads the long-horizon comparison. The negative transfer is: "The 8M S5 separation does NOT replicate at 1.3B on this pair." Lean 4 results describe the underlying state computations: E97 applies one tanh-of-affine state update per token, while a linear-state GDN fold under a fixed, state-independent parameter schedule is affine in its initial state, with zero nonlinear temporal depth. The artifacts also establish matched-signature delta-versus-raw-write separation and an S_5 routing bridge; minimal circuit-depth lower bounds and a full-stack impossibility remain open. The practical training result and iterated-map advantage support repeated nonlinear state transformation as a useful computational bias. Feedback, optimization, reservoirs, and dynamics define prospective structural applications; demonstrated deployments remain open. Recurrent tracker realization and constrained affine-channel realizability are explicit research targets.

Keywords recurrent neural networks · language modeling · nonlinear recurrence · state-space models · expressivity · Lean 4

1 Introduction

Keeping balance, following a moving object, and adjusting a movement require continual revision of an internal estimate as observations arrive. Conversation and perception likewise require a model to update what it understands while the interaction unfolds. Fast feedback puts this computation under a per-input time budget. A complementary question is how much of this ongoing computation can happen directly in carried memory as inputs arrive. Efficient nonlinear recurrence offers such a path: at inference, a fixed-width, fixed-depth model transforms fixed-size state with bounded work per update.

A sequence can require more than a record of what arrived. Its elements may be instructions that change a running calculation, measurements that alter a feedback response, or observations that revise an earlier association. In each case the current state is part of the next computation. A recurrent model reads an input, updates its internal state, and carries that state into the following step. The question is what computation the update performs, and how to train it efficiently at language-model scale.

This choice creates a systems problem. Modern linear-state recurrences use the composability of affine state updates to parallelize training along time [1], [2], [3], [4]. E97 keeps each head's nonlinear time loop serial and obtains parallelism across heads and batch elements. We call this *width-axis multi-programming*: many small memory programs run side by side, with each state tile kept on the accelerator (Section 5). This complements hybrid recurrent-attention stacks and iterative methods for parallelizing nonlinear recurrence [5], [6].

Restricted linear-state recurrence can have limited circuit complexity under explicit arithmetic and architectural assumptions [7]; the prefix products of a non-solvable group are a canonical diagnostic [8].

The resulting program joins a practical architecture to a testable computational hypothesis. Feedback systems, iterative optimization, and driven dynamics provide further settings in which a running state must be transformed repeatedly (Section 8).

The first empirical question is whether this route is practical. A purely nonlinear recurrent language model — *E97*, the current 1.3 B-class instance of the Emender family — made trainable by running many small recurrent programs in parallel across the network's *width* rather than across time, trains through 150.8 billion tokens on The Pile [9] using a single eight-GPU workstation, with no cluster and no sequence parallelism, reaching an estimated 0.93 bits per byte (Section 6). Its linear-recurrent baseline — *GDN2* (Gated DeltaNet-2 [4]), a mixer followed by the same SwiGLU MLP, selected by the same CMA-ES [10] search protocol at a parameter count within 0.01% — trains on literally matched tokens under the same 2k-chunk Pile recipe and lands in the same loss-versus-tokens band: at the matched 150.79-billion-token cutoff the smoothed loss gap between the two arms is 0.010 nats/token, with the sign of the gap depending on the averaging window (Section 6). The 1.3 B result is therefore a replication across the *update rule and tuning regime* at a single scale: both arms sit in the same 1.3 B-class parameter band, and whether the separation or the loss parity survives at 7 B and beyond remains a limitation in Section 10. On bulk training-log loss the two update families are operationally tied, so language-modeling loss does not separate them. Their differences emerge instead in state tracking. At 8 M scale, the delta-correcting update learns the S_5 prefix-product task more efficiently than the raw-write and GDN baselines under the tested protocols. At 1.3 B, the modular-quadratic probe reproduces a depth-wall signature, but the S_5 ordering does not transfer (Section 7). The Lean core proves matched-signature one-step separation, exact non-equality at every finite k on a constructed witness, and a 120-state recognizer floor. Its generator-table lookup construction proves the lookup bridge; a theorem that the tanh recurrence implements the full tracker is the trajectory bridge still open as Open item O1 (Section 4). The duality and the cited complexity results hold at their stated resource classes; a blanket linear-state S_5 impossibility remains open.

The second question is what these trained models can learn to compute. An iterated nonlinear map repeatedly transforms a running value, directly testing the computational pattern that motivates E97. After fine-tuning on a modular-quadratic recurrence, E97 reaches 99.96% versus 76.09% accuracy at the trained length and 76.07% versus 53.36% at eight times that length. Its

higher accuracy and greater length robustness support nonlinear transformation in the carried-state path as a useful inductive bias. The S_5 group-state task instead asks the models to compose permutation actions. GDN2 leads that long-horizon comparison. Section 7 gives the protocols, learning trajectories, and length-dependent results for both tasks.

Emender uses a nonlinear matrix memory. At each step, a head reads its current prediction at an input-supplied key, writes a correction toward the supplied value, and passes the updated state through tanh. The write uses what the memory already contains; the nonlinearity transforms what the next step will receive. Processing a long sequence therefore executes a long composition of nonlinear state updates.

An *emender* is one recurrent layer, and *Emender* is the family of networks formed by stacking these layers. The current instance, *E97*, uses separate key-axis erase/read and value-axis write gates. Each head has a small matrix state, while projections, gates, and a SwiGLU MLP connect the memories to the rest of the network (Section 3). The memory is revisable: later inputs can correct earlier associations.

Tanh-with-latching complements delta correction. Under the hypotheses of theorem set F (Section 4), a saturated slot resists perturbations, keeps its sign below a write threshold, and releases under a sufficient counter-delta. These are conditional slot-level guarantees; the hold theorem `EmenderLatching.emender_latch_holds_by_default` assumes $\lambda \geq 1$, while production decay sits below one. Persistent, revisable memory is the design motivation; a proved default persistence behavior of the trained stack remains open. E88 is the developmental predecessor (Section APPENDIX A).

The formal analysis describes the state computations behind this comparison. Lean 4 verifies E97′s time-ordered nonlinear composition and the affine GDN state fold under a fixed, state-independent parameter schedule. It also relates their split-gated cores and formalizes the S_5 tracker and routing conditions (Section 4). These statements make the mechanism precise and supply concrete targets for further experiments and proofs.

It also proves matched-signature delta-versus-raw-write separation, split-gated relations, slot-wise latching guarantees under stated hypotheses, and the S_5 recognizer floor (Section 4). These are structural facts; application-level approximation lower bounds remain open. The empirical work asks what SGD finds: at parameter-matched 8 M scale the Emender reaches 0.79 on S_5 against 0.36 for GDN and 0.22 for raw-write (Section 7), whereas the current 1.3 B pair supplies a positive iterated-map result and a negative group-state result. Width-axis multi-programming makes this theory-to-learning comparison practical.

Delta correction is one response; hybrids are another

The recurrent state-tracking problem admits more than one architectural response. *Hybrid* architectures interleave recurrent layers with attention layers and recover much of what attention-free recurrence loses on state tracking and long-range mixing. M²RNN [5], OLMo-Hybrid [11], Titans [12], and Griffin [13] are recent instances, and the limit-of-pure-recurrence story is documented by Merrill, Petty and Sabharwal [7]. *Delta correction* is a different response. It changes what the recurrent state writes at each step from a raw additive overwrite to a correction against the existing slot. The two responses are complementary: a hybrid stack can use any update rule in its recurrent layers, and delta correction is well-defined wherever a matrix-state recurrent layer is.

We evaluate delta correction in the attention-free, time-serial recurrent arena, the *pure nonlinear recurrent* (PNR) setting in the sense of "no cross-token attention mechanism"; gating, projections, normalizations and other intra-token nonlinear operations are permitted and load-bearing (Section 3 ingredient (d) and Section 6 on gradient conditioning). The PNR arena is where the contrast is cleanest (a delta-vs-raw-write contrast in the specified update-resource class, Section 4). Set D uses synthetic state-pass charges, excluding actual dense *HW* and feature-network costs; actual matched-compute certification remains open. The Lean 4 separation theorems are tractable in this arena (sets C and C′), and the scaling demonstration leaves no architectural escape hatch from the recurrent state.

The two recent attempts to scale nonlinear recurrence each took a different concession. M²RNN [5] trains nonlinear matrix-state recurrence at 7 B MoE in *hybrid form* (nonlinear-recurrent layers interleaved with attention layers); ParaRNN [6] trains nonlinear-recurrent LSTM and GRU at 7 B by parallelizing the time loop via Newton iteration on a block-bidiagonal Jacobian. These routes use attention or time-axis parallelization. This work takes the pure-recurrent route and compares a delta-correcting instance directly with a raw-write instance under matched conditions.

M²RNN already identifies the same basic hardware fact: a nonlinear recurrence cannot be scanned across time, so useful parallelism must come from batch and head parallelism rather than from the time axis. The difference here is how far that principle is pushed and what it is used to test. M²RNN promotes nonlinear matrix recurrence mainly as a sparse ingredient in hybrid stacks; under the addressing-bottleneck hypothesis, the pure setting may benefit from many independent *address programs* at a given value-head count — what we call *address-program multiplicity*. The CMA-reshaped raw-write comparator used here moves M²RNN into that same pure multi-programmed regime and is stable where the paper-default shape diverged. This controls the broad pure-recurrent systems setting for comparing raw overwrite with delta correction; width, depth, and head count also changed, leaving the cause of divergence unresolved.

Contributions

The contributions connect state-channel structure, practical training, and direct capability measurements.

1. *Viability.* E97 reaches an estimated 0.93 bits per byte on The Pile after 150.8 billion tokens of training on a single eight-GPU workstation, with no cluster and no sequence parallelism. The throughput route is width-axis *multi-programming*: 9,504 small bounded recurrent programs per token (216 heads × batch size 4 × depth 11), each a 32×32 matrix-state tile that fits in registers, with the time loop serial inside each program (Section 5). Pure-nonlinear-recurrent language models train into this regime at this scale without a time-axis parallelization trick or attention hybridization.

2. *Power separation within the formalized resource class: for the matched-signature update family studied here, delta correction supplies a mixed-key step absent from the fixed-right raw-write class in a shared synthetic state-pass accounting model.* A Lean 4 trusted core machine-verifies that no fixed-weight raw-write matrix RNN with row, column, or cell forget gates can match the Emender's delta-correcting update in one step (`emender_m2rnn_one_step_resource_separation_embeds`, set C in Section 4), and that the gap persists for every finite k under composition on a constructed

witness alphabet (`emender_m2rnn_k_step_separation`, set C′). The cost comparison (`emender_m2rnn_flop_class_equiv`, set D) uses coarse state-pass charges, excluding actual dense right-multiplication HW and feature-network costs; empirical compute matching requires separate measurement. This is a representability statement inside the formalized fixed-weight raw-write resource class; general or learned-weight superiority remains open. Empirically the delta-vs-raw-write contrast is a matched-budget learning-efficiency gap: at matched no-tuning budget raw-write under-reaches the delta update's S_5 length generalization, and even at a tuned best-effort budget it does not catch up (Section 7). The predicted ordering already appears at the 8 M parameter-matched probe shape where the Section 7 state-count argument gives large storage headroom: the Emender reaches 0.79 on the S_5 word problem against 0.22 for raw-write M²RNN-CMA (Section 7).

3. *Paired 1.3 B comparison: loss does not distinguish the update rules; matched tokens do not favor either.* Two 1.3 B-class models — E97 (delta-correcting, nonlinear state) and GDN2 (linear-state, the strongest closely related recurrent baseline, CMA-ES-hardened under the same search protocol) — train from scratch on the same 2k-chunk Pile recipe with literally matched aggregate tokens (150.79 B) at a 0.01% parameter gap, and land in the same loss-versus-tokens band (Section 6). At 480 M scale, four CMA-ES-tuned recurrent families share one FLOPs-per-bit curve (Section 6). The contribution is the architectural *option* plus the *mechanism*; training speed supplies no win, and bulk loss is a convergent null across the tuned families.

4. *Expressivity at 1.3 B: a task-dependent separation.* On the trained pair itself, the modular-quadratic probe separates terminal accuracy and length robustness under a pre-registered common optimization cap (Section 7, Table 8). The S_5 separation does *not* replicate: GDN2 is ahead in the long-horizon probe (Table 9). Realizability describes representational scope; the two learning findings jointly delimit the theory-to-empirics transfer.

The 1.3 B pairing is E97-MLP (1,286,589,072 parameters; the Emender split-edit delta-correcting update — a key-axis erase/read gate and a value-axis write gate around the bounded delta correction, Section 3 — with a bias-free SwiGLU MLP per layer) versus GDN2 (1,286,713,448 parameters; the official-style Gated DeltaNet-2 [4] mixer with the same SwiGLU MLP), both selected by per-architecture CMA-ES [10] search at the 1.3 B band on The Pile and both trained from scratch under the identical 8-island DiLoCo recipe (Section 6). The raw-write contrast (M²RNN [5]) is not a 1.3 B arm — its public XMA kernels do not run on this machine's hardware stack, and a local reimplementation would add an implementation-fidelity confound — so it is carried at the 8 M probe scale and in the Lean separation instead (Section 7, Section 4). The exact per-arm search provenance, budgets, and training identities are reported in Section 6.

The paper proceeds from background (Section 2) and architecture (Section 3) through formal results (Section 4), systems (Section 5), matched-token language modeling (Section 6), and expressivity probes (Section 7). The applications class (Section 8) extends the structural question without claiming new deployments; related work (Section 9), limitations (Section 10), conclusion (Section 11), testable predictions (Section 12), and future work (Section 13) complete the body.

The release targets for the 1.3 B pair are the immutable per-arm authorities: E97 (`https://huggingface.co/spinozans/emender-e97-1.3b`, tag `tokens-150793748480`, the 150.79 B-token checkpoint of `emender_E97_1.3B_20260709_084606`) and GDN2 (`https://huggingface.`

co/spinozans/gdn2-mlp-1.3b, tag tokens-152280498176), with the public paper PDF mirrored at http://hypervolu.me/~erik/ndm/Garrison_2026_Emender.pdf.

Claims, evidence, and scope

The table below states each load-bearing claim with its evidence, its scope, and — in the last column — the boundary the same result observes. The body sections substantiate each row; the pointers are forward references.

Claim	*Evidence*	*Scope*	*Non-claim*
Pure nonlinear-recurrent LMs train at the 1.3 B-class	E97 reaches an estimated 0.93 bpb on The Pile at 150.8 B tokens, no time-axis trick, no attention hybrid (Section 1, Section 6)	One E97 run, single seed; viability demonstration	Not a scaling law and not seed-averaged; one artifact shown to exist
Bulk LM loss does *not* distinguish the 1.3 B pair (a measured null result)	Matched-token cutoff at 150.79 B: smoothed gap 0.010 nats/token, sign window-dependent; both arms CMA-ES-hardened (Section 6)	A measured null result on *training-log* loss at exactly matched tokens; the CMA-stage 5-model held-out band (2.01–2.10 bpb) corroborates at search scale (Section 6)	Held-out bpb for the 150 B pair itself is pending (TODO-NEEDS-EVIDENCE, Section 6); does not license a loss ordering
Delta-correcting > raw-write on state tracking, at every length	8 M S_5 probe, 3 seeds (Figure 3); Lean matched-resource separation (Section 4, sets C/C′/D)	Budget-robust *ordering* and representability statement inside the formalized fixed-weight raw-write class	Not "delta solves / length-generalizes S_5": at 8 M both nonlinear updates plateau below ceiling at length; it is an efficiency gap, not impossibility
A controlled tanh/ identity update-map intervention tests length transfer	Iterated-map probes: E97 holds $\approx$ 0.98 flat from $T = 128$ to $T = 4096$ where byte-identical E97-linear and GDN2 collapse; two negative task families are consistent with the mechanism hypothesis (Section 7)	Small-scale (dim 256–1024) grokking-regime statement, with an isolated update-map flag but different successful-seed subsets	Not an unconditional causal effect or exact algorithm-recovery result; the two negative families reject a universal superiority claim
Group-state tracking is protocol-dependent	8 M and predecessor GDN probes decay at length; the current GDN2 pair reverses the predicted ordering (Table 9)	Measured learning behavior; Lean proves the architecture-agnostic 120-state recognizer floor (Section 4)	No blanket affine-channel impossibility; constrained realizability is open (G3), and TC^0 versus NC^1 is unresolved
Expressivity at 1.3 B on the trained pair	Modular-quadratic terminal accuracy: 99.96% vs 76.09% E97 is more length-robust (Table 8); S_5 does not separate in the predicted direction (Table 9)	One seed, released checkpoints, common pre-registered cap and plateau rule; unequal realized step counts (Section 7)	Not universal nonlinear superiority; GDN2′s long-length plateau is not the random baseline, and S_5 is a boundary finding

Table 1: *Claim / Evidence / Scope / Non-claim.* Each row is substantiated in the cited section; the final column marks the boundary the result does not cross. The architectural separations are *efficiency* and *computability* statements, not blanket can/can't claims; the bulk-loss row is a measured null result; the last row reports a task-dependent separation together with its negative boundary finding.

M²RNN uses width and batch parallelism for a nonlinear recurrence that is serial in time. Its published scaling route emphasizes *hybrid* use, while the CMA-reshaped raw-write control here shows what the raw-write family does in a stable pure-recurrent configuration. Under the addressing-bottleneck hypothesis, that reshape may reduce shared-address concentration, but the runs do not isolate this mechanism.

2 Background

The central distinction is where nonlinear computation occurs. Mamba [1] and Gated DeltaNet [3] use nonlinear input processing and gates, and their full networks are nonlinear functions of the sequence. Once a recurrent layer's input-derived transition parameters are fixed, its carried-state update is affine in the previous state. E97 instead transforms the carried state nonlinearly at each step. These choices organize computation across time differently and call for different execution strategies.

The stakes are the general ones of sequence modeling at scale, and one workload makes them concrete. Pangenomic sequence data runs to terabases per study [14], [15], [16], so modeling it at the byte level means ingesting trillions of tokens for any downstream operation, which rules out routine use. Linear-recurrent byte-level foundation models were the first attempt in this regime and did not scale reliably. The Merrill–Petty–Sabharwal results [7] give conditional circuit-containment results for restricted linear SSMs under their transition and arithmetic assumptions. A non-solvable-state-tracking impossibility would additionally require the open complexity-class separation. This motivated studying nonlinear-in-time updates as an empirical research direction; the Emender of Section 3 is the construction evaluated here.

Notation

The same state-channel symbols are used in the architecture, applications, and formal sections. Head and time subscripts may be omitted for a single step. A prime denotes the returned state. Matrix products and scalar scaling use juxtaposition; $\odot$ denotes componentwise multiplication, and T in a superscript denotes transpose.

Symbol	*Meaning and source-name correspondence*
S, S_h, S_t, S'	Carried state, head state, time-indexed state, and returned state of E97/E88 and GDN. Lean names this matrix `H`.
H, H_t	M²RNN comparator state only; H also denotes the head count when explicitly described as a count.
N, V, d	Key-axis and value-axis dimensions; square formal state dimension $d = N = V$. Lean key dimension is `K`.
k, q, v	Write key, read query, post-activation value injection.
e, w, g	Key-axis erase/read gate (Lean `b`), value-axis write gate, output gate. b_t in an affine fold is an additive offset, not the erase gate.
δ, r	Value-vector correction and retrieved value in the cell.
d_h, λ, D	Production per-head decay, abstract scalar decay, and GDN pre-decay matrix. d_h plays λ's role; the formal E97 fold fixes λ, while production decay depends on the input. In the contractive control, λ instead denotes a Lyapunov exponent.
x_t, u_t, θ	Token/layer input, generic input injection, and fixed update-law parameters; v is the cell's value injection, not every application's input.
t, T, h, ℓ, (i, j)	Step, sequence length, head, layer, and matrix-slot indices. Local latch threshold T is not length.

Symbol	*Meaning and source-name correspondence*
ω, η, ε, M, μ	Slot write contribution, saturation slack, post-update slack, perturbation bound, and release margin. Scalar perturbation δ in set F is not the value vector.
s, r_t, w_t, g_t, z_j, h_t, n_t, x_t	Local states: group element, converter residue, filter coefficients, log-gain, optimization iterate, reservoir, population, and plant. These application-specific coordinates instantiate generic state S; x_t also denotes the iterated-map task state in the probe sections.

Linear-state and nonlinear-state recurrence

The recurrent language model lineage runs from Elman networks [17] and the LSTM [18] through the modern linear-recurrent variants: state-space models such as Mamba and Mamba2 [1], [2], gated linear attention [19], and the delta rule [20]. Many modern designs favor time-axis parallelism via linear-state updates, whereas nonlinear-in-time recurrence generally uses a different execution strategy. Gated DeltaNet-2 [4], the split erase/write-gate extension of Gated DeltaNet [3], supplies the linear-state mixer in the current GDN2 control (Section 6).

Gated DeltaNet [3] is the linear-recurrent family the Section 7 8 M probes use as the linear baseline, and Gated DeltaNet-2 [4] — its split erase/write-gate extension — is the linear-recurrent baseline of the 1.3 B matched-token comparison (Section 6). Mamba-3 [21] is reported by its authors to outperform GDN; its compilation-heavy HPO does not fit the Section 6 matched protocol, so it is left to future work.

A single explicit criterion classifies recurrent architectures. A recurrent layer is *linear-state* if its update can be written $S_t = A_t S_{t-1} + b_t$ with A_t and b_t determined by input-derived features, independently of that layer's previous carried state. Conditioning on these parameters makes the update affine in S_{t-1}. Here the state is expressed in vector coordinates; a matrix state may be vectorized for this criterion. If the previous state S_{t-1} appears nonlinearly in its own update equation, the layer is *nonlinear-state*. The distinction is structural across activation choices: a tanh, an exponential, a tanh-of-product, or a sigmoid.

Linear-state designs admit a chunkwise or scan-based parallel implementation along the time axis because composing $S_T = A_T S_{T-1} + b_T = \ldots$ unfolds into a product of inputs only. This is the reason linear-state recurrence dominates the billion-parameter landscape. Mamba, Mamba2 (SSD), RetNet, GLA, DeltaNet, Gated DeltaNet, RWKV-4/5/6/7, HGRN2, mLSTM, MinGRU/MinLSTM [22] and Griffin's RG-LRU [13] are all linear-state by this criterion. At fixed precision and width, both state regimes are finite-state recognizers. Circuit-complexity containment results for restricted linear SSMs require additional arithmetic and transition assumptions [7]. Containment in TC^0 places restricted linear SSMs in that class; an inability to solve an NC^1-complete word problem would require the still-unproved separation $\mathrm{TC}^0 \neq \mathrm{NC}^1$. Affinity admits a finite-group action; realizability in the constrained GDN channel remains open (G3, Section 10). For fixed-depth transformers in the same formal-language setting, this is also the relevant ceiling under explicit arithmetic assumptions: log-precision transformers are captured by first-order logic with majority quantifiers [23], and average-hard attention transformers exactly, plus softmax-attention transformers with $O(\mathrm{poly}(n))$ floating-point precision or $2^{-O(\mathrm{poly}(n))}$ absolute-error approximation, lie in DLOGTIME-uniform TC^0

[24]. The classical empirical witness is the symmetric-group S_5, which has 120 elements and is the smallest non-solvable symmetric group.

Nonlinear-state recurrences include the classical LSTM and GRU; the sLSTM block of xLSTM [25] (memory-mixing through the hidden state inside exponential gates); M²RNN [5] (matrix state inside tanh with raw-write injection); Titans [12] (MLP memory with online gradient updates); and the architecture introduced here. The Siegelmann–Sontag result [26] concerns nonlinear recurrent nets under its stated precision and resource assumptions; Turing-completeness of a fixed-width, fixed-precision E97 remains outside that result. Barrington's theorem identifies the non-solvable S_5 word problem as an NC^1-complete witness; separation of NC^1 from TC^0 remains open.

Two nonlinear matrix-state designs (the Emender and M²RNN) therefore share the necessary preconditions (matrix state, nonlinearity on the state, no attention, no linearization, no hybridization) that define the pure-nonlinear-recurrent class introduced in Section 1 and differ in one place: the per-step update rule on the matrix state. The Emender uses a *delta-correcting* update; M²RNN uses a *raw-write* update; both are pure-nonlinear-recurrent instances. The within-class observation in Section 1 is a task-dependent learning-efficiency ordering under the reported budgets and per-architecture CMA-ES; a general learnability theorem remains open. The Lean cost anchor `RecurrentResourceFormalism.emender_m2rnn_flop_class_equiv` establishes only equivalence in a synthetic, coarse state-pass accounting model (Section 4 Theorem set D). It excludes the actual dense right-multiplication HW and feature-network costs, charging a state-sized atom rather than the arithmetic of that multiplication. Actual matched empirical compute requires separate measurement.

Matrix state

Matrix state $S \in \mathbb{R}^{N \times V}$ is shared by several recurrent architectures, replacing a vector hidden state $h \in \mathbb{R}^d$. Linear-state designs (mLSTM, RWKV-5/6, DeltaNet) already use matrix or expanded states; RetNet's accumulation $S_t = \gamma S_{t-1} + k_t v_t^T$ is matrix-valued. The point of matrix state is that an outer-product update $S_t = ... + k_t \delta_t^T$ provides $O(NV)$ scalars of dynamic state at $O(NV)$ computational cost per token, with content-addressable retrieval via $S^T q$. Matrix state is common to the post-Mamba landscape (Mamba2 with `d_state`, GLA, DeltaNet, RWKV-5/6/7, mLSTM, M²RNN) and is treated here as a precondition shared by the Emender and its baselines.

The S_5 state-tracking probe

The symmetric group S_5 is the group of permutations of five items; it has 120 elements and is the smallest non-solvable symmetric group. Its *word problem* is the following task: the model reads a sequence of permutations — here, adjacent transpositions, the swaps of two neighboring items — one at a time, and after each one must report the running composition of everything read so far, its *prefix product*. By Barrington's theorem [8] this task is complete for the complexity class NC^1 when exact recognition is required over all words. The experiments measure prefix-product accuracy at specified lengths. The solvable-group control S_3 (permutations of three items, 6 elements) is included to factor out the part of difficulty that comes from prefix tracking *per se* rather than from non-solvability. Training-length learning failure is an empirical observation about SGD, separate from circuit-class membership.

A dynamics taxonomy of head-types: eigenvalue placement × saturation

The linear/nonlinear hinge above, and the choice of where a head's state may decay, reflect, or oscillate, are two independent axes of a single design grid. The *dynamics* of a head's per-token state map locate it in a two-axis grid:

1. *Eigenvalue placement in the unit disk.* Where the along-key eigenvalue of the per-token state transition sits in the complex unit disk. A real-positive eigenvalue is a *decay* — the stored value fades toward zero along the key. A real-negative eigenvalue is a *reflection* that flips sign each step (`reflect`); reaching it is the lever that lets a recurrence track non-solvable group structure such as S_5 in the restricted transition analyses motivating this grid. A complex-conjugate pair is a *rotation* (`rot`) that oscillates the stored value. Real-positive and real-negative are the two ends of the *real diameter*; complex placement walks off that diameter into the rest of the disk.
2. *State map: linear or saturated.* Whether the state update is left linear or passed through a bounded saturating nonlinearity (canonically hardtanh; the `-nonlin` axis; the Emender uses the smooth variant tanh). Saturation latches a driven slot and makes the head bounded; finite precision then gives a finite state space.

The grid these two axes span names every Emender head-type (Figure 1). Real × linear at the positive end is `decay`, which is vanilla Gated DeltaNet; at the negative end it is `reflect`, Gated DeltaNet permitted a negative eigenvalue. Real × saturated is `nonlin` — the saturated delta-correcting head, historically tagged E97, whose split-gated instance is the 1.3 B model of this paper with split erase/read and write gates. Complex × linear is `rot`, the complex-eigenvalue oscillator head. Complex × saturated is `rot-nonlin`, a nonlinear oscillator left to future work, included as the combination of the two axes. E97 identifies the trained instance; the architecture vocabulary is this grid.

The adjacent split-gated cores locate the linear-recurrent baseline in this grid.

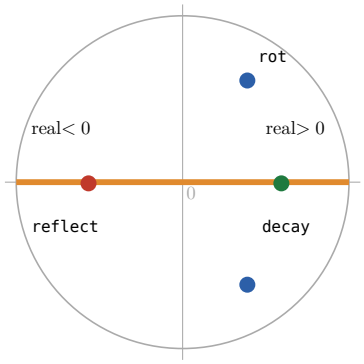
Shared split-gated cores, with different decay placement. Gated DeltaNet-2 and E97 have adjacent split erase/write cores. E97 uses $\lambda S - k(e \odot k)^T S + k(w \odot v)^T$, whereas GDN-2 uses $(I - k(e \odot k)^T)DS + k(w \odot v)^T$. The exact Lean identity applies E97′s unit-decay linear core to the *pre-decayed* state DS; with identity decay the cores coincide (Section 4). The grid organizes these heads by their state dynamics. Whole-production-stack inclusion and production-domain parameter remapping remain open.

3 Architecture

Per-head update

Section 1 named the emender layer by what it does — read what the memory predicts at the addressed slot, compute the prediction error, and write the correction — and named E97 as the 1.3 B instance carried through the paper. This section makes the layer concrete: the per-step computation, the choices that fix it, and the shape numbers that make E97 one specific member of the family.

A. Eigenvalue placement in the unit disk



Real diameter (orange) = GDN-2; off-axis = rotation.

B. Head-type grid: eigenvalue placement × saturation

	decay (real > 0)	*reflect* (real < 0)	*rot* (complex)
linear	`decay`	`reflect`	`rot`
+ hardtanh (`-nonlin`)	`nonlin`	`nonlin`	`rot-nonlin` (future)

Shading: GDN-2-adjacent linear cores; decay placement differs.

Figure 1: *The Emender head-type taxonomy: eigenvalue placement × saturation. (A)* Each head is fixed by where its along-key eigenvalue sits in the complex unit disk — real-positive `decay`, real-negative `reflect`, or complex `rot` — and by whether the state map is linear or saturated. The orange real diameter is the locus GDN-2 occupies. *(B)* The two axes name five cells. Shading marks GDN-2-adjacent linear cores; the Emender also explores off-axis rotation (`rot`) and saturated state maps (`nonlin`) as a within-layer pool. This taxonomy is descriptive: different decay placement prevents an exact equal-parameter restriction of the production cores. `rot-nonlin` is reserved for future work. E88/E97/E98/E99 are historical run tags, not architecture names. The GDN-2 corner of this grid is not a strawman: it is the mixer of the 1.3 B linear baseline of Section 6 (GDN2), selected by the same search protocol as E97.

An emender layer runs H independent heads side by side. Each head h owns its own matrix-state memory $S_h \in \mathbb{R}^{N\times V}$ — the matrix state of Section 2, here addressed by an N-dimensional key and holding a V-dimensional value at each addressed slot. (Section 4′s resource theorems use a single symbol d for the common matrix dimension; in E97, $d = N = V = 32$, so each head's memory is a 32×32 tile.) At each token, the layer's input projections supply, per head, a key k_h and query q_h in $\mathbb{R}^N$ that name the slots to write and read, a value $v_h \in \mathbb{R}^V$ to store, a scalar decay $d_h \in (0, 1)$ that fades the old state, and an output gate $g_h \in \mathbb{R}^V$. E97 controls its write with two input-dependent gates, split along the two axes of the matrix state: a key-axis *erase/read* gate $e_h \in (0, 1)^N$ that scales the address under which the old content is retrieved, and a value-axis *write* gate $w_h \in (0, 1)^V$ that scales what is written (the GDN-2-inspired half of the design; Section 4′s split-gated theorem set pins the relationship). The display below is one head's full step in execution order: normalize the two addresses, gate the read address and the write value, read the memory, form the retrieval error, update the bounded state, and gate the read output. The formal core identities in Section 4 use abstract real-valued key/gate/state domains. The production ranges are specified above. Reachable-trajectory proofs for the production ranges remain open.

$$
\begin{aligned}
k_h &\leftarrow \frac{\mathrm{silu}(k_h)}{\|\mathrm{silu}(k_h)\|_2} \quad (\mathrm{L}^2\text{ -normalized key}) \\
q_h &\leftarrow \frac{\mathrm{silu}(q_h)}{\|\mathrm{silu}(q_h)\|_2} \quad (\mathrm{L}^2\text{ -normalized query}) \\
e_h &= \sigma(E_h x_t) \quad \text{(key-axis erase/read gate)} \\
w_h &= \sigma(W_h x_t) \quad \text{(value-axis write gate)} \\
r_h &= S_h^T (e_h \odot k_h) \quad \text{(gated read at the key)} \\
\delta_h &= w_h \odot v_h - r_h \\
S'_h &= \tanh\big(d_h S_h + k_h \delta_h^T\big) \\
y_h &= \mathrm{silu}(g_h) \odot (S'_h)^T q_h
\end{aligned}
$$

The displayed normalization is idealized for nonzero addresses; production regularizes with `norm+1e-6` (shared fused normalization: `elman/models/e88_fused.py:239-242`).

Here silu is the smooth pointwise activation $x \odot \sigma(x)$, with σ the logistic sigmoid, v_h is the post-activation value input (the $\mathrm{silu}(v)$ of the E88 cell, so the write gate scales an already-activated write), and $\|\cdot\|_2$ is the Euclidean norm. The first two lines put the key and query on the unit sphere so the addressing is bounded; the middle lines make the write a correction against the content already read — under a gated address and at a gated magnitude; and the final line gates only the read output. With both split gates fixed to one ($e_h == 1,\ w_h == 1$) the step specializes exactly to the E88 cell (Lean: `SplitGatedDelta.e97_specializes_to_e88_all_one_gates_direct`), which is how the predecessor instance was defined; the state stays inside tanh, so the update is nonlinear-state for every gate setting. This is the per-head primitive. Algorithm 1 spells out the same read-before-write order; its inputs are the projected quantities described above, and its output uses the returned state. It is the per-head core, bound to the full stack by limitation G2 (Section 10).

Algorithm 1 — E97 per-head step.

```
Input: S, projected k, q, activated v, d, e, w, g
1. k <- silu(k) / norm2(silu(k))
2. q <- silu(q) / norm2(silu(q))
3. r <- transpose(S) (e hadamard k)
4. delta <- (w hadamard v) - r
5. S_next <- tanh(d S + outer(k, delta))
6. y <- silu(g) hadamard (transpose(S_next) q)
Return: S_next, y
```

E97 repeats the same per-head equation across a residual stack.

E97 fixes these family choices to one billion-parameter instance: model width 1792, depth 11, $H = 216$ heads per layer, $N = V = 32$, expansion 1.0, a bias-free SwiGLU MLP of hidden width 4,032 after each mixer (mlp ratio 2.2623), per-rank batch size 4, a recurrence chunk length of 2048 tokens, and 1,286,589,072 parameters in total (1{,}048{,}152{,}720 non-MLP plus 238{,}436{,}352 in the eleven MLPs). For E97, at depth 11, $H = 216$, and batch size 4 the stack runs $11 \times 216 \times 4 = 9{,}504$ independent recurrent programs per token, each carrying a 32×32 state tile ($216 \times (32 \times 32)$ recurrent-state scalars per layer). Around the recurrent core the language-model wrapper is conventional — a pre-norm residual stack with a final normalization and tied input/output embeddings. Heads never share recurrent state; each layer learns its own

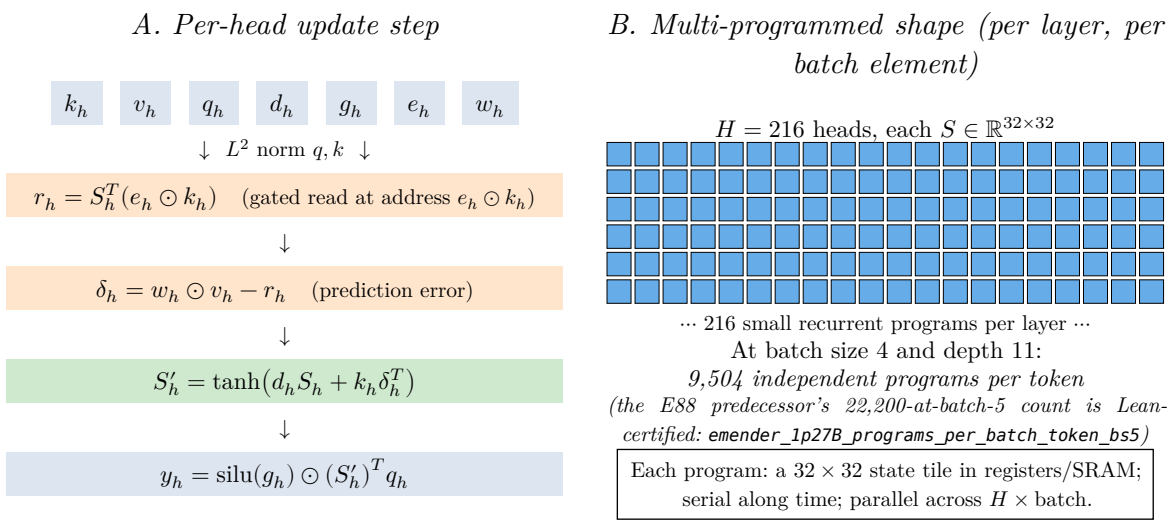


Figure 2: *Emender architecture and multi-programmed shape. (A)* Per-head recurrent dataflow (E97). Each head owns a small bounded associative matrix memory S_h. The model reads under the gated address $e_h \odot k_h$, computes a prediction error δ_h against the gated write value, writes the bounded delta correction into S_h, and gates the read at q_h for output. The tanh on the state, the delta-correcting write, the split erase/write gates, and the L^2-normalization of q, k are visible here; Section 3 enumerates the four load-bearing ingredients of the Emender family. *(B)* The 1.3 B stack exposes 216 small independent heads per layer per batch element. Each head is a 32×32 state tile that fits in registers/SRAM and runs its time loop serially; parallelism is harvested across heads, depth and batch (Section 5), giving $11 \times 216 \times 4 = 9\{,\}504$ programs per token at batch size 4. The Lean theorem `emender_1p27B_programs_per_batch_token_bs5` certifies the corresponding count (22,200) for the E88 predecessor geometry.

key, query, value, decay, split-gate, and output-gate projections; and the recurrent state starts at zero when no prior state is carried in.

For the Emender family, four ingredients are load-bearing; E97 uses all four in the instance above.

(a) *Bounded matrix state via* tanh *on* S. The state-channel duality (Section 4) distinguishes the executed nonlinear composition from an affine fold. Applying the nonlinearity to the carried state makes an emender layer nonlinear-state.

(b) *Delta-correcting write* $w \odot v - S^T(e \odot k)$. The emender layer reads what memory predicts at the gated address $e \odot k$, computes the prediction error δ against the gated write value $w \odot v$, and writes the correction. Under unit decay and all-one gates, the pre-tanh core with orthonormal keys gives exact overwrite at one slot while preserving orthogonal slots (`linearDeltaWrite_overwrites_one_preserves_others`); full-state tanh, split gates, and arbitrary keys change that guarantee. A raw-write update $S \leftarrow S + kv^T$ (the M²RNN family) accumulates without correction and cannot, with fixed weights, satisfy a uniform one-step overwrite specification (Section 4).

(c) *Saturation latching of the* tanh *bound.* A slot driven near ±1 resists perturbations under the saturation theorem's hypotheses; sign-preserving hold and counter-delta release have separate decay and write-budget assumptions. These motivate revisable memory; persistent bindings in production E97 remain a conditional property. Section 4′s Theorem set F (saturation insensitivity, sign-preserving hold, counter-delta release) formalizes all three as slot-wise statements on the full emender-layer update.

(d) *Many independent small heads.* E97 at 1.3 B

uses $H = 216$ heads of 32×32 each ($11 \times 216 \times 4 = 9\{,\}504$ independent programs per token at batch size 4; the E88 predecessor's 22,200-at-batch-5 count is Lean-witnessed by `RecurrentResourceFormalism.emender_1p27B_programs_per_batch_token`). Per-head L^2 -normalized q, k give many independent addressing programs and avoid the gradient-conditioning failure mode seen when a single shared q, k pair feeds hundreds of value heads.

GDN2 state channel

Algorithm 2 defines the adjacent GDN-2 linear core used by the formal comparison: decay is applied *before* retrieval and correction, and no tanh is applied to the returned state. Its inputs are an already supplied key, split gates, value, and pre-decay matrix. E97 applies tanh to its updated state. Both trained models also use nonlinear input processing and the SwiGLU MLP described in Section 6. GDN2 retains its own projection processing. Algorithms 1 and 2 present the abstract core; the released baselines add projection, normalization, gate-domain, and residual machinery whose full-stack binding is carried by limitation G2 (Section 10).

Algorithm 2 — GDN-2 split-gated linear core.

```
Input: S, D, k, e, w, v
1. P <- D S
2. r <- transpose(P) (e hadamard k)
3. delta <- (w hadamard v) - r
4. S_next <- P + outer(k, delta)
Return: S_next
```

Parameterization choices

E97 parameterizes the per-head decay d_h in log space, following Mamba2 [2]: each head learns a scalar $A_{\log}$ and a bias δ_{bias} — the code names `A_log` and `dt_bias` — and forms the decay as $d = \exp\big(-\exp\big(A_{\log}\big) \cdot \text{softplus}(\alpha(x) + \delta_{\text{bias}})\big)$, where $\alpha(x)$ is the input-dependent term and σ is again the logistic sigmoid. Both decay parameters are exempt from weight decay, as is standard for such timescale parameters. The recurrence is computed in float32 and cast back to the storage precision, and the tanh on the state uses the numerically stable form $\tanh(z) = 2\sigma(2z) - 1$ in the fused kernel to avoid overflow at large pre-activations. Three remaining choices are: there is no output normalization inside the recurrent layer — an ablation removed it for a 0.10-nat gain, and the wrapper's block-level pre-norm plus final norm suffice — no short convolution on the input. E97 uses the split erase/read and write gates above; its output gate $\text{silu}(g)$ still acts on the read alone.

The 1.3 B stack

Ingredient (d) sets the 1.3 B stack's shape — many small heads — and the numbers follow from it: each head's 32×32 tile is small enough to live in registers (Section 5), the head count $H = 216$ is large enough to give that many independent addressing programs per layer, and eleven such layers — each followed by its SwiGLU MLP, the channel-mixer that pairs with the token-mixing recurrence — reach the 1.3 B-class budget the paired comparison fixes. The exact values were selected by the per-architecture search of Section 6. At batch size 4, depth 11 and $H = 216$ multiply to $11 \times 216 \times 4 = 9\{,\}504$ independent recurrent programs per token, each carrying its own 32×32 state tile. The reference implementation and the fused Triton recurrence kernel that steps these tiles are identified by the source register and release manifests.

The matched architectural contrast

E97 and GDN2 both use matrix memory, delta correction, split editing gates, and a SwiGLU channel-mixing MLP. The key distinction for the present hypothesis is the operation performed on carried state: E97 applies tanh after its correction, whereas the GDN2 state update is affine when its input-supplied parameters are fixed. Decay placement also differs, as the core identities above specify. The full networks contain nonlinear projections, gates, and MLPs. The comparison centers on the transformation applied to carried state. The matched-pair probes test what the trained architectures learn under the recorded protocol (Section 7); the formal analysis states the narrower update-map facts (Section 4).

Architectural contrasts: delta correction and state nonlinearity

Three architectural properties are candidates for the one that lets a model track state: *matrix state*, *temporal nonlinearity on that state*, and *delta correction in the write*. The closest update-rule comparator to the Emender in the literature is M²RNN [5]. Its nonlinear matrix-state update has two parts, written here as one display: the first line builds a nonlinear write candidate Z_t from the matrix state H_{t-1}, a learned weight W, and the current key–value outer product $k_t v_t^T$; the second blends that candidate back into the state through a forget gate f_t.

$$Z_t = \tanh(H_{t-1}W + k_t v_t^T)$$
$$H_t = f_t H_{t-1} + (1 - f_t) Z_t.$$

Algorithm 3 makes the displayed scalar-forget version explicit. The row/column/cell external-forget resource classes in Section 4 retain their separately stated broadcasting signatures; this is not a substitution for those quantified definitions.

Algorithm 3 — raw-write nonlinear comparator step.

```
Input: H, fixed W, k, v, scalar forget gate f
1. Z <- tanh(H W + outer(k, v))
2. H_next <- f H + (1 - f) Z
Return: H_next
```

The Emender carry sits inside the bounded delta-correcting update, and E97′s split gates act inside the delta path. M²RNN is *nonlinear-state* by the criterion of Section 2, since H_{t-1} appears inside tanh via $H_{t-1}W$, but the write into H is a raw outer product kv^T rather than a delta correction. A three-row structural comparison shows the shared and distinct ingredients; it is not a causal ablation:

Property	*GDN*	*M²RNN*	*Emender*	*What it shows*
Matrix state	yes	yes	yes	Cannot separate: GDN has it and fails S_5
Temporal nonlinearity on state	no	yes	yes	Cannot separate: M²RNN-CMA stalls at 0.22 on S_5
Delta correction in write	yes	no	yes	Distinguishes delta from raw-write, not GDN from Emender

Table 2: *Structural comparison, not causal elimination.* Matrix state is shared by all three families; temporal state nonlinearity is shared by M²RNN and the Emender; delta correction is shared by GDN and the Emender. The S_5 ordering (0.79 / 0.36 / 0.22, Section 7) motivates testing their combination, but does not isolate every architectural choice.

Two checks support a learning-efficiency reading. M²RNN-CMA reaches only 0.31 on solvable S_3, so non-solvability alone cannot explain its shortfall. At the 8 M shape, the recurrent state has 131,072 scalars versus the $\log_2 6 \approx 2.6$-bit information floor. That headroom rules out a bare state-count shortage, not constrained realizability or optimization failure. The formal counterpart is the matched-signature mixed-key one-step separation (`emender_m2rnn_one_step_resource_separation_embeds`, Section 4), not a trained-model accuracy guarantee. Delta correction distinguishes the Emender from raw-write; nonlinear state distinguishes it from GDN, which also uses a delta correction.

4 Formal Results

A trusted Lean 4 [27] core built on Mathlib [28] carries the formal spine: each result is stated as a theorem and compiled together with a proof of it. The import closure of the `ElmanProofs.PaperCore` module — 17 project source files in the recursive local closure — uses none of Lean's escape hatches: no `sorry`, `admit`, added `axiom`, `opaque` declarations, or `native_decide`. The gate is `formal/lean/scripts/check_paper_core.sh`; compiling `ElmanProofs.PaperCore` additionally checks elaboration of the proofs. The refreshed machine-extracted inventory is committed at `paper/data/lean-trust-gate-theorems.json` (857 named declarations, 723 user-written, 17 files; extraction source commit `e58ae937`), including the split-gated, latching, duality, routing, Lipschitz, and E1H-definition modules. Both the gate and extractor traverse the recursive local imports rather than a selected module allowlist. The checked project sources use Lean and Mathlib as their proof foundation. The project-source gate excludes the axioms and source files of Lean and Mathlib dependencies.

Every result below is named by its exact theorem so a reader can open the source and find it. What is proved here is realizability and representational scope; what a trained model actually learns under SGD is the separate empirical question of Section 6 and Section 7.

The formal spine starts with the compositional-depth duality and routing bridge (set H below), which explain the lens used in Section 7 and Section 4. The remaining sets specify exact lookup, recognizer count, matched-signature update separation, resource cost, and conditional latching. Exact non-equality on a constructed real-valued witness is not a uniform error margin, and table lookup is not recurrent tanh-tracker realizability.

Theorem set A: finite-state ceiling and S_5 tracker

At fixed width and precision the Emender is a finite-state machine, and the S_5 word problem it is tested on is a concrete, fully specified finite-state target.

- *Finite-state ceiling at fixed precision.* At fixed width and precision, every online recognizer of this kind has only finitely many distinct states (`S5Witness.fixed_precision_state_space_finite`). This gives a finite online state space.
- *S_5 word problem.* The target is pinned down exactly in Lean. The group S_5 has 120 elements (`S5Witness.s5_state_count`) and is non-solvable (`S5Witness.s5_not_solvable`); the prefix-product tracker that reads a word of transpositions and reports the running product is a 120-state recognizer (`S5Tracker.recognizer_state_count`) whose state composes correctly as tokens are appended (`S5Tracker.run_append`). A bridge lemma confirms the Lean tracker agrees with the Python evaluation harness on every input (`S5Tracker.pythonRun_eq_tracker_tuple`). The four adjacent transpositions generate all of S_5: the tracker's run map is surjective (`S5Inseparability.run_surjective`, via Mathlib's `Equiv.Perm.mclosure_swap_castSucc_succ`).
- *Capacity lower bound (recognizer floor).* Any finite-state recognizer with a state decoder that reproduces the tracker on every input word needs at least 120 states (`S5Inseparability.capacity_lower_bound_120`); equivalently, no recognizer with fewer than 120 states can realize it under any decoder (`S5Inseparability.small_recognizer_cannot_realize_s5`). The bound has a distinguishing-pair form: any recognizer with too few states admits two words it cannot tell apart that the tracker does distinguish (`S5Inseparability.exists_distinguishing_word_pair`). This is the existential form of the S_5-coset inseparability target: it holds for *any* finite recognizer family, with no architectural assumptions. The recognizer floor is the state-counting lemma required by a future bounded-precision raw-write bound.
- *Lookup-table realization.* The tracker is small enough to write down as a table: it uses exactly $120 \times 4 = 480$ state/input keys (`S5NDMRealization.s5_transition_key_count`), and any finite recognizer admits such an exact lookup-table realization (`S5NDMRealization.exactTransitionMemory_run`).

Theorem set B: generator-table lookup and the recurrent-tracker target

The exact statement of `NDMRealizesS5.emender_realizes_s5_tracker` supplies $d = 12$, four orthonormal keys and one-hot values, queries, a decoder, and $\lambda = 1$. Reading a *preloaded* table retrieves the current generator; decoding it and *externally multiplying a supplied* S_5 state s reproduces the transition $s \cdot g$. The linear readout supplies the generator value, and the external multiplication updates the group state. The supporting `OnlineMemory.linearDeltaWrite_overwrites_one_preserves_others` is an exact *pre*-tanh overwrite statement.

A recurrent realization would carry the running group product in its own state.

Open item O1 — recurrent tanh*-tracker realization unavailable.* The loaded generator memory does not carry the running group product; external composition supplies the missing state

computation. Closing O1 requires a configuration of the actual nonlinear update, an explicit initial state and decoder, and an all-words trajectory theorem. This is a stronger, currently unproved architecture claim, beyond the already-proved lookup identity.

Theorem set C: update-family separation

In a single step the delta-correcting write does something no fixed-weight raw-write update of the matched shape can reproduce on the abstract witness domain.

The Emender's delta-correcting write and M²RNN's raw outer-product write are provably distinct as update families over unrestricted real-valued keys, values, and states. The mixed key is $(1, 1)$ (not unit-normalized), and the prescribed initial state is nonzero. These witnesses are not production zero-state trajectories with normalized keys and finite-logit sigmoid gates, nor full-network separation under arbitrary reparameterization.

- *Mechanism separation.* The delta write exactly overwrites the addressed slot while leaving every orthogonal query untouched (`OnlineMemory.linearDeltaWrite_overwrites_one_preserves_others`), whereas the raw outer-product write cannot satisfy that uniform one-step overwrite specification (`OnlineMemory.rawOuterWrite_not_uniformOneStepOverwrite`).
- *One-step resource separation.* For every $K \geq 2$ and $V \geq 1$, no fixed-weight M²RNN parameterization with row, column, or cell forget gates can match the Emender's mixed-key delta correction in a single recurrent step (`RecurrentResourceFormalism.emender_m2rnn_one_step_resource_separation_embeds`). The result is sharp: it covers every external-forget shape the M²RNN signature allows.
- *Positive embedding.* The separation concerns the specified resource class. *Given* the extra read-then-delta resource, M²RNN can embed one Emender step (`M2RNNComparison.m2rnn_read_then_delta_embeds_e88_delta_update`); the previous bullet says that without that extra resource it cannot.

Theorem set C′: multi-step (k-step) separation

The one-step gap persists at every length under composition on a constructed input.

The one-step separation persists as exact non-equality at each finite length; a length-independent accuracy margin remains open.

> *For every $k \geq 1$, there is an explicit k-token input sequence such that the k-step trajectory of any fixed-weight raw-write resource (in the row/column/cell external-forget class) diverges from the Emender's k-step trajectory by a nonzero entry for each finite k (Lean: `emender_m2rnn_k_step_separation`).*

The witness alphabet is the 2-dimensional construction from set C followed by zero-input filler tokens (`kStepWitnessInputs k = (mixedKey, 0) :: zeroSteps (k - 1)`). It starts from the prescribed nonzero `lowerLeftState`, with unnormalized key $(1, 1)$ and zero-key fillers. Like set C, set C′ is over an unrestricted abstract real-valued signature, not the production normalized-key, zero-initial-state domain. The proof inducts on the tail, using a lemma that keeps entry $(0, 0)$ pinned at zero on the raw-write side for every k (`FixedRightRawExternalForget2_preserves_zero_row`), while the Emender side reduces to the k-fold composition of tanh at -1, which is nonzero because tanh is injective. Its

magnitude need not stay bounded away from zero as k grows; the statement uses exact real arithmetic. A second form of the theorem packages the same statement existentially over k -step input sequences (`emender_m2rnn_k_step_separation_exists`), and the two-step case is exposed separately as the inductive base (`emender_m2rnn_two_step_separation`). All three live in `ElmanProofs.Architectures.MultiStepSeparation`, part of the trusted import closure of `PaperCore`. The Section 7 length-extrapolation curves retain positive, non-monotone gaps on parity, FSM tracking, and modular counter. The parity gap peaks near 0.359 at $T = 320$ before narrowing to about 0.337 at $T = 500$. These measurements are separate evidence of retained advantage, not consequences of the exact-real finite-k theorem or an increasing-gap prediction.

Frontier and unproven targets

The k-step result is the ceiling reachable by direct composition; the existential *form of the S_5 capacity inseparability is now proved (set A), and what remains open is the bounded-precision raw-write form with an explicit $T(d)$.*

The k-step separation above is the *strongest separation reachable from the one-step proof by direct composition.* Its scope is named below, before stating Theorem set D.

- *What is proved.* k-step trajectory separation on a constructed 2D input alphabet, for every $k \geq 1$ and every resource in the row/column/cell external-forget class (`emender_m2rnn_k_step_separation`). The outputs differ at every finite k on this alphabet; no compounding or uniform margin is proved.
- *What is now also proved (set A).* The *existential* capacity form of the S_5 inseparability target: no finite recognizer with fewer than 120 states realizes the tracker, for any architecture (`S5Inseparability.capacity_lower_bound_120`), together with generator surjectivity (`run_surjective`). Two items that earlier drafts of this section listed as open — adjacent-transposition generation of S_5, and a state-counting lower bound — are closed by these theorems.
- *What is still not proved.* That the gap compounds *on the S_5 generator alphabet specifically* against the raw-write resource class with an explicit length bound $T(d)$, i.e., a statement of the form "for any fixed-weight raw-write RNN at state dimension d there is an explicit $T(d)$ past which S_5 coset tracking is unreachable, while the Emender at the same d tracks it". That is the stronger inseparability claim; the trusted Lean core does not contain it, and the empirical sections stand in for it.
- *Why the stronger claim is not in the trusted core.* The remaining form is a *capacity-over-continuous-state* statement: the raw-write class is parameterised by continuous-valued matrices and vectors, so the finite recognizer argument of set A does not directly apply. It requires a bounded-precision discretization (Liu et al.-style fixed-point weights and activations) or a direct trajectory-distinguishability counting argument, plus Merrill-style state-counting machinery in the style of Merrill, Petty and Sabharwal [7]. Mathlib does not currently provide NC^1 / TC^0 circuit classes, finite-state-tracking lower bounds for parameter-bounded RNNs, or pigeonhole capacity arguments specialized to recurrent maps with bounded weight matrices. Each piece is a research-grade mechanization project in its own right; the integration is what would close the gap. The set A bound is the load-bearing state-counting lemma any such argument must land against.

- *Connection to the empirical claim.* The S_5 accuracy curves of Section 7 (Figure 3, Table 6) and the canonical-sweep length-extrapolation curves are the *empirical* evidence for the stronger claim that the formal core does not yet reach. Section 4 proves finite-k non-equality on a constructed witness; Section 7 measures finite-length learning behavior on the S_5 generators, not the stronger $T(d)$ impossibility.

The full status of this push, including what bridging machinery would be required (bounded-precision raw-write class, reachable-state counting, Merrill-style capacity lemma), is documented in `formal/lean/LEAN_FRONTIER.md`.

Theorem set D: synthetic state-pass cost class

The updates share a quadratic envelope in a coarse state-pass accounting model; actual FLOP counts require measurement.

`RecurrentResourceFormalism.emender_m2rnn_flop_class_equiv` bounds the declared six/seven-state-pass charges by a common $c_1 d^2 + c_2 d$ form. This synthetic model excludes the actual dense right-multiplication HW and feature-network costs. An arbitrary dense square HW has cubic arithmetic cost, whereas the model charges one state-sized atom. Thus this theorem does not certify actual matched empirical compute or explain away implementation-level cost differences.

The separately valid E97-versus-E88 result uses the stated *precomputed-gate recurrent-update accounting model*: at matched square state dimension the heads share the same leading d^2 charge (`e97_e88_same_leading_per_head_recurrent_cost`), and with gate applications counted separately, for $d \geq 2$ E97 is bounded by $7d^2$ where E88 is $6d^2$ (`e97_precomputed_gates_same_quadratic_cost_class_as_e88`). This excludes gate/feature-network computation and carries no empirical efficiency claim.

The resource scope of the separation

Set C proves nonrepresentability at the specified update signature; set D supplies only a shared synthetic state-pass envelope. Neither shows how many extra parameters would suffice, or that extra state is the only remedy: `M2RNNComparison.m2rnn_read_then_delta_embeds_e88_delta_update` supplies a different resource, the missing read-then-delta operation. No minimal parameter-count corollary is claimed.

Theorem set E: multi-programming as a structural predicate

A signature-field predicate is shared by E88 and a declared M²RNN proxy; kernel concurrency requires a separate certificate.

The predicate `IsMultiProgrammed` requires four declared fields: at least two heads per layer, positive state scalars per head, a pure recurrent stack, and `sequentialManyProgram` implementation mode. `RecurrentResourceFormalism.multiProgrammed_admits_m2rnn_and_emender` proves it for `e88NDM_1p27B` and `m2rnnCMAShape = m2rnnPure 22 70 256`. That M²RNN instance is a CMA-representative proxy, not the trained 1.3 B predecessor (depth 21, $H = 370$, $N = 16$, hence 256 state scalars/head). The trained geometry meets the same field conditions by inspection/arithmetic. The predicate does not verify kernel scheduling, per-batch independence, or

concurrent execution across layers; its class-level meaning is limited to these declared signature conditions.

Theorem set G: the split-gated update — E97, E88, and GDN-2

Over abstract real-valued gate/key/state signatures, E97 extends E88; GDN-2 has an adjacent scan-compatible linear core with different decay placement. This is not a whole-stack production inclusion theorem for the Section 6 pairing.

The module `SplitGatedDelta` identifies adjacent split-gated linear cores with different decay placement. The identities are over abstract real-valued gate/key/state signatures. The E97/E88 specialization statements below retain their abstract-domain hypotheses.

- *E97 extends E88 in the abstract gate domain.* With both split gates fixed to one, the E97 update specializes exactly to the E88 update, in both direct and expanded form (`e97_specializes_to_e88_all_one_gates_direct`, `e97_specializes_to_e88_all_one_gates_expanded`), so E97 weakly generalizes E88 (`e97_expresses_e88_by_specialization`). The E88 parameterization is therefore a developmental special case, not a rival in this abstract domain. Exact all-one gates are limiting values, not finite-logit production sigmoid outputs.
- *The extension is strict on an abstract concrete witness.* E97 realizes a 2-dimensional split erase/write transition with non-parallel write and erase directions (`e97_realizes_splitWitness_transition`) that *no* E88 coupled transition can realize (`e88_cannot_realize_splitWitness_transition`; the witness directions are non-parallel by `splitWitness_dirs_not_parallel`), and a 1×1 zero-state witness separates the strict-gate E97 update from every all-one-gate E88 setting on the same state/input (`e97_split_gate_strict_witness_not_e88_all_one`). The 2D witness uses key $(1, 1)$ and erase gate $(1, 0)$ and compares transition factors, not arbitrary reparameterized full-network functions. The 1×1 witness uses write gate *2*, outside the production sigmoid range $(0, 1)$; it does not demonstrate production-gate expressivity.
- *Adjacent split-gated cores with different decay placement.* The GDN-2 linear core is the E97 linear core applied to a pre-decayed state (`gdn2LinearCore_eq_e97LinearCore_on_decayed_state`), and under identity decay the two coincide (`gdn2LinearCore_identity_decay_eq_e97LinearCore_one`); the resource signatures then split exactly on the axis the paper tests empirically: E97 has matrix state, full-state temporal nonlinearity, delta writes, and no scan-compatible state path (`e97_split_gated_resource_signature_like_e88`), while the GDN-2-style split erase/write resource has matrix state and delta-style writes but *no* temporal state nonlinearity and a scan-compatible linear state (`gdn2_split_erase_write_is_scan_compatible_linear_state`).

The current pair therefore shares adjacent split-gated delta cores and differs on the nonlinear state path and decay placement, as well as geometry and input processing in the trained stacks. This identifies the formal structural distinction relevant to the iterated-map hypothesis. The `e97` versus `e97-linear` update-map intervention isolates this axis; its post-selected curve comparison has separate selection limitations (Section 7).

Theorem set F: latching

Saturation, hold, and release are described by slot-level bounds.

These three results formalize the latching half of the Emender primitive: saturation makes a slot insensitive to further bounded writes, sign is preserved under sub-threshold counter-input, and a sufficient counter-delta releases the latch. All three are slot-wise statements on the full abstract Emender update, where v denotes the bounded post-nonlinearity value input already supplied to the recurrent body, $S' = \tanh\Big(\lambda S + k(v - S^T k)^T\Big)$ at slot (i, j), with $\omega \coloneqq k_i\Big(v_j - (S^T k)_j\Big)$ the write contribution into that slot. The proofs run through the standard tanh-perturbation foundation. The module carrying them (`EmenderLatching`, introduced at `696299b4`) is included in the refreshed inventory and the current clean `lake build ElmanProofs.PaperCore`.

- *Saturation insensitivity.* If the slot is saturated in the sense $|S'_{i,j}| > 1 - \varepsilon$, and a perturbation δ to the pre-activation entry has $|\delta| \le M$ with $\varepsilon + M < 1$, then the post-tanh slot moves by at most $2(\varepsilon + M)\,|\delta|$. The bound shrinks linearly in both the saturation slack ε and the perturbation magnitude M; at deep saturation a bounded write barely moves the slot. Lean: `EmenderLatching.emender_saturation_insensitivity`. A two-input corollary `EmenderLatching.emender_saturation_two_inputs_close` lifts the bound to two value vectors sharing one key.
- *Sign-preserving hold.* With $\lambda \ge 1$ and a bounded write $|\omega| \le T < \lambda(1 - \eta)$ at slot (i, j), a slot already saturated with $|S_{i,j}| > 1 - \eta$ keeps its sign after one Emender step and stays bounded below in magnitude by $\tanh(\lambda(1 - \eta) - T)$. For $\lambda > 1$ and small η, T the argument $\lambda(1 - \eta) - T$ exceeds 1, so the lower bound exceeds $\tanh 1 \approx 0.76$ and the latch holds with margin. The hypothesis $T < \lambda(1 - \eta)$ is the "no sufficient counter-delta" clause; counter-delta release is the next result. Lean: `EmenderLatching.emender_latch_holds_by_default`.
- *Counter-delta release.* With $\lambda > 0$, $S_{i,j} > 0$, and a write contribution $\omega < -\lambda S_{i,j}$ in the opposite sign, the post-update slot is strictly negative — the latch flips out of its basin in one step. Symmetrically for the negative basin. The release threshold is $T^* \coloneqq \lambda\,|S_{i,j}|$; any counter-delta exceeding T^* in the opposite sign suffices. Lean: `EmenderLatching.emender_latch_releases_on_counter_delta`. The quantitative companion `EmenderLatching.emender_latch_release_quantitative` shows that if the counter-delta exceeds the threshold by margin $\mu > 0$, the post-update slot lies on the opposite side of the origin with magnitude at least $\tanh \mu$.

Theorem set H: compositional-depth duality and routing

The concrete repeated-map family is the schedule of `CompositionalDepthDuality.E97Step` values (k, e, w, v) at fixed λ (Lean field `b` is the paper's erase gate e), acting through `e97Step` on matrix state. `e97Step_eq_tanh_comp` identifies the step as elementwise tanh after the state-affine split-gated core. `e97StateAfter_eq_compose` gives its time-ordered composition, and `e97_compositional_depth_grows_in_time` states that appending a step applies one more such map. Thus the *executed* composition contains one tanh per token. Minimal representation-depth lower bounds remain open.

For a fixed, state-independent schedule of `CompositionalDepthDuality.GdnStepParams`, `gdn_state_channel_depth_bounded_in_time` gives matrices A, C with $S_T = AS_0 + C$ for every initial state (Lean writes H for this matrix). The theorem's state-channel depth is zero under

this fixed, state-independent schedule. Multilayer stacks, state-dependent gates, approximation error, and alternative encodings require separate statements.

`S5GDNBridge.gdn_s5_realization_requires_per_token_action` assumes a token-only schedule and a decoder that recovers `S5Tracker.run` on every word and separates reachable states. It then gives a state map F through the group element, with affine word actions implementing right multiplication on the reachable image. `gdn_s5_constant_parameters_cannot_realize` rules out a token-independent schedule. The bridge identifies the token-dependent affine actions required by the decoder. Section 10 develops the associated realization questions. Resources for generating those parameters remain open in G1–G3 (Section 10).

Complexity statements and explicit non-claims

Lean-proved: finite-state memory at fixed precision (`S5Witness.fixed_precision_state_space_finite`), a non-solvable 120-element group (`s5_not_solvable`, `s5_state_count`), the exact abstract tracker, the recognizer floor (`S5Inseparability.capacity_lower_bound_120`), and the generator-table lookup of set B. Full recurrent tanh-tracker realization is O1, not a theorem. *Cited, not formalized here:* Barrington's NC^1 completeness result [8] and circuit containments for restricted SSMs and transformers under their respective arithmetic assumptions [7], [23], [24]. No strict separation of TC^0 and NC^1 is known or assumed here.

The core establishes the separations and bounds stated above; a families-wide linear-scan S_5 impossibility, Barrington's theorem, a per-token circuit-size/depth lower bound, learned weight recovery of a recurrent tracker, and the raw-write S_5-generator-specific $T(d)$ target remain open targets. Exact finite-k non-equality holds at exact precision. Latching also remains conditional: the hold theorem assumes $\lambda \geq 1$, whereas production decay is below one, a conditional persistence property for the trained E97. These are the boundaries that the empirical and applications sections retain.

5 Multi-Programming and Systems

The formal results establish the mechanism; the next question is whether a serial nonlinear recurrence can be trained to competitive scale at all.

Multi-programming: the throughput-enabling design choice

Width-axis multi-programming supplies throughput while each head's time loop remains serial. A linear recurrence earns its throughput on the *time axis*: because $S_t = A_t S_{t-1} + b_t$ unfolds into a product that depends on the inputs alone, the step-by-step loop can be re-expressed as a prefix scan or a chunkwise matrix multiplication over the whole sequence at once. E97 executes its nonlinear state update in a serial time loop. *Multi-programming* obtains throughput on the other axis by replicating the recurrent computation across many independent heads — each with its own small bounded matrix state — and harvests parallelism across those heads, across state tiles, and across batch elements, while the time loop inside each head still runs one step at a time.

The design choice is a decoupling: it frees throughput from the *shape* of the update. The recurrent body must stay a bounded, register-resident per-step map at a fixed matrix-state signature — that is what keeps each head cheap. This implementation supports a programmable

nonlinear update body. The cost is paid in per-head serial time, most exposed at long sequence lengths where the serial loop is longest; the gain is that a nonlinear recurrence can keep a modern accelerator busy with no time-axis scan at all.

The route is agnostic to the update rule: both pure-nonlinear instances of the predecessor cohort — the Emender and the raw-write M²RNN-CMA — have geometries consistent with the same simple resource predicate. The named Lean theorem `RecurrentResourceFormalism.multiProgrammed_admits_m2rnn_and_emender` checks E88 and a different M²RNN proxy (depth 22, 70 heads, 256 state scalars/head), not the trained depth-21, 370-head M²RNN-CMA. The latter satisfies the four signature-field conditions by inspection/ arithmetic, not by this exact-instance theorem (Section 4).

The 1.3 B Emender shape under multi-programming

A modern GPU exposes many independent streaming multiprocessors, each able to run a small program out of its own registers and shared memory, and multi-programming is built to fill them. E97, the 1.3 B Emender instance, runs 216 such programs per layer per batch element — one per head — so at depth 11 and batch size 4 the layer stack issues $11 \times 216 \times 4 = 9\{,\}504$ small independent recurrent programs per token, each carrying its own 32×32 state tile small enough to live in registers. Keeping the accelerator busy therefore uses parallelism across memory programs. The paired throughput and memory measurements below test the practical consequence at the actual E97/GDN2 geometries; direct E97 occupancy profiling remains to be measured.

Measured throughput and utilization (CUDA only)

All throughput measurements and throughput-budget smokes reported here are CUDA-only; none establishes correctness, performance, or parity on ROCm.

At the actual 1.3 B pair geometries, the matched-batch preflight on seven RTX 6000 Ada GPUs under DDP measures E97-MLP at 3,211 tokens/s per GPU and GDN2 at 3,290, both at batch 4. The ratio is 0.976: comparable throughput, with GDN2 slightly faster. E97 uses 28,942 MB peak memory per GPU against 35,715 MB for GDN2. E97 therefore fits batch 6 under DDP, reaching 4,470 tokens/s per GPU, where the control cannot fit that batch (`experiments/preflight_100b/RESULTS.md`). The lower memory use enables the larger-batch throughput gain. The earlier 1.26–1.56× speedup at small grok-scale dimensions is not reproduced at 1.3 B.

The preflight uses DDP at the actual pair geometries. The long-run training below uses eight independent local-DiLoCo islands. Parameter count, measured throughput, and peak memory are the reported resource comparisons; measuring each implementation's actual FLOPs is a further systems experiment.

Predecessor-cohort occupancy, MFU, and the linear-scan kernel comparison are recorded in Section APPENDIX A.

Fused Triton recurrence kernel

The forward kernel uses an internal `[T, B, H, *]` layout (time, batch, head, ...) and dispatches one program per `(batch, head_block)`. The `BLOCK_H` parameter is autotuned over $\{1, 2, 4, 8, 16\}$ because at $H \geq 256$ per-program-per-head launch overhead dominates. The state tile lives in registers and shared memory for the duration of the T-step loop. The following pieces are fused

into the same Triton program: tanh/silu activations on input projections, L^2 normalization of q, k, the recurrent delta write, and the output gate. Each fusion removes one to two `torch.cuda` launches per layer call; at depth 12 the aggregate saving is approximately 50–60 ms per step.

Sparse-checkpoint backward

The forward kernel saves S only every $K = 16$ steps. The backward kernel processes one K-step segment at a time: it forward-replays the K steps to rebuild per-step S_{t-1} from the saved checkpoint, then walks backward to apply the chain rule. The activation memory used by the kernel is $\frac{T}{K} + 1$ checkpoint slots instead of T. At $K = 16$ this is approximately a $16\times$ shrink, at the cost of a single extra forward pass per segment.

Triton source portability; CUDA-only validation

The recurrence is implemented in Triton [29]. The paired production runs, throughput measurements, and expressivity smokes use NVIDIA CUDA. Porting the pair to ROCm would add a direct test of correctness and performance on that backend. The successor E97 Frontier continuation is a separate single-arm study; paired backend evidence remains pending.

Distributed training

The 1.3 B-class result reported here was produced on a single eight-GPU workstation, with no cluster and no sequence parallelism — both 150 B-token arms of Section 6 ran the same *local DiLoCo* recipe on that one machine: eight independent islands (one per GPU), schedule-free AdamW [30] inside each island, and plain model-weight averaging across islands every $\tau = 250$ local steps (outer learning rate 1.0, momentum 0 — the conservative plain-average form of DiLoCo [31]). Scaling the same recipe past one machine pairs schedule-free AdamW per island with hierarchical local-SGD averaging: each island is one node of 8 GPUs with intra-island DDP, and inter-island synchronization averages model weights on the same $\tau = 250$ cadence. The structural point holds either way: because the Emender distributes work across programs, the 1.3 B implementation uses ordinary head and batch parallelism while preserving each head's time loop.

ParaRNN [6] offers a different systems route, using an iterative solver to parallelize nonlinear recurrence along time. The present implementation instead retains one serial pass per head and obtains its parallelism across heads and batch elements.

In the implementation study, we tried it on the $\tanh(dS + k\delta^T)$ map at 32×32 block size and found it markedly slower in throughput than the multi-programmed Triton kernels above. Newton iteration carries a data-dependent solve count per step, and its convergence on the bounded tanh map is unestablished, whereas multi-programming is a single serial pass per head.

6 Language-Modeling Results

The systems design earns its place if the model can be trained on a substantial language-modeling workload. We compare E97 with a closely parameter-matched GDN2 control under the same long-run recipe. The result is a narrow training-loss band at matched tokens. The following identities, alignment rules, and search records define that comparison.

The matched pair and its training recipe

The 1.3 B comparison is a *pairing*: one nonlinear-state model and one linear-state model, each selected by its own CMA-ES search, trained from scratch with the shared data, tokenizer, optimizer, and local DiLoCo recipe, and read out at matched aggregate tokens. Geometry and learning rate are selected separately for each arm. The two arms are E97 — the Emender split-edit instance of Section 3, paired with a bias-free SwiGLU MLP per layer — and GDN2 — the official-style Gated DeltaNet-2 [4] mixer with the *same* SwiGLU MLP, bound to external source commit `95709fc`. Holding the channel-mixer (MLP) fixed on both arms makes this a comparison of token mixers — the recurrence.

Training identity	*E97-MLP*	*GDN2*
Parameters	1,286,589,072	1,286,713,448 (+0.0097%)
Mixer	Emender split-edit delta (Section 3)	Gated DeltaNet-2, 4-token conv
Geometry	dim 1792, depth 11, $H = 216$, $N = V = 32$, mlp ratio 2.2623	dim 2176, depth 12, $H = 30$, expansion 1, mlp ratio 3.2587
Base LR (CMA-selected)	1.007×10^{-3}	4.743×10^{-4}
Data	The Pile, 2,048-token chunks, `p50k_base`	identical
Optimizer	schedule-free AdamW, bf16, per-rank batch 4	identical
DiLoCo	8 islands (one per GPU), $K = 250$, plain average, outer lr 1.0, $\beta = 0$	identical
Tokens per optimizer step	$8 \times 4 \times 2048 = 65\{,\}536$	identical
Run	`emender_E97_1.3B_20260709_084606`	`gdn2_gdn2-mlp_1.3B_20260722_083444`
Final step	2,300,930	2,323,616
Final aggregate tokens	150,793,748,480	152,280,498,176
Final last-100 loss (nats/token)	2.4365	2.4034
Source .pt checkpoint SHA-256	`0239706e1f67e4823008a3a2754894b5b94dc1663580d2e40c1c74f7dd6a72b2`	`ffae61cfeeeff820f469f7d66dd31c60e48c455872ff9102e57b15ad75bad59b`

Table 3: *Training identities of the 1.3 B pair.* Both arms ran the same launcher harness on the same eight-GPU workstation: same data file, chunk size, tokenizer, optimizer, DiLoCo policy, token cadence, and checkpoint cadence (saves every 500 steps, on merge boundaries); the arms differ only in mixer, geometry, and the CMA-selected learning rate. Sources: launch manifests `scripts/launch_emender_8gpu_diloco.sh` and `scripts/launch_gdn2_mlp_8gpu_diloco.sh`; per-run `args.json`/release validation receipts (`docs/EMENDER_DILOCO_OPS_HANDOFF_20260722.md`, `docs/validation/e97-huggingface-dense-base-and-cli-agent-release.md`, `docs/validation/gdn2-huggingface-152b-control-release.md`).

Both arms consumed 65,536 tokens per optimizer step (8 ranks × batch 4 × chunk 2,048, validated from the launch manifests and log deltas), so matched optimizer steps *are* matched aggregate tokens. GDN2 was run 1.49 B tokens past E97′s final step so that the comparison could be read near E97′s terminal authority. The final common logged read-out is five optimizer steps earlier than that checkpoint; no paired loss statistic uses GDN2′s extra tokens.

Algorithm 4 separates selection, training, and comparison. It summarizes the recorded pair protocol; the selection account below identifies the retained winner configurations and the missing first-generation search records. The comparison reports several averaging windows.

Algorithm 4 — pair selection, training, and loss alignment.

```
Input: per-family search bounds, candidate budget, Pile recipe
1. For each family, run CMA-ES with its candidate budget.
   Train candidates; score the recorded late-loss window.
   Retain the winner geometry and learning rate.
2. Train each winner on the common chunk/tokenizer recipe.
   Each island takes local schedule-free AdamW steps.
   Every 250 steps, average parameters (outer LR 1, beta 0).
   Record rank-0 losses and merge-boundary checkpoints.
3. Deduplicate each loss log by optimizer step.
4. Intersect logged step IDs through the common cutoff.
5. Compute raw and declared trailing-window loss differences.
Return: pair identities, aligned losses, windowed differences
```

Training loss at matched tokens

The primary read-out is the training-log loss of both arms aligned on exact common logged steps through step 2,300,925 (150,793,420,800 tokens): 92,037 aligned records, zero E97-only and zero GDN2-only records through the cutoff, no interpolation. This is 327,680 tokens (five optimizer steps) before E97′s terminal checkpoint at step 2,300,930. The table uses the last common logged step.

Statistic at the 150.79 B-token cutoff	*E97*	*GDN2*	Δ *(GDN2 − E97)*
Raw loss at cutoff step	2.4770	2.4324	−0.0446
80-point trailing smoothed	2.4370	2.4267	−0.0103
Mean, last 100 steps	2.4284	2.4423	+0.0139
Mean, last 1,000 steps	2.4358	2.4273	−0.0085
Early comparison, 5.118 B tokens (smoothed)	2.9300	2.8588	−0.0713

Table 4: *Matched-token loss comparison (nats/token, The Pile).* The common logged cutoff is step 2,300,925, not checkpoint step 2,300,930. At the matched cutoff the two arms sit within 0.010 nats/token smoothed ($\approx$ 0.4% relative), and the *sign of the gap depends on the averaging window*: GDN2 is lower on the full-trajectory and 1,000-step windows, E97 is lower on the most recent 100-step window. Early in training (5.1 B tokens) GDN2 led by 0.071 nats; the gap closed as training progressed. Source: `docs/GDN2_OPS_SNAPSHOT_20260728.md` (final matched cutoff) and `docs/GDN2_E97_MATCHED_LOSS_COMPARISON_20260723.md` (early comparison); both parse rank-0 logs only, deduplicate by optimizer step, and align on exact common steps.

The two current models reach *comparable training loss* after each family's geometry and learning rate have been selected by search. The nonlinear recurrence trains competitively under this substantial shared workload, with a small difference whose sign varies by averaging window. Under the run's recorded byte normalization (`bytes/token` = 3.783 from the frozen `args.json`), E97′s final-window loss of 2.4365 nats/token corresponds to an estimated *0.93 bits per byte* — sub-1-bpb territory for a purely nonlinear recurrence.

Each arm has one training seed. The quantities above come from training logs; held-out tokenizer-invariant bpb and downstream evaluation of the released pair are pending. The carried-in winner configurations and available search records are described below. The window-dependent sign of the Δ in Table 4 describes the observed closeness; statistical equivalence remains unestablished. The retained sweep decomposition in Table 5 keeps the missing winner-series evidence distinct (Open item U3.1).

The convergent-loss null already appears at search scale

The 150 B parity extends a null that is visible much earlier and much more broadly. At the CMA-search stage — five CMA-best 1.3 B models at their own found geometry, one fixed held-out Pile-tail slice of 131,072 scored tokens, and the same 15-minute per-candidate training budget the searches used — all five architectures land in a 0.088-bpb held-out band: pure-E97 2.0126, M²RNN 2.0344, Emender-mix 2.0402, E97-MLP 2.0911, GDN2 2.1013 bits per byte (non-averaged weights, the basis consistent with the search fitness; `experiments/lb_compare_20260613/LEADERBOARD.md`). Within that band E97-MLP is ahead of GDN2 on *both* primary metrics — search fitness 5.8606 vs 5.8949 and held-out bpb 2.0911 vs 2.1013 — but these values show descriptive closeness with the reported ordering; statistical superiority or equivalence remains unestablished: bulk loss occupies a narrow observed band across the tuned families, and the distinguisher must be capability (Section 7), not loss.

Matched-control transparency: search breadth, budgets, and configurations

Because the central 1.3 B claim is a *null* on loss, the credibility of the baseline is load-bearing: the search protocol and records must establish GDN2's optimizer hardening. The provenance chain is as follows.

Search protocol (all families). Each family was searched with the same driver (`scripts/cmaes_search_v2.py`): CMA-ES [10] with population 8 over the family's structural knobs (width, depth, head count, state width, learning rate, batch size), 15 minutes of training per candidate at 2,048-token context on The Pile with `p50k_base` (vocab 50,281), bf16 with fused kernels mandatory (no eager fallback), at a 1,300 M-parameter target with ±3% tolerance. Fitness is the candidate's mean training loss over the 15-minute trajectory; ties are broken by nothing else.

Search breadth. The corrected 1.3 B redo sweep covered *eleven families* with 64–128 completed evaluations each (`docs/E97_RAW_1P3B_LEADERBOARD.md`): `e97-raw` (103 evals, best avg loss 5.9511), `gdn2-mlp` (64, 5.9613), `e97` (109, 5.9733), `e88-linear` (104, 5.9854), `e88` (92, 5.9900), `e88-raw` (106, 6.0390), `e97-linear` (109, 6.0516), `fla-gdn` (109, 6.0854), `m2rnn` (107, 6.1161), mixer-only `gdn2` (96, 6.3850), and `transformer` (128, 6.4606) — every family's best lands within a 0.5-nat band, and the transformer baseline was searched under the same budget rather than copied from the literature. The normalized corpus behind these sweeps — 1,285 candidate trajectories across 13 family panels, 192,147 parsed step-loss rows, all inside the 15-minute candidate window — is committed at `paper/results/cmaes_burst_v2/` (per-family trajectory data and manifest in `cmaes_trajectory_points.csv.gz`, `cmaes_eval_summary.csv`).

The two carried-in winners. E97-MLP carries the emender-mlp search winner (dim 1792, $H = 216$, depth 11, n_state 32, mlp ratio 2.2623, lr 1.007×10^{-3}; search fitness 5.8606, `experiments/lb_compare_20260613/REPRODUCTION.md`), and GDN2 carries the `lb-gdn2-mlp` winner (dim 2176, $H = 30$, depth 12, expansion 1, mlp ratio 3.2587, lr 4.743×10^{-4}; 104 completed evaluations over 13 generations, best fitness 5.8949 at eval 86, `docs/repro/lb_gdn2_mlp_20260612/best.json`). One asymmetry is disclosed: in the earlier redo sweep the `gdn2-mlp` arm had completed only 64 evaluations against the > 96 target, so its standing there was provisional; the committed `lb_gdn2_mlp_20260612` search (104 evals) is the one the 150 B run launched from. M²RNN was searched under the same protocol (fitness 6.1161 at 1.3 B) but is not a 150 B arm (Section 1).

Per-sweep decomposition (U3). For each retained sweep, report the number of candidate trajectories and generation records, the best candidate mean-loss fitness in generation zero, and the best observed fitness across the snapshot. Their difference is the within-sweep search gain, not a causal attribution of improvement to architecture or tuning. Complete, partial, and failed snapshots remain distinct. These earlier retained snapshots are not the later eleven-family leaderboard totals or the two carried-in winner searches.

Sweep ID	*Status*	*Evals*	*Gens*	*Gen-0 best*	*Observed best*	*Gain*
redo 1300m e88	partial	34	7	6.2124	6.1086	0.1038
redo 1300m e97	complete	64	8	5.9980	5.9733	0.0247
redo 1300m e97 linear	complete	64	8	6.2218	6.0967	0.1250
redo 1300m e97 raw	complete	80	10	6.1658	5.9511	0.2146
redo 1300m gdn2	complete	96	12	6.4741	6.3850	0.0891
redo 1300m transformer	partial	60	7	6.8822	6.5673	0.3149
ctx2k e88 delta	complete	128	16	6.1167	5.9974	0.1193
ctx2k e88 raw	complete	136	17	6.1702	6.0395	0.1307
ctx2k fla gdn	complete	160	20	6.3642	6.1104	0.2538
ctx2k m2rnn home xma	complete	144	18	6.2198	6.0626	0.1573
ctx2k mamba2	partial	157	19	6.1704	6.0560	0.1144
ctx2k transformer	complete	160	20	5.9357	5.9046	0.0312
emender e97	failed	1	0	—	6.3260	—
emender gdn2	failed	1	0	—	7.1571	—
Carried-in E97-MLP	U3.1	—	—	—	see prose	—
Carried-in GDN2	U3.1	see prose	see prose	—	see prose	—

Table 5: *Per-sweep CMA audit at the loss-parity binding.* Fitness and gains are nats/token, from retained candidate mean-loss fields, not the final long-run loss. Counts are snapshot coverage, including failed one-evaluation attempts; generation count is retained generation records, not an inferred total. The first 14 rows derive directly from `paper/results/cmaes_burst_v2/cmaes_{eval,generation}_summary.csv` (`899e91cd`), a different snapshot from the later leaderboard. U3.1 applies to the last two rows only, the carried-in E97-MLP and GDN2 searches; they are not mapped to an earlier normalized sweep. GDN2′s retained `docs/repro/lb_gdn2_mlp_20260612/best.json` supplies its count and winner fitness in the provenance prose above, but not first-generation data. Dashes indicate unavailable fields, not zeros or no improvement.

Open item U3.1 — carried-in winner first-generation fitness unavailable; within-sweep gain not computable from retained artifacts. This covers both final carried-in rows. First-generation spread is also unavailable; E97-MLP's candidate/generation totals are not retained in this normalized snapshot. Recover the winner searches' generation logs before computing these quantities; do not estimate them from the earlier sweeps.

One search per family, then one training run per family. No post-search per-arm tuning was applied to either 150 B arm: the launchers freeze the searched geometry and learning rate verbatim (Table 3), so the arms' only degrees of freedom are the ones their own searches selected.

FLOPs-per-bit corroboration at 480 M. The same protocol run at 480 M parameters over four recurrent families — the vanilla nonlinear Elman cell (E1), E88, FLA-GDN, and Mamba2 — shows the same null from the other side: independently tuned, the four families share one FLOPs-per-bit curve over more than two orders of magnitude of training FLOPs (final losses 1.14–1.34 nats at matched FLOP budgets; `paper/results/cma_flop_rate/`). Once tuned, the choice of recurrent family changes *what* is trained, not *how much compute per bit* — consistent with the 1.3 B loss null. Capability and formal results appear in Section 7 and Section 4.

Predecessor-cohort downstream capability panels are recorded in Section APPENDIX A.

7 Expressivity Results

Comparable language-modeling loss leaves open what the carried state can learn to compute. We fine-tune the same 1.3 B E97 and GDN2 checkpoints on two explicit computations. The modular-quadratic task repeatedly applies a nonlinear map to a running value; E97′s advantage supports the mechanism hypothesis that motivates its state update. The S_5 task composes finite-group actions; GDN2′s strong result shows a contrasting learned capability on group actions. Both studies concern learned performance under stated budgets and protocols.

State-count headroom at 8 M parameters

State-tracking probes in this section run at 8 M parameter-matched scale (dim = 384, depth = 4, $H = 32$, $N = 32$, schedule-free AdamW, 10K–20K steps per task, three seeds; GDN uses dim = 640 to match parameter count). The S_5 and S_3 transition tables have information-theoretic floors of $\log_2 120 \approx 6.9$ bits and $\log_2 6 \approx 2.6$ bits; an 8 M-parameter model exceeds those floors by roughly seven orders of magnitude in parameter bits and five in recurrent-state scalars per token. These counts rule out fewer-than-120-state storage as the explanation, not a constrained-map expressivity or optimization bottleneck. A learning-failure interpretation is a hypothesis supported by this headroom and the 1.3 B parallel-decay boundary result; deriving it from the lookup theorem remains Open item O1 (Section 4). The probes measure what SGD finds under the stated no-tuning protocol.

Matched no probe-tuning, and the one selection asymmetry it does not cover

The 8 M probe scale received no probe-specific hyperparameter search and no seed sweep for any family in the comparison. The Emender ran on the default configuration carried down from its 1.3 B stack (the E88-lineage default; the E97 instance of Section 6 postdates these probes); M²RNN-CMA ran on the analogous default from its CMA-tuned reshape; GDN and the M²RNN-paper shape ran on their respective published defaults. Each family is therefore evaluated on the reasonable-defaults configuration it would arrive at without probe-targeted optimization, rather than matched-after-HPO. With large state-count headroom and no probe-specific tuning, the gaps are consistent with an inductive-bias explanation, but do not uniquely identify it.

That matched-no-tuning condition controls for probe-specific *effort*; it does not control for one *selection-criterion* asymmetry. The Emender's defaults are the endpoint of an ablation lineage ranked on language-modeling loss — the same objective GDN's and M²RNN's authors used for their published defaults — with one exception: the state nonlinearity (tanh versus a linear state) was descriptively close to the linear variant on language-modeling loss and was kept on state-tracking grounds. That one component is load-bearing for these probes, so the Emender's configuration was settled partly on a proxy of the very quantity the probes measure, whereas no baseline's was. The asymmetry favors the Emender, and it bears on the *magnitude* of the within-class S_5 gap.

Two observations bound, but do not eliminate, that confound. Raw-write M²RNN under-reaches even on solvable S_3; GDN is stronger on S_3 than S_5. Neither observation proves the cause of

the S_5 shortfall. The recognizer floor is architecture-agnostic, and the lookup theorem is not a recurrent realizability theorem (Section 4, Open item O1). A matched state-tracking search on each family at the 8 M shape would test the inductive-bias hypothesis and the magnitude of the advantage (Section 13).

The S_5 permutation-composition probe

The symmetric-group word problem in S_5 (track the running product of adjacent transpositions, output the 120-way class at each prefix) is the canonical NC^1-complete witness. Random baseline is $\frac{1}{120} = 0.0083$. We train at $T = 128$, evaluate at $T \in \{128, 256, 512, 1024\}$, three seeds.

Model	S_3 *T=128*	S_5 *T=128*	S_5 *T=256*	S_5 *T=512*
Emender	*1.0000*	*0.7918*	*0.4158*	*0.2150*
GDN	0.7185	0.3552	0.1843	0.0974
M²RNN-CMA	0.3124	0.2157	0.1120	0.0593
M²RNN-paper	0.3773	0.1698	0.0884	0.0488
random	0.1667	0.0083	0.0083	0.0083

Table 6: *State-tracking accuracy on the permutation-composition probes.* Mean over three seeds. S_3 is the solvable-group control; S_5 is the non-solvable NC^1 witness. The Emender separates from all three baselines *at training length*, not only under length extrapolation. M²RNN, the head of the raw-write nonlinear matrix RNN family, underperforms both at S_5 training length and at S_3, supporting the mechanism claim that nonlinear matrix state *alone* is not sufficient. Source numbers in `paper/ndmpapernotes.md` lines 153–173.

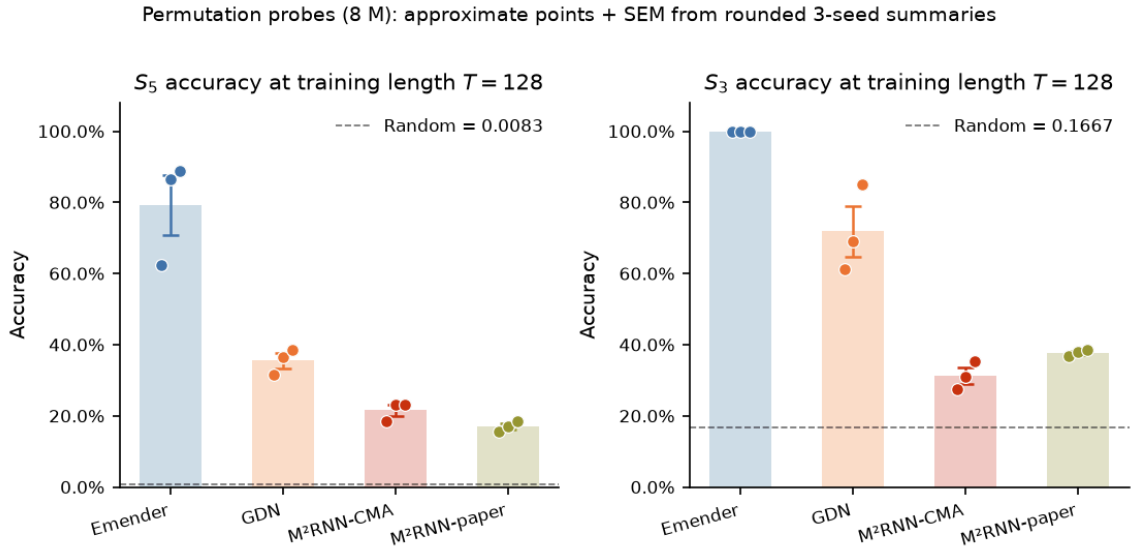


Figure 3: *Expressivity separation on the permutation-composition probes at parameter-matched 8 M scale.* Dots and SEM are approximate reconstructions from rounded three-seed mean/min/max summaries, not retained individual observations. The middle point uses $3 \times \text{mean} - \text{min} - \text{max}$, clamped to the reported min/max interval; SEM is computed from this reconstructed triple. Bars show the reported rounded mean (light fill). The Emender separates from linear-recurrent (GDN) and raw-write nonlinear matrix RNN (M²RNN) baselines on the non-solvable S_5 probe at training length. On the solvable S_3 control, the Emender has rounded mean 1.0000; GDN reaches 0.72; both M²RNN variants stall in the 0.31–0.38 band, indicating that at matched no-tuning budget the raw-write update under-reaches on the prefix-tracking task even on the solvable-group control. Source: `paper/ndmpapernotes.md` lines 153–173; figure script: `paper/figures/plot_expressivity_seeds.py`.

At S_5 training length the Emender reaches 0.79, GDN 0.36, M²RNN-CMA 0.22, M²RNN-paper 0.17. On S_3 the Emender is at 1.00, GDN at 0.72, and both M²RNN variants stall in the 0.31–0.38 band; at matched no-tuning budget the raw-write update under-reaches on the prefix-tracking task even when the group is solvable. The Emender starts at 0.79 at $T = 128$ and declines to 0.42, 0.22, and 0.11 at $T = 256$, 512, 1024; it climbs higher and falls slower than the baselines but does not reach ceiling at length.

The S_3/S_5 split is also where the scope of the M²RNN paper's own state-tracking evaluation matters. Mishra et al. [5], section 3.2 report length generalization on S_3 alone (a nonabelian *solvable* symmetric group, whose word problem is in TC^0) and do not evaluate S_5 or any other non-solvable group. The "perfect state-tracking generalization" framing in that paper does not bear on the NC^1 regime: at parameter-matched 8 M, the Emender reaches 0.79 on S_5 at training length against the paper-default M²RNN's 0.17.

Algorithm 5 gives the common train/evaluate structure of the probe studies, not a claim that they share one recipe. The 8 M from-scratch studies, successful-seed-conditioned grok curves, and 1.3 B group capacity curriculum retain their own seed sets, stopping rules and trained lengths. The modular-quadratic current-pair protocol is specified separately in Algorithm 6; the capacity OOM outcome below is not replaced by a successful curriculum stage.

Algorithm 5 — probe-study train/evaluate loop.

```
Input: arm, seed set, task generator, recorded study recipe
1. For each seed and arm, initialize from scratch or bound weights
   as specified by that study; initialize its optimizer.
2. Generate training batches at the declared trained length(s).
3. Optimize the declared objective until the study's stop rule.
   Record completed stages/updates and any failed stage.
4. For each declared evaluation length, generate held-out batches.
5. Run grad-free forward; count correct predictions on the mask.
6. Save per-seed records; aggregate only the declared seed subset.
Return: accuracy-by-length records and protocol/failure metadata
```

The six-task canonical sweep

Alongside the S_5 probe we run a six-task canonical sweep covering parity (binary XOR over a stream), modular counter (K=5), FSM tracking (K=4 states), Dyck-1 (balanced brackets), associative recall (key→value lookup), and selective copy (mark-and-copy). At 8 M parameter-matched scale, the Emender ties or exceeds GDN on five of six tasks (parity 1.00 vs 0.86; modular counter 0.90 vs 0.65; FSM tracking 1.00 vs 0.83; Dyck-1 1.00 vs 1.00; selective copy 1.00 vs 1.00). GDN is ahead of the Emender on associative recall (0.997 vs 0.881), the only attention-natural task in the suite.

Under length extrapolation (train $T = 40$, evaluate up to $T = 500$), the Emender retains 0.89 accuracy on parity at $T = 500$ where GDN collapses to 0.55 (near random 0.50). On FSM tracking at $T = 500$, the Emender is at 0.59 versus GDN 0.39. The parity gap widens to a peak of about 0.359 at $T = 320$, then narrows to about 0.337 at $T = 500$.

Positive gaps persist across the measured lengths for parity, FSM tracking, and modular counter, but they are not monotone: the FSM gap also peaks at $T = 320$, while the counter gap shrinks throughout. This retained empirical advantage is distinct from the exact finite-k non-equality of Section 4 (`emender_m2rnn_k_step_separation`, Theorem set C′) and the one-step

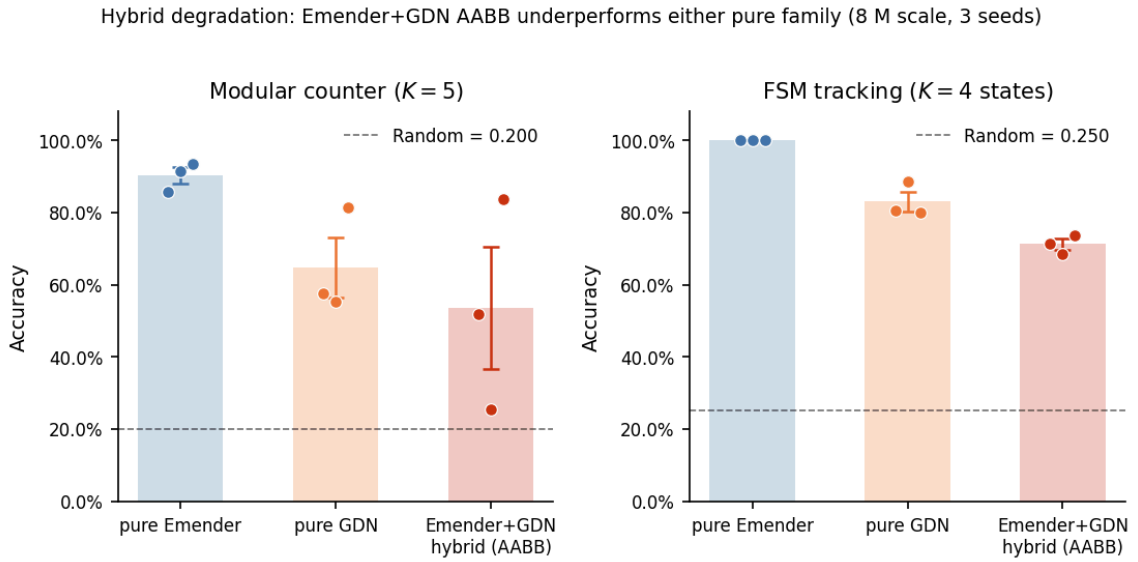


Figure 4: *Hybrid degradation: per-seed accuracy at 8 M scale, 3 seeds per condition.* Interleaving Emender layers with linear-scan (Gated DeltaNet) layers in an `[Emender, Emender, GDN, GDN]` AABB pattern *underperforms* pure Emender on both modular counter ($K = 5$) and FSM tracking ($K = 4$ states), and underperforms pure GDN on modular counter. Bars show the seed mean; error bars span the SEM (standard error of the mean across 3 seeds); individual seed points are overlaid (seeds 42, 123, 456). The dashed line is the random baseline ($\frac{1}{K}$). State-tracking capability is reduced in this tested AABB pairing; Section 12 prediction 5 addresses Emender + attention. Source: per-seed JSON under `paper/results/figure_4_hybrid/`; figure script: `paper/figures/plot_hybrid_degradation.py`. Mean ± std mirrors `experiments/expressivity_tasks/CANONICAL_SWEEP_RESULTS.md`.

resource separation (`emender_m2rnn_one_step_resource_separation_embeds`, set C). Neither formal statement predicts an increasing empirical accuracy gap. The remaining frontier, an S_5 -generator-specific capacity bound with explicit $T(d)$, is named in Section 13.

Hybrid degradation: purity matters

A natural question for any architecture that leads on state tracking is whether the gain survives mixing with linear-scan blocks. We test the pattern [Emender, Emender, GDN, GDN] (four-layer "AABB" hybrid) on the same canonical sweep and find that *hybridization degrades state tracking below either pure family*:

The hybrid result is the same finding as Section 3 from the other side: linear-scan blocks do not inherit state tracking from neighboring Emender blocks.

A controlled update-map ablation on iterated maps

Before measuring the trained 1.3 B pair, the *mechanism* — the per-step state nonlinearity — is tested by a controlled tanh/identity update-map ablation. This does not control every confound in the post-selected seed averages or cross-architecture comparison. On the iterated map $x_t = (x_{t-1}^2 + c_t) \bmod p$ with per-position supervision (train length $T = 128$, evaluate to $T = 4096$), three arms are trained with width applied *symmetrically* across all of them: `e97` (the nonlinear cell), `e97-linear` (byte-identical code on the same fused kernel — the only difference is the per-step state map, tanh versus identity), and `gdn2` (the linear baseline of Section 6′s pairing). Far-length scoring tests transfer beyond the training-length distribution. Length failure

is consistent with finite-length heuristics or memorization, but also with approximate recurrence learning and accumulated error; it does not establish algorithm recovery or its absence.

The nonlinear arm is length-invariant where both linear arms collapse: at $p = 256$, dim 1024, E97 holds $0.982 \to 0.981$ from $T = 128$ to $T = 4096$, while e97-linear falls to 0.619 and GDN2 to 0.560. These source length curves condition on successful training-length seeds (`final_test >= 0.9`): 2 E97, 1 e97-linear, and 4 GDN2 seeds in this cell; the reported mean is conditional on that subset. Separately, an 8-seed confirmation run puts the E97−GDN2 gap at +0.17 to +0.32 at $T = 4096$ across the decisive cells, and the 4-seed symmetric-width sweep shows decisive-cell gaps up to +0.42 (`experiments/grok_symmetric_width/RESULTS.md`, `experiments/grok_symmetric_width/CONFIRM.md`). Because `e97` and `e97-linear` differ by a single flag in the same kernel, the intervention isolates the update map rather than capacity, depth, or kernel choice. The reported successful-seed subsets nevertheless differ, so their conditional curve gap is not an unconditional causal effect estimate or proof of algorithm recovery.

The tested mechanism hypothesis is also probed on two other task families lacking one of its hypothesized conditions; the observed separation is *absent* there. On a contractive logistic map (no long memory to memorize-then-fail), 0 of 24 runs grok and all arms plateau flat across length. On $a^n b^n c^n$ counting (additive accumulation, which linear cumulative state extrapolates as well as bounded nonlinear state), all 24 runs fit and all arms decay together, the gap tiny and sign-flipping. The tested mechanism hypothesis combines repeated non-affine updates with persistent history. The observed negatives support that hypothesis, not a theorem that these conditions are necessary or sufficient for a learned-stack separation. The duality counts the executed updates; it does not lower-bound minimal representation depth (Section 4).

The parameter-matched contractive control

The pre-registered parameter control closes the historical small-GDN mismatch without changing the task or optimization protocol. On the contractive logistic family, `gdn-8m` has 7.94 M parameters versus E97′s 7.38 M (7.6% more, favoring GDN). Its accuracy falls only from 0.896 to 0.854 over the evaluation range (Table 7). The measured mixed-driver Lyapunov exponent is $\lambda = -0.201 < 0$: this is a contractive control, not a chaotic depth-wall vehicle. Larger parameter count improves the linear arm's accuracy but does not change its mild-decay regime; parameters are not the lever for a compounding wall on this family. Protocol: `iterated-map/PLAN.md`, pre-registration `0038ad84`; completed control `384e657c`.

Arm	*Parameters (M)*	*T=128*	*T=256*	*T=512*	*T=1024*
e97	7.38	0.885	0.879	0.878	0.876
e97-linear	7.38	0.893	0.887	0.887	0.885
e97-raw	7.38	0.683	0.677	0.677	0.673
gdn	1.60	0.869	0.857	0.847	0.822
gdn-8m	7.94	0.896	0.887	0.877	0.854

Table 7: *Contractive logistic-map control, from scratch.* Seed-mean accuracy; the historical `gdn` row is parameter-mismatched, whereas `gdn-8m` has 7.6% more parameters than E97, conservatively favoring the linear baseline. All arms use the same training/evaluation protocol and 3 seeds. Source: `phase1_8m_control_table.json` (commit `384e657c`), under `paper/results/expressivity_1p3b/iterated-map/`; protocol pre-registered in `PLAN.md` (`0038ad84`).

Iterating a nonlinear map with the trained pair

The decisive experiment uses the E97 and GDN2 checkpoints of Section 6, with Schedule-Free y-mode recovery, on $x_t = (x_{t-1}^2 + c_t) \bmod 256$, 5 input drivers and running supervision. Both arms receive the identical full-fine-tune recipe: bf16 AdamW(betas=0.9,0.95), constant learning rate 0.00005, warmup 300, weight decay 0.01, gradient clip 0.5, batch 32, trained length 128, seed 42. The throughput-derived common cap of 18269 steps was fixed before accuracy was seen (`budget.json`, `1989656c`). It projects 4.35 GPU-hours if both arms reach the cap. The same pre-registered validation-loss plateau rule stopped E97 after 2500 steps and GDN2 after 2200 steps; actual training elapsed 0.649 GPU-hours, excluding loading and final evaluation. The common rule therefore yields different realized training durations.

The protocol below summarizes the retained driver, `scripts/expressivity_1p3b_depthwall.py`, including its zero-based probe indices. Evaluation uses the weights at each arm's plateau stop. The terminal sweep is grad-free and scores every masked position using full-vocabulary argmax; reduced evaluation batches on OOM are recorded with the results. These are terminal stop weights; best-validation checkpoints are a separate comparison.

Algorithm 6 — matched-cap fine-tune and terminal evaluation.

```
Input: bound y-mode weights; common recipe, cap, seed
1. Build fixed validation batches; initialize AdamW.
2. For step j = 0,...,cap-1:
   a. Draw a batch at T=128; run bf16 forward.
   b. Backpropagate masked cross-entropy over all positions.
      On OOM, split the same batch and accumulate gradients.
   c. Warm up LR for 300 steps, then keep LR constant;
      clip gradient norm to 0.5; take an optimizer step.
   d. Every 100 updates (and at cap), probe validation loss.
      With 4 probes and j >= min(1000,cap), stop if
      loss <= 1e-4 OR both 200/300-step gains are < 1%.
   e. Every 250 updates, record trajectory probes unless stopped.
3. Retain stop weights and actual update count for each arm.
4. For T in {128,256,512,1024,2048,4096,8192}:
   a. Set B=max(1,floor(32768/T)); build 8 seeded batches.
   b. Compute masked accuracy and loss without gradients.
      On OOM, halve B and rebuild with the same length seed.
      If B=1 fails, record an error rather than a score.
Return: training record, stop reason, terminal length sweep
```

Relative gain divides the loss decrease by the earlier loss magnitude (floored at 10^{-12}); both trailing windows must pass. The implemented minimum uses the zero-based index j. Both arms use the stated cap, stopping criterion, and recipe.

Length	*Train-length multiple*	*E97*	*GDN2*
128	1×	99.96%	76.09%
256	2×	99.14%	62.85%
512	4×	90.16%	56.55%
1024	8×	76.07%	53.36%
2048	16×	62.92%	51.91%
4096	32×	55.79%	50.74%
8192	64×	51.79%	50.53%

Table 8: *Modular-quadratic length extrapolation on the released 1.3 B pair.* Full-vocabulary argmax accuracy at the pre-registered plateau stops; trained length 128, seed 42, 262144 scored token positions per length. The identical cap is 18269 steps, not identical realized training: E97 stopped after 2500 steps and GDN2 after 2200. E97 decays gracefully rather than remaining length-invariant; GDN2 approaches a roughly 50% plateau, *not* the declared 0.39% random baseline. Source: `paper/results/expressivity_1p3b/iterated-map/`, `ft_{e97,gdn2}_modular_quadratic{,_sweep}.json` (commit `9dc3afd7`); `budget.json` (pre-registration `1989656c`).

At the terminal sweep E97 reaches 99.96% at trained length, retaining 90.16% at four times and 76.07% at eight times that length. GDN2 ends at 76.09% at trained length and 53.36% at eight times. E97 at eight times approximately matches GDN2′s terminal trained-length accuracy, then decays further. GDN2 approaches a roughly 50% plateau at the longest lengths, above the declared 0.39% random baseline. Intermediate GDN2 trajectory probes had higher accuracy than its terminal checkpoint. These single-seed trajectories motivate repeats that compare fixed training budgets and checkpoint-selection rules. This single-seed, unequal-duration comparison measures terminal accuracy and graceful length transfer; optimization and checkpoint selection remain possible contributors.

This is the empirical complement to the duality theorems `CompositionalDepthDuality.e97_compositional_depth_grows_in_time` and `gdn_state_channel_depth_bounded_in_time` (`32cfcc9c`): E97 composes one nonlinear state update per time step, whereas the input-only-scheduled GDN state fold remains affine in its initial state, with bounded nonlinear temporal depth. The theorem concerns the state channel, not an impossibility or depth lower bound for a multilayer GDN2 stack. The bridge theorem `S5GDNBridge.gdn_s5_realization_requires_per_token_action` (`82eec14d`) assumes a token-only schedule and a reachable-state-separating decoder; under those hypotheses the word action factors through S_5 and is affine on the reachable image; a token-independent schedule cannot track it. It is a *routing* theorem, not a per-token circuit-depth lower bound (`formal/lean/GAPS-D2.md`). The S_5 results below mark exactly that boundary: an affine state channel does not itself imply failure on group actions.

S5 group-state tracking on the same pair

The S_5 word problem tests repeated composition of permutation actions. Finite-group actions admit affine representations, making this a useful contrast with repeated application of the modular-quadratic map.

The initial suite's negative is retained verbatim: "The 8M S5 separation does NOT replicate at 1.3B on this pair." (`paper/results/expressivity_1p3b/SUMMARY.md`, `5e1e60ef`). Zero-shot did

not separate either model; matched fine-tuning favored E97 on the solvable S_3 control but did not reproduce the small-scale S_5 ordering. The longer-horizon capacity probe and phase-0 salvage confirmed the boundary finding, verbatim: "in the capacity-exceeded regime on S5 there is no wall for either model — both decay in parallel toward the random baseline, and GDN2 is ahead of E97 at every length." (`capacity/SUMMARY.md`, `6f916843`).

Length	*E97*	*GDN2*
256	0.397	0.685
512*	0.204	0.363
1024	0.109	0.190
2048	0.061	0.103
4096	0.033	0.054
8192	0.021	0.034

Table 9: *S5 capacity/long-horizon boundary finding, not a depth-wall separation.* Full-vocabulary argmax accuracy, seed 42 both curves decay and GDN2 stays ahead. *GDN2 completed one optimizer step at length 512 before the curriculum stage proved OOM-infeasible even at sub-batch 1; this point is not wholly untrained. Source: `capacity/cap_{e97,gdn2}_s5_sweep.json` and `cap_gdn2_s5_train.json` under `paper/results/expressivity_1p3b/` (E97 `f1ed5615`, GDN2 salvage `6f916843`).

Both capacity curves decline with length; held-out transitions remain weaker than seen ones. GDN2′s attempted length-512 curriculum stage was OOM-infeasible even at sub-batch 1, after one completed optimizer step. The next comparison should use a curriculum feasible for both models and additional seeds. This hardware limitation and the single seed constrain the result.

Interpreting the current-pair evidence

The iterated-map result is the direct empirical test of the motivating mechanism: E97′s nonlinear state path repeatedly transforms the value that will enter the next update. Its measured advantage is consistent with that hypothesis, while optimization, geometry, and checkpoint selection remain possible contributors. The S_5 result demonstrates that GDN2 learns a different carried-state computation strongly. Together with comparable language-modeling loss, these measurements make the useful question concrete: which computations benefit from placing the nonlinearity directly in the temporal state path, and under what training and resource conditions?

Micro and mega: the two probes bracket the mechanism

The expressivity claim rests on two complementary measurements that are load-bearing only *together*. The *micro* probe is the 8 M from-scratch sweep (Figure 3, three seeds): it isolates expressivity in the cleanest setting, where the Section 7 state-count argument gives large storage headroom, so a gap plausibly reflects a difference in inductive bias; raw state count is controlled by that headroom. Its limitation is that an 8 M model is small. The *mega* probe is the completed 1.3 B suite above, on the released E97/GDN2 artifacts the loss-parity claims bind to. The micro probe controls parameter count and seeds; the mega probe controls scale and the trained artifact but supplies only one seed. Together they support a modular-quadratic separation, with graceful rather than invariant E97 extrapolation at 1.3 B, and a negative S_5 transfer. The predecessor cohort's ordering (Section A.4) cannot override this current-pair boundary finding.

8 Repeated nonlinear maps in the wild

Balance, motion tracking, and movement correction offer an intuitive starting point: each new observation revises an estimate that guides the next action. The iterated-map study isolates a related computational question: how well can a recurrent model learn to keep transforming an evolving state? A common description is

$$S_{t+1} = F_{\theta(S_t,u_t)},$$

where θ is fixed across steps, u_t injects the current input, and $F_{\theta(\cdot,u)}$ is generally non-affine in the carried state. A common form is $S_{t+1} = \varphi(A(u_t)S_t + b(u_t))$; neither an additive injection nor a smooth φ is required. Fixed θ describes the update law, not identical token-dependent coefficients: in the Lean schedule, each step supplies its own key, gates, and value. The formal E97 fold fixes λ; the architecture additionally supplies input-dependent decay. Autonomous maps have no input injection. The Emender's coupled update

$$S' = \tanh\Big(\lambda S + k(v - S^T k)^T\Big)$$

is an instance with state S, input (λ, k, v), and a fixed delta-correcting tanh update; it is the all-one split-gate specialization of E97 (Section 3). This is a structural family: the systems below share its update form, while each keeps its own weights, learning rule, and computational difficulty. Algorithm 7 is its time-ordered execution: each returned state is the next step's input. For the abstract E97 fold, the schedule supplies (k, e, w, v) at fixed λ; for GDN it additionally supplies D and is independent of the carried state. The application displays below specify F_θ and the state/input coordinates for each instance of this same loop.

Algorithm 7 — repeated-map fold.

```
Input: initial state S_0, fixed update law F_theta, input schedule
1. S <- S_0
2. For each input u in time order:
   S <- F_theta(S, u)
Return: S
```

The examples below suggest further tests of learned recurrent surrogates. The depth-duality characterizes executed state-channel composition; application-level lower bounds remain open. `CompositionalDepthDuality.e97_compositional_depth_grows_in_time` records one additional nonlinear update per time step; `gdn_state_channel_depth_bounded_in_time` proves that a GDN state fold is affine in its initial state *conditional on a state-independent input/gate/value schedule*. Neither proves that the resulting function has no shallower representation in other coordinates. The S_5 bridge (`S5GDNBridge.gdn_s5_realization_requires_per_token_action`) is a routing statement under its reachable-state-separating decoder hypothesis, not a depth lower bound. It does not transfer a separation to the applications below; group actions can admit affine representations. Here *EXHIBIT* means a definitional or structural instance of the family, *ANALOGY* a related but non-identical mechanism, and *OPEN* a prediction open for future test. In each case the question is whether a fixed-depth GDN-style linear-state approximator can reproduce the required trajectories, not whether the original engineered system already works.

Sigma–delta conversion: quantization inside the loop

EXHIBIT. A first-order sigma–delta modulator accumulates an analog sample and feeds back a quantized output [32]:

$$a_t = r_t + x_t, \quad q_t = Q(a_t), \quad r_{t+1} = a_t - q_t.$$

The residue r_t is the state, x_t is the sample, and the fixed map is $F(r, x) = r + x - Q(r + x)$. For a one-bit converter, $Q(a)$ selects one of two output levels: a hard, discontinuous nonlinearity, distinct from E97′s smooth saturation. Its discontinuous output changes the residue used at the next step. Such streaming error-feedback loops are implemented directly in converter circuitry; they need no large learned processor to execute the update. *OPEN:* the duality suggests testing a GDN surrogate near quantizer thresholds over long streams, where residual errors can change later bits. A nonlinear state update represents this feedback directly; a linear residue integrator with an output-only quantizer omits the feedback dependence. No GDN lower bound follows: finite-resolution states, alternative encodings, or a different error metric may make a linear-state surrogate adequate. Predicting filtered signal quality is also weaker than predicting the exact bitstream.

Adaptive filtering and automatic gain control

ANALOGY for ordinary normalized LMS; EXHIBIT for the nonlinear variants. For regressor x_t, desired sample d_t, and coefficient state w_t, normalized LMS [33] uses

$$e_t = d_t - w_t^T x_t, \quad w_{t+1} = w_t + \mu x_t \frac{e_t}{\varepsilon + \|x_t\|^2}.$$

For fixed inputs this update is affine in w_t. A componentwise coefficient-limited variant, $w_{t+1} = \mathrm{clip}_{B\left(w_t + \mu x_t \frac{e_t}{\varepsilon + \|x_t\|^2}\right)}$, is one: state w, map the normalized correction followed by clipping, injection (x, d). The clipping is an additional engineering choice, an extension of standard normalized LMS. Automatic gain control similarly feeds the measured output level back into the gain [34]. A representative bounded log-gain controller is

$$y_t = \exp(g_t) x_t, \quad g_{t+1} = \mathrm{clip}_{[g_{\min}, g_{\max}]}(g_t + \mu(P_0 - y_t^2)).$$

Its state is g_t, its injection is the sample x_t and target power P_0, and its fixed map includes the state-dependent power measurement and gain limiter. These equations specify representative variants, not universal implementations. *OPEN:* the duality predicts a possible surrogate gap when repeated limiter crossings or gain changes retain consequential history. Ordinary normalized LMS needs no such gap, and a stable AGC loop may forget errors rapidly enough for a linear-state approximation to suffice. These are natural per-sample edge and sensor workloads, but no IoT deployment advantage is measured here.

Proximal iterations and unrolled optimization

EXHIBIT. For the objective $\frac{1}{2}\|Az - b\|^2 + \rho\|z\|_1$, iterative soft-thresholding (ISTA) [35] repeatedly applies

$$z_{j+1} = \mathrm{soft}_{\eta\rho}\big((I - \eta A^T A) z_j + \eta A^T b\big),$$

where $\mathrm{soft}_{\tau(a)} = \mathrm{sign}(a)\max(|a| - \tau, 0)$ acts componentwise. The iterate z_j is the state, the fixed map is a gradient step followed by soft-thresholding, and $\eta A^T b$ injects the problem data at every iteration. Here time means optimization iteration. FISTA [36] adds momentum:

$$z_{j+1} = \mathrm{soft}_{\eta\rho}\big(y_j - \eta A^{T(Ay_j - b)}\big), \quad y_{j+1} = z_{j+1} + \beta_{j(z_{j+1} - z_j)}.$$

Its state must include the previous iterate and momentum schedule (or its clock), yielding a fixed map on that augmented state beyond z_j alone. Unrolling these updates exposes their nonlinear depth explicitly; changing weights at every unrolled stage forfeits the fixed-map identification. *OPEN:* the duality predicts difficulty for a fixed-depth linear-state surrogate when support

changes must be recomputed over many iterations. It does not say ISTA always requires growing nonlinear depth: after support and signs stabilize, soft-thresholding is affine on the visited region, and convergence can make a bounded-accuracy shortcut sufficient.

Reservoirs and echo-state networks

EXHIBIT. The standard driven echo-state network [37] has

$$h_{t+1} = \tanh(Wh_t + Uu_t + b).$$

The reservoir state is h, the fixed recurrent map is tanh after mixing by W, and the injection is $Uu_t + b$. With recurrent parameters fixed, E97 also defines a driven nonlinear dynamical system, with state mixing determined by its input keys and gates; its matrix state passes through tanh at each step. *ANALOGY:* it is not the conventional fixed-W echo-state architecture or training protocol. E97′s effective state mixing depends on the input key and gates, its recurrent weights are trained, and bounded tanh state alone does not establish the echo-state property (forgetting the initial condition). *OPEN:* for a reservoir task with persistent nonlinear history the duality motivates comparing E97 with a linear-state surrogate across length. A contractive reservoir instead has fading memory and may admit a good short-history approximation. The structural match does not predict a universal advantage over linear reservoirs with nonlinear readouts, nor establish that E97 satisfies the echo-state property.

Discrete ecology: the logistic map

EXHIBIT. May's population-dynamics example [38] is the autonomous logistic map

$$n_{t+1} = rn_{t(1-n_t)}.$$

State is normalized population n_t; r is a fixed model parameter; $F_{r(n)} = rn(1-n)$ is applied to its own output with *no* input injection. An environmentally driven variant can supply r_t as the input, but is then an extension of the autonomous model. This is the same logistic family as the contractive control in Section 7, not a claim that its parameter regime represents all ecological dynamics. *OPEN:* the duality motivates a long-horizon surrogate comparison only when nonlinear dependence on prior state remains relevant. The control is a countercase: contraction suppresses trajectory errors and no depth-wall signature is observed. Periodic attraction can also simplify prediction; chaotic regimes can make precise long-horizon prediction difficult for *both* architectures. Nonlinearity alone therefore cannot imply a separation, and an ecological population equation is not evidence of a deployed forecasting improvement.

Feedback control with limited actuators

EXHIBIT. A controller must update its estimate and response while a system moves. A discrete-time plant with a smooth actuator limit and state feedback can be written

$$a_t = a_{\max} \tanh\left(\frac{Kx_t + Lr_t}{a_{\max}}\right), \quad x_{t+1} = Ax_t + Ba_t + Ed_t.$$

Here x_t is plant state; reference r_t and disturbance d_t are the injections; the fixed closed-loop map is $F(x,(r,d)) = Ax + Ba_{\max}\tanh\left(\frac{Kx+Lr}{a_{\max}}\right) + Ed$. Saturation in feedback is a standard nonlinear-control concern [39]; tanh is a smooth bounded-actuator model, not a claim that all actuators implement it. *ANALOGY:* unlike E97, the nonlinearity acts on the control path and the plant state need not be bounded. *OPEN:* the duality predicts a possible linear-state-surrogate deficit when state-dependent saturation repeatedly determines subsequent control decisions. It makes no prediction of a gap in the unsaturated, approximately linear regime, or

for a sufficiently contracting closed loop. A learned surrogate would additionally need stability and safety validation; a state-tracking result is not a control guarantee.

These tasks put the computational idea to work: learn a state update whose output remains consequential over many iterations. When a workload requires persistent, repeated nonlinear transformation of its carried state, the duality says that nonlinear *state computation* cannot simply be dropped while leaving the same update in the same coordinates: an affine state channel does not perform that operation. In that restricted sense nonlinear-state recurrence is not optional; an alternative must supply an adequate representation or approximation elsewhere. Whether a fixed-depth GDN-style stack can do so at acceptable width, precision, latency, and horizon remains *OPEN* for these applications. Stable, fading-memory and affine-representable cases are explicit exceptions to any blanket advantage claim. The practical opportunity is many small, low-cost, low-intelligence streaming processors operating in parallel at the edge: local feedback and correction. Width-axis multi-programming suggests how many such nonlinear programs can coexist without linearizing their internal time loops; low-power silicon, energy savings, and task-level superiority would require separate measurements.

9 Related Work

Ancestry: the fast-weight line on delta correction

The Emender update belongs to a long line that treats memory as a structure updated by prediction error. The earliest formulation is Widrow and Hoff's least-mean-square rule [40], which writes the error between target and current output back into the weight vector of an adaptive linear element. Schmidhuber recast the same idea in a recurrent setting under the name *fast-weight programmers* [41]: a slow controller emits keys and values that update a fast-weight matrix online, with the update being a correction against what the matrix currently predicts at the addressed slot. Schlag, Irie, and Schmidhuber made the bridge to modern attention explicit [42], showing that linear-transformer kernels with cumulative outer-product updates *are* fast-weight programmers with a raw additive write, and proposing the delta-rule write as a stability and capacity fix. DeltaNet [20] carried the delta rule into a parallelizable linear-recurrent language model, demonstrating the rule at billion-parameter scale in the linear-state regime. Emender combines delta correction with a nonlinear matrix-state update and width-axis multi-programming. Its saturating update also motivates the conditional latching analysis in Section 4. The hold result assumes $\lambda \geq 1$ plus a write budget; production persistence remains conditional.

Linear-state recurrent language models

The dominant comparison cohort is the family of linear-state billion-parameter recurrent LMs surveyed in Section 2: Mamba [1] and Mamba2 [2] (selective SSMs); RetNet [43] (decayed linear attention); GLA [19] (gated linear attention); DeltaNet [20] (delta rule, linear in S); Gated DeltaNet [3] (gated delta rule); RWKV-4/5/6/7 [44], [45] (linear-state WKV / generalized delta rule with linear state); HGRN2 [46] (gated linear RNN with state expansion); MinGRU/ MinLSTM [22] (input-only gates, linear scan); mLSTM/xLSTM-7B [47] (covariance update); Griffin/RecurrentGemma [13] (RG-LRU, often hybridized with local attention). Circuit-containment results for restricted SSM families require their stated transition and arithmetic

assumptions [7]. The current empirical comparison uses GDN2, which leads the long-horizon S_5 result in Section 7.

In the broader SSM/sequence-model lineage, S5 [48] introduces simplified state-space layers with a multi-input multi-output linear state-space recurrence. It is distinct from the permutation group S_5 used as a probe here; the cited S5 results concern sequence models, not billion-parameter recurrent LMs.

A newer linear-state result, Gated DeltaNet-2 [4], decouples channel-wise erase and write gates and is reported by its authors to outperform GDN. It extends retrieval and state-tracking behavior inside the linear-state envelope, and it is the linear-recurrent baseline of this paper's 1.3 B pairing: the GDN2 arm of Section 6 is the official-style Gated DeltaNet-2 mixer (external source commit `95709fc`) with the same SwiGLU MLP as E97, selected by the same CMA-ES protocol. The E97 split gates of Section 3 are the Emender's corresponding edit — key-axis erase/read plus value-axis write — inside a *nonlinear* state map, so the two architectures are adjacent by design (Section 4, set G). Its decoupled gates are a natural next ablation for the multi-programmed substrate, as a bounded edit to the per-step update body.

Pure-nonlinear-recurrent peers and adjacent nonlinear-state work

M²RNN [5] (Mishra, Tan, Stoica, Gonzalez, Dao 2026) introduces nonlinear matrix-state recurrence with a raw-write update $Z = \tanh(HW + kv^T)$. It shares the central systems premise of this work — that a nonlinear recurrence is serial in time and must take its parallelism from the batch and head axes — and demonstrates that the update trains at 410 M dense and 7 B MoE scale on Nemotron-CC-v2. Its headline demonstrations use hybrid stacks that interleave attention and nonlinear recurrence. The current study instead evaluates the pure recurrent E97 mixer against a matched GDN2 control. This connects the present work to other nonlinear matrix-state designs.

Our M²RNN-CMA comparator, used as a within-class baseline at the 8 M probes of Section 7 and in the Lean separation of Section 4, is the same raw-write family after CMA-ES has moved it into the stable, pure-recurrent, address-program-multiplexed regime — the *pure-recurrent variant* of M²RNN's architecture with no attention layers, CMA-tuned to restore stable training at 1.3 B. The contribution of this work relative to M²RNN is to extend the viability demonstration to that *pure-recurrent* setting and to add the direct comparison with the Emender under matched per-architecture CMA-ES. In the predecessor-cohort regime (Section A.4) the raw-write model was descriptively close to the Emender on held-out language-modeling bpb but remained behind on S_3/S_5 tracking. The empirical comparison in Section 7 controls the broad pure-recurrent systems setting for comparing the *write rule* — delta-correcting (the Emender) versus raw-write (M²RNN-CMA) — without uniquely isolating a causal mechanism; the formal one-step resource separation in Section 4 applies under its declared accounting model. Mishra et al. also report hybrid M²RNN configurations favorably against Mamba2 and Gated DeltaNet hybrids at matched parameter and token budgets under a uniform fixed-hyperparameter protocol (their section 5.2); those configurations interleave attention layers and so lie outside the pure-nonlinear-recurrent setting this paper studies.

xLSTM-1.3B [25] is a 7:1 mixture of mLSTM (linear) and sLSTM (nonlinear) blocks; with 87.5% of its blocks linear-state, it sits closer to the linear-state class than to the pure-nonlinear-recurrent one. It is the closest scale band among prior nonlinear-recurrent results, included here

as a peer with a small nonlinear-block share. The xLSTM-7B follow-up [47] uses *only* mLSTM (linear) blocks, further from the pure-nonlinear-recurrent class still.

Titans [12] uses an MLP memory with online gradient updates and is qualitatively nonlinear-state, but it interleaves attention layers and has not been evaluated as a pure recurrent model at Pile-class scale. *Zeroth-order LSTM scaling* [49] demonstrates 1 B-scale LSTM training under a non-gradient regime, outside the standard-gradient comparison drawn here.

Classical LSTM/GRU [18] is the historical background: no published classical LSTM/GRU model has reached ≥ 500 M parameters on a Pile-class corpus. It is the unsuccessful empirical record that motivated the field's move to linear-state and hybrid alternatives. This is the assumption this paper revisits.

10 Limitations

O1: realizing the group tracker. Find an E97 update configuration, initial state, and decoder that carry the running S_5 product, then prove the trajectory property over all words.

Formal scope: G1–G3 and O1

G1 — no per-token circuit-size or depth lower bound. `S5GDNBridge.gdn_s5_realization_requires_per_token_action` routes a hypothesized realizer's affine group actions through token-dependent parameters, but a four-symbol schedule may be a shallow lookup. This matters because routing cannot establish that a GDN stack must pay extra network depth or size. Closing it requires first resolving constrained realizability (G3), then an explicit resource lower bound for the parameter-generating network.

G2 — state-dependent gates are unmodeled. `CompositionalDepthDuality.gdn_state_channel_depth_bounded_in_time` and the bridge condition on a state-independent schedule, not gates that read the carried state. This matters because dynamic routing can introduce nonlinearity into that path and invalidate the affine-fold argument. Extend the formal update class to gates that read carried state, and characterize its composition and cost.

G3 — constrained affine/reflection realizability is open. Whether the split-gated decay-minus-rank-one transition can realize a non-solvable representation, including the S_5 action, is not settled; affinity alone is no obstruction. This matters because an affine realizer could track the group with shallow token functions, while an obstruction would be algebraic rather than a temporal-depth theorem. Closing it requires either a concrete representation satisfying the production channel constraints (including allowed decay and gate ranges), or a non-realizability proof for precisely those constraints. The Lean decay parameter is an arbitrary matrix, so the formal family must not be silently identified with scalar-decay production restrictions.

O1 — the nonlinear recurrent tracker is not realized by the lookup artifact. `NDMRealizesS5.emender_realizes_s5_tracker` proves a preloaded pre-tanh generator lookup plus external multiplication of a supplied group state, not all-words execution of the recurrent tanh cell. This matters because the claimed architecture-to-task existence bridge otherwise exceeds the checked theorem. Closing Open item O1 requires an actual update configuration, initialization, decoder, and trajectory proof as specified in Section 4, set B.

These gaps are recorded in `formal/lean/GAPS-D2.md` (G1–G3) and Section 4 (O1). Circuit containment does not establish $\mathrm{TC}^0 \neq \mathrm{NC}^1$ no conclusion here assumes that open complexity separation.

Exact separation versus robust tracking

The k-step witness has distinguishing entry $\tanh^{k(-1)}$ versus zero (`MultiStepSeparation.emender_m2rnn_k_step_separation`), nonzero for each finite k, not uniformly bounded away from zero. This matters because exact real-valued non-equality is weaker than robustness at finite precision, and the witness is not the S_5 generator alphabet. Closing that gap requires quantitative error and precision bounds and a task-specific statement; the raw-write $T(d)$ target is currently a research question, not an empirical result. Likewise the latching hold theorem assumes $\lambda \geq 1$, unlike production decay below one; a production-range persistence theorem and a layer-level lift would close that transfer gap.

Single-seed 1.3 B evidence and protocol boundaries

The current pair supplies one language-model training seed and one seed for each reported computational probe. Repeated seeds, common fixed-step budgets, and comparison of terminal with best-validation checkpoints would measure the stability of the observed gaps. A feasible matched S_5 curriculum would test the contribution of the length-512 memory limit encountered by GDN2.

Both language-modeling arms and the current 1.3 B expressivity suite have one seed; the multi-seed 8 M and grok-scale evidence is not replication of these trajectories. This matters because seed variance and fine-tuning choices can change a terminal gap or its apparent length wall. Closing it requires pre-registered multi-seed repeats on the released pair with confidence intervals, common realized-budget controls, and fixed evaluation grids. The current modular-quadratic comparison shares a cap and plateau rule but realizes 2,500 versus 2,200 steps; intermediate GDN2 accuracy exceeds its terminal value, and its long-length plateau is not chance. The S_5 prediction fails on the pair, and GDN2′s attempted length-512 curriculum is OOM-infeasible after one optimizer step. These facts matter because protocol and hardware limits are alternative explanations; feasible matched curricula and fixed-step controls would test them without rewriting the recorded negative. Large state-count headroom supports, but does not prove, a learning-failure rather than constrained-capacity explanation.

Evaluation and winner-series evidence

Held-out tokenizer-invariant bpb and downstream evaluation are the next language-model measurements. Recovering the original CMA winner logs would also supply generation-zero fitness, spread, and total search effort for the carried-in configurations (U3.1). These records would support a matched-total-budget search comparison.

The 150 B pair has no held-out tokenizer-invariant bpb or downstream panel; estimated 0.93 bpb and the matched-loss null use training logs, not a held-out evaluation or an equivalence test. This matters because generalization and useful capability need not follow training-loss parity. Closing it requires running the released evaluation pipeline on both immutable checkpoints and reporting sample uncertainty without substituting predecessor results.

U3.1 — carried-in winner first-generation fitness unavailable. The retained CMA snapshots do not supply the final E97-MLP/GDN2 winner searches' generation-zero fitness or spread;

E97′s normalized candidate/generation totals are also absent (Section 6). This matters because their within-search improvement cannot be computed or attributed to tuning from the retained rows. Closing it requires the winner searches' original generation logs and identity mapping, not imputation from earlier sweeps. Equal per-candidate rules are not proof of equal total search effort; broader, matched-total-budget searches would test the residual optimization confound.

Untested application predictions

All six applications in Section 8 are structural identifications or analogies, and every E97/GDN surrogate separation is OPEN. This matters because contraction, fixed support, alternative encodings, finite precision, and the chosen error metric can erase an apparent nonlinear-depth advantage. Closing it requires task-specific matched surrogates, long-horizon error and latency measurements, and stability/safety checks for feedback control; energy or IoT deployment claims additionally require hardware measurements. No such experiment is included here.

Scale, architecture selection, and comparators

The 1.3 B pair is a viability demonstration, not a scaling law; the single-arm 513 B-token Frontier continuation has no matched control. This matters because loss parity, memory costs, and probe ordering may change at 3–7 B. Closing it requires a matched pair across scales under one recipe. The raw-write comparator is only an 8 M and predecessor arm, and Emender defaults partly retain tanh on state-tracking grounds; this matters because selection and geometry affect learned ordering. Matched probe-specific searches, a faithful current 1.3 B raw-write arm, and internal E97 ablations would close those comparisons. The AABB hybrid result is one pattern, not a verdict on attention hybrids; a matched hybrid/pure comparison would test that scope. A transformer was searched at short-budget CMA scale but has no matched 150 B-token arm; training one under the paired recipe would close the long-run comparator gap.

Backend and citation verification

A matched 3–7 B pair would test how training loss, memory use, and computational behavior change with scale. Current-scale internal ablations would separate the effects of state nonlinearity, gates, and geometry. Direct E97 occupancy profiling, actual FLOP accounting, and paired ROCm measurements would extend the systems evidence.

The application tests in Section 8 call for long-horizon error and latency measurements in their stated regimes; feedback applications additionally call for stability testing. The bibliography's flagged publication metadata should be resolved during submission preparation.

Performance evidence is CUDA-only, and E88 occupancy does not establish E97 occupancy or ROCm parity. This matters because source portability and shared kernels do not guarantee backend correctness or throughput. Closing it requires paired ROCm correctness and performance receipts at the reported geometry, with direct E97 occupancy measurements. The bibliography audit separately flags entries whose publication metadata could not be independently retrieved; correctness-first submission requires resolving those flags, not treating citation resolution in Typst as literature verification.

11 Conclusion

Emender makes recurrent memory a site of ongoing computation. Each input corrects the stored state and transforms what the next step receives. Running many small heads in parallel makes this sequential computation practical: E97 trains through 150.8 B Pile tokens at 1.3 B parameters and reaches comparable training loss to GDN2. The estimated training-log normalization is 0.93 bpb; the current pair's held-out evaluation is still unavailable (Section 6).

The iterated nonlinear-map experiment connects that practicality to the architectural motivation. E97 learns the recurrence more accurately and retains a larger fraction of its accuracy as sequence length grows. GDN2 learns the distinct S_5 group-state task strongly and leads its long-horizon comparison. The formal state-channel results make the underlying computational distinction precise: repeated nonlinear composition in E97, and an affine GDN fold under a fixed, state-independent schedule.

The negative transfer is: "The 8M S5 separation does NOT replicate at 1.3B on this pair." The modular-quadratic comparison uses one seed, a common cap, unequal realized training, and graceful length transfer (Section 7). Real-valued update-family separation and the 120-state recognizer floor describe representational scope; learning is a separate empirical question. The generator-table lookup leaves recurrent tracker realization open (O1), and affine-channel S_5 realizability remains open (G3).

These findings support studying nonlinear memory as a computational resource. The next step is to test that resource across training budgets, scales, and tasks in which an evolving state must keep being transformed: feedback, iterative optimization, and driven dynamics.

Repeated nonlinear maps in engineering and dynamics (Section 8) define prospective structural applications. Priority, universal nonlinear superiority, and a circuit-class separation lie outside the contribution. This identified Arkheion build and the separate anonymous TMLR build provide journal-specific submission packaging; journal-specific submission packaging remains a separate step from manuscript claims.

Reproducibility. The checkpoints, loading code, and measurement scripts behind the 1.3 B pairing are public: the two immutable per-arm authorities are published as E97 (`https://huggingface.co/spinozans/emender-e97-1.3b`, tag `tokens-150793748480`, the 150.79 B-token checkpoint of `emender_E97_1.3B_20260709_084606`, source .pt checkpoint SHA-256 `0239706e1f67e4823008a3a2754894b5b94dc1663580d2e40c1c74f7dd6a72b2`) and GDN2 (`https://huggingface.co/spinozans/gdn2-mlp-1.3b`, tag `tokens-152280498176`, source .pt checkpoint SHA-256 `ffae61cfeeeff820f469f7d66dd31c60e48c455872ff9102e57b15ad75bad59b`), The hashes above identify the source `.pt` checkpoints in the release provenance. The immutable HF revisions are `119c6623f414aa7fc6e101667d3ad89b7ed6af73` (E97) and `e6087fa6984265404d99c419c095692d840da472` (GDN2). Converted-weight digests are in each revision's `SHA256SUMS`. The releases ship with the per-architecture CMA-ES configurations, the training protocol (launcher manifests in `scripts/launch_emender_8gpu_diloco.sh` and `scripts/launch_gdn2_mlp_8gpu_diloco.sh`), and the Triton multi-programming kernel source in the public `emender` repo; the paper PDF is mirrored at `http://hypervolu.me/~erik/ndm/Garrison_2026_Emender.pdf`. The predecessor-cohort held-out bits-per-byte appendix (Section A.5) is reproducible from the predecessor v0.3 checkpoints: the re-exported y-mode weights ship in the HuggingFace v0.3 trees of that cohort, and the measurement scripts (`scripts/measure_pile_bpb.py`, `scripts/measure_pile_bpb_elman.py`) and sha-verified held-out slice

manifests are in the public `emender` repo; the same pipeline is the one the pending 150 B-pair held-out measurement will run.

The historical 8 M S_3/S_5 means/minima/maxima are retained as four-decimal summaries in `paper/ndmpapernotes.md`, but the exact three-seed records have not been located in the checked local results and archives. Thus Figure 3 shows approximate reconstructed points and SEM using the $3 \times \text{mean} - \text{min} - \text{max}$ midpoint method, clamped to the reported extrema. Rounding can alter that triple and its SEM, especially near the S_3 ceiling; seed identity/order is not recoverable. This limits seed-level statistical reanalysis. Newer S_5 JSONs describe different experiments and are not provenance for this historical figure.

12 Testable Predictions

The results above suggest concrete predictions, most of them testable at the next scale step or under modest additional training. Each is stated as a claim with its scope and the observation that would falsify it, so that later training rounds can settle them.

- *Width-axis multi-programming scales beyond 1.3 B without throughput collapse.* The multi-programmed substrate harvests throughput from many small bounded heads rather than from time-axis linearization; the per-head register/SRAM working set is independent of total parameter count, so the recipe is expected to survive scale-up to the 3–7 B band. Falsified if per-token throughput degrades faster than the parameter count grows, beyond the kernel-arithmetic baseline.
- *The Emender's state-tracking advantage persists at scaled training budgets.* The Section 7 S_5 and S_3 gaps are at the 8 M parameter-matched probe scale under matched no-tuning, and the iterated-map separation is at grok scale. At longer training-token budgets at 1.3 B and beyond, the within-PNR ordering and the Emender-vs-GDN2 gap on state-tracking probes should hold qualitatively — the completed expressivity suite of Section 7 is the first direct test, on the trained 150 B pair. The predicted S_5 ordering already fails on this pair under the tested protocols; the modular-quadratic result supports only a task-dependent version. Further scale/budget persistence is falsified if that map gap closes under a common optimization protocol.
- *Emender fine-tuned on reasoning benchmarks separates from GDN2 on tasks requiring multi-step state-tracking.* The Section 7 QA panels at the predecessor cohort's training budget sit below the threshold where reasoning benchmarks differentiate. After targeted fine-tuning on a reasoning suite (BIG-Bench Hard, ReCLor, FOLIO, multi-step object tracking), the Emender is expected to separate from the linear baseline specifically on items where multi-step prefix tracking is load-bearing (object tracking, formal fallacies, multi-step entailment). Falsified if E97 and GDN2 remain within standard error on state-tracking-shaped subsets after matched fine-tuning.
- *PNR update-rule variants are per-step-body edits under the multi-programmed kernel.* The fused forward/backward Triton kernel (Section 5) is parameterized by the update map and the gating. A different bounded PNR update rule at the same matrix-state signature should be expressible by editing the per-step body, without satisfying an associative-scan or chunkwise-WY compatibility constraint and with the multi-programmed time loop unchanged. Falsified if a representative variant (e.g., decoupled erase/write gates in the

style of Gated DeltaNet-2 [4], or a higher-order delta with two-step memory) requires kernel rewrites beyond the per-step body.

- *Emender combined with hybrid attention should outperform either alone.* The Section 7 hybrid-degradation result (Emender/GDN AABB underperforms either pure family on state tracking at 8 M) is read here as a property of the Emender/GDN pairing specifically. The prediction is that an Emender-with-attention hybrid (Emender layers interleaved with attention layers, in the style of M²RNN's hybrid configuration) outperforms both pure Emender and pure attention on combined state-tracking and long-range mixing benchmarks at matched parameter and token budgets. Falsified if the hybrid underperforms either constituent at matched budget on the combined benchmark.
- *A broadened CMA-ES sweep places the head-count interior optimum above the predecessor's* $H = 370$. The predecessor-cohort searches (Section A.4) arrived at $H = 370$ for E88 after successive bound repositionings against persistent upward pressure from the optimizer. The conjecture is that the same fairness search under broader bounds, subject only to the kernel-occupancy ceiling of the multi-programmed substrate, returns an interior optimum at or above that band on the Emender at 1.3 B (E97′s own search selected $H = 216$ within its configured bounds); $H \sim 1000$ is the next bracket worth testing. Falsified if a broadened CMA-ES sweep under matched per-token compute returns an interior optimum at or below the current selection.

Some of these will falsify; the falsifications are the next round of evidence.

13 Future Work

A matched 3–7 B E97/GDN2 study would test training viability, memory cost, and computational behavior across scale. The feedback, optimization, and driven-dynamics tasks in Section 8 would test utility beyond language. Mathematical targets are a recurrent tanh tracker (O1), constrained affine/reflection realizability (G3), production-range persistence, and finite-precision robustness.

The results above leave several directions open. The first few bear directly on the empirical comparison of Section 6 and Section 7 the formal targets that follow are the open ends of the Lean development in Section 4.

A stronger raw-write baseline

The within-class ordering reported here — the Emender ahead of the raw-write update M²RNN-CMA at the 8 M probes — is conditional on the no-tuning probe protocol, and the raw-write family is not a 1.3 B arm (Section 1). One contrast in the current evidence stays entangled because of that: the gap reflects either a genuine difference in what the two update rules express or the residue of a raw-write configuration that received less effective optimization. The way to separate the two is a stronger raw-write baseline — a matched state-tracking search on the raw-write family at the 8 M probe shape, and a 1.3 B raw-write arm once its kernels run on this stack — which distinguishes the conditional reading ("under the effort used here") from the family-level reading ("under any matched optimization"). This experiment would locate the Emender-versus-raw-write gap, and Section 10 names it as the load-bearing follow-up.

Scale beyond 1.3 B

The matched-token loss parity is measured at the 1.3 B class on The Pile, at a single 30-GPU-day training extent per arm. This is the named scale limitation of Section 10: whether the parity holds at larger parameter counts, and whether the state-tracking separation grows or vanishes with scale, is open. The load-bearing experiment is a matched 3–7 B E97/GDN2 pair under the same recipe; the single-arm E97 513 B-token Frontier continuation that exists cannot answer a paired question, and a corpus move (e.g. to a license-clean stream) would need its own matched pair. Section 12 states the throughput-scaling form of this question as a falsifiable prediction.

Additional seeds and architecture-internal revalidation

Each 1.3 B run is a multi-week trajectory per architecture, so additional seeds at this band are a multi-week investment apiece; the 1.3 B comparison is single-seed per architecture for that reason, and a seed-averaged repeat is open. Several architecture-internal choices in the Emender — the output gate, the state non-linearity (tanh versus linear), and the decay parameterization (simple sigmoid versus Mamba2-style log-space) — tie on loss at small scale; revalidating each at 1.3 B is open.

A capacity bound on the S_5 generators themselves

The k-step separation proved in Section 4 (`emender_m2rnn_k_step_separation`, set C′) shows that for every finite k the gap between the delta-correcting and raw-write updates persists, but it runs on a constructed two-dimensional witness alphabet rather than on the S_5 generators. The natural next-paper target is a capacity bound stated directly on the S_5 generator alphabet: for any fixed-weight raw-write recurrence at state dimension d, an explicit length $T(d)$ past which tracking the S_5 cosets becomes unreachable, while the Emender at the same d still tracks them. That bound needs a bounded-precision raw-write class and Merrill-style state-counting lower bounds not yet available in Mathlib. The Section 7 length-extrapolation curves on parity, finite-state tracking, and the modular counter show positive but non-monotone gaps. For parity the gap peaks near 0.359 at $T = 320$, then narrows to about 0.337 at $T = 500$. These are retained-advantage observations, not evidence of the proposed S_5 capacity bound or of a uniformly widening gap.

A partial order on PNR update rules

The ordering established here, delta-correcting above raw-write, is a noncontainment witness between two matched-signature update families, not a proved strict set-inclusion order on the PNR class (`emender_m2rnn_one_step_resource_separation_embeds`, set C in Section 4). Whether that relation extends to a *partial order* over the broader PNR space — whether antisymmetry and transitivity hold across the full space, under an explicit common resource class — is open. The horizon is the maximal element: which PNR update rule carries the highest one-step expressive power for its compute class. The `RecurrentResourceFormalism`, the Lean signature in which the Section 4 separations are stated, is the tool for climbing that order. More flexible bounded update rules are the natural rungs, among them split erase/write variants in the style of Gated DeltaNet-2 [4]; placing them in the order is open work.

Lifting the latching theorems to the whole layer

The Section 4 latching set establishes the slot-level half of the Emender primitive: three theorems covering saturation insensitivity, default hold under a bounded delta, and release on a sufficient counter-delta. Two formal targets sit on top of it. The first is a formalism refinement — an architecture-level lift that promotes the slot-wise latching theorems to the `RecurrentResourceFormalism.MemorySemantics.latchAttractor` label on the Emender signature in the `RecurrentResourceFormalism`, recasting slot-level behavior as a statement about the matrix-state attractor structure of the whole layer. The second is a Lean-targeted theorem that connects Section 4 to the Section 7 empirics: a basin-survival bridge that takes an S_5 coset in a future recurrent realization (first closing O1) and shows it survives an adversary whose per-slot perturbation budget stays below the Section 4 release threshold, lifting the slot-level guarantees to a coset-level guarantee on the state-tracking task.

References

[1] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv preprint arXiv:2312.00752*, 2024.

[2] T. Dao and A. Gu, "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," *arXiv preprint arXiv:2405.21060*, 2024.

[3] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated Delta Networks: Improving Mamba2 with Delta Rule," *arXiv preprint arXiv:2412.06464*, 2024.

[4] A. Hatamizadeh, Y. Choi, and J. Kautz, "Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention," *arXiv preprint arXiv:2605.22791*, 2026.

[5] M. Mishra, S. Tan, I. Stoica, J. Gonzalez, and T. Dao, "M²RNN: Non-Linear RNNs with Matrix-Valued States for Scalable Language Modeling." 2026.

[6] F. Danieli, P. Rodriguez, M. Sarabia, X. Suau, and L. Zappella, "ParaRNN: Unlocking Parallel Training of Nonlinear RNNs for Large Language Models," *arXiv preprint arXiv:2510.21450*, 2025.

[7] W. Merrill, J. Petty, and A. Sabharwal, "The Illusion of State in State-Space Models," *arXiv preprint arXiv:2404.08819*, 2024.

[8] D. A. M. Barrington, "Bounded-Width Polynomial-Size Branching Programs Recognize Exactly Those Languages in NC^1", in *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing (STOC 1986)*, ACM, 1986, pp. 1–5. doi: 10.1145/12130.12131.

[9] L. Gao *et al.*, "The Pile: An 800GB Dataset of Diverse Text for Language Modeling," *arXiv preprint arXiv:2101.00027*, 2020.

[10] N. Hansen, S. D. Müller, and P. Koumoutsakos, "Reducing the Time Complexity of the Derandomized Evolution Strategy with Covariance Matrix Adaptation (CMA-ES)," *Evolutionary Computation*, vol. 11, no. 1, pp. 1–18, 2003, doi: 10.1162/106365603321828970.

[11] W. Merrill *et al.*, "Olmo Hybrid: From Theory to Practice and Back." [Online]. Available: https://arxiv.org/abs/2604.03444

[12] A. Behrouz, P. Zhong, and V. Mirrokni, "Titans: Learning to Memorize at Test Time," *arXiv preprint arXiv:2501.00663*, 2025.

[13] S. De *et al.*, "Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models," *arXiv preprint arXiv:2402.19427*, 2024.

[14] W.-W. Liao *et al.*, "A draft human pangenome reference," *Nature*, vol. 617, no. 7960, pp. 312–324, 2023, doi: 10.1038/s41586-023-05896-x.

[15] A. Guarracino *et al.*, "Recombination between heterologous human acrocentric chromosomes," *Nature*, vol. 617, no. 7960, pp. 335–343, 2023, doi: 10.1038/s41586-023-05976-y.

[16] E. Garrison *et al.*, "Building pangenome graphs," *Nature Methods*, vol. 21, pp. 2008–2012, 2024, doi: 10.1038/s41592-024-02430-3.

[17] J. L. Elman, "Finding Structure in Time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990, doi: 10.1207/s15516709cog1402_1.

[18] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[19] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim, "Gated Linear Attention Transformers with Hardware-Efficient Training," *arXiv preprint arXiv:2312.06635*, 2023.

[20] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, "Parallelizing Linear Transformers with the Delta Rule over Sequence Length," *arXiv preprint arXiv:2406.06484*, 2024.

[21] A. S. Lahoti *et al.*, "Mamba-3: Improved Sequence Modeling using State Space Principles," in *International Conference on Learning Representations (ICLR)*, 2026.

[22] L. Feng, F. Tung, M. O. Ahmed, Y. Bengio, and H. Hajimirsadeghi, "Were RNNs All We Needed?," *arXiv preprint arXiv:2410.01201*, 2024.

[23] W. Merrill and A. Sabharwal, "A Logic for Expressing Log-Precision Transformers," in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/a48e5877c7bf86a513950ab23b360498-Abstract-Conference.html

[24] D. Chiang, "Transformers in Uniform TC0," *Transactions on Machine Learning Research*, 2025, [Online]. Available: https://openreview.net/forum?id=ZA7D4nQuQF

[25] M. Beck *et al.*, "xLSTM: Extended Long Short-Term Memory," *arXiv preprint arXiv:2405.04517*, 2024.

[26] H. T. Siegelmann and E. D. Sontag, "On the Computational Power of Neural Nets," *Journal of Computer and System Sciences*, vol. 50, no. 1, pp. 132–150, 1995, doi: 10.1006/jcss.1995.1013.

[27] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Automated Deduction – CADE 28*, 2021. doi: 10.1007/978-3-030-79876-5_37.

[28] The Mathlib Community, "Mathlib 4: Library Documentation." [Online]. Available: https://leanprover-community.github.io/mathlib4_docs/

[29] P. Tillet, H. T. Kung, and D. Cox, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," *Proceedings of the 3rd ACM SIGPLAN*

International Workshop on Machine Learning and Programming Languages, pp. 10–19, 2019, doi: 10.1145/3315508.3329973.

[30] A. Defazio, X. A. Yang, H. Mehta, K. Mishchenko, A. Khaled, and A. Cutkosky, "The Road Less Scheduled," *arXiv preprint arXiv:2405.15682*, 2024.

[31] A. Douillard *et al.*, "DiLoCo: Distributed Low-Communication Training of Language Models," *arXiv preprint arXiv:2311.08105*, 2023.

[32] J. C. Candy and G. C. Temes, Eds., *Oversampling Delta-Sigma Data Converters: Theory, Design, and Simulation*. IEEE Press, 1992.

[33] S. Haykin, *Adaptive Filter Theory*, 4th ed. Prentice Hall, 2002.

[34] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Prentice Hall, 2011.

[35] I. Daubechies, M. Defrise, and C. De Mol, "An Iterative Thresholding Algorithm for Linear Inverse Problems with a Sparsity Constraint," *Communications on Pure and Applied Mathematics*, vol. 57, no. 11, pp. 1413–1457, 2004, doi: 10.1002/cpa.20042.

[36] A. Beck and M. Teboulle, "A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems," *SIAM Journal on Imaging Sciences*, vol. 2, no. 1, pp. 183–202, 2009, doi: 10.1137/080716542.

[37] H. Jaeger, "The "Echo State" Approach to Analysing and Training Recurrent Neural Networks," technical report 148, 2001. [Online]. Available: https://www.ai.rug.nl/minds/uploads/EchoStatesTechRep.pdf

[38] R. M. May, "Simple Mathematical Models with Very Complicated Dynamics," *Nature*, vol. 261, pp. 459–467, 1976, doi: 10.1038/261459a0.

[39] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Prentice Hall, 2002.

[40] B. Widrow and M. E. Hoff, "Adaptive Switching Circuits," in *IRE WESCON Convention Record, Part 4*, 1960, pp. 96–104. [Online]. Available: https://isl.stanford.edu/~widrow/papers/c1960adaptiveswitching.pdf

[41] J. Schmidhuber, "Learning to Control Fast-Weight Memories: An Alternative to Dynamic Recurrent Networks," *Neural Computation*, vol. 4, no. 1, pp. 131–139, 1992, doi: 10.1162/neco.1992.4.1.131.

[42] I. Schlag, K. Irie, and J. Schmidhuber, "Linear Transformers Are Secretly Fast Weight Programmers," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, in Proceedings of Machine Learning Research, vol. 139. 2021, pp. 9355–9366.

[43] Y. Sun *et al.*, "Retentive Network: A Successor to Transformer for Large Language Models," *arXiv preprint arXiv:2307.08621*, 2023.

[44] B. Peng and others, "RWKV: Reinventing RNNs for the Transformer Era," *arXiv preprint arXiv:2305.13048*, 2023.

[45] B. Peng and others, "RWKV-7 "Goose" with Expressive Dynamic State Evolution," *arXiv preprint arXiv:2503.14456*, 2025.

[46] Z. Qin *et al.*, "HGRN2: Gated Linear RNNs with State Expansion," *arXiv preprint arXiv:2404.07904*, 2024.

[47] M. Beck and others, "xLSTM-7B: A Recurrent LLM for Fast and Efficient Inference," *arXiv preprint arXiv:2503.13427*, 2025.

[48] J. T. H. Smith, A. Warrington, and S. W. Linderman, "Simplified State Space Layers for Sequence Modeling," *arXiv preprint arXiv:2208.04933*, 2022.

[49] F. Chaubard and M. J. Kochenderfer, "Scaling Recurrent Neural Networks to a Billion Parameters with Zero-Order Optimization," *arXiv preprint arXiv:2505.17852*, 2025.

[50] P. Clark *et al.*, "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge," *arXiv preprint arXiv:1803.05457*, 2018.

[51] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a Machine Really Finish Your Sentence?," in *ACL*, 2019. doi: 10.18653/v1/P19-1472.

[52] J. Welbl, N. F. Liu, and M. Gardner, "Crowdsourcing Multiple Choice Science Questions," in *Proceedings of the 3rd Workshop on Noisy User-generated Text*, 2017. doi: 10.18653/v1/W17-4413.

[53] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, "Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering," in *EMNLP*, 2018. doi: 10.18653/v1/D18-1260.

[54] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions," in *NAACL*, 2019. doi: 10.18653/v1/N19-1300.

[55] M. Suzgun and others, "Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them," *arXiv preprint arXiv:2210.09261*, 2022.

[56] W. Yu, Z. Jiang, Y. Dong, and J. Feng, "ReClor: A Reading Comprehension Dataset Requiring Logical Reasoning," in *ICLR*, 2020.

[57] S. Han and others, "FOLIO: Natural Language Reasoning with First-Order Logic," *arXiv preprint arXiv:2209.00840*, 2022.

APPENDIX A Development history and source register

The project developed through successive recurrent-memory designs, implementation changes, and increasingly controlled comparisons. This register records that process and points to its source material. The empirical study in this paper is the current E97/GDN2 pair; the entries below preserve the development trail.

The source record remains available for provenance:

- *Architecture development.* `paper/main.typ` at base commit `f17b3be20b395cef914cd7034e8691c997164230` records the E63-to-E97 development sequence, ablations, and implementation decisions. It records how the program evolved.
- *Exploratory task and search work.* `experiments/expressivity_tasks/`, `experiments/grok_symmetric_width/`, `paper/ndmpapernotes.md`, and `paper/results/cmaes_burst_v2/` retain development records. These records identify their own protocols and model versions.
- *The present comparison.* The two immutable model releases and launcher manifests in the reproducibility section identify the actual E97 and GDN2 arms. `paper/results/expressivity_1p3b/` contains their computational-probe records. The current-pair tables in this paper read the corresponding result JSONs directly.
- *Formal development.* `formal/lean/` retains the full proof project. The theorem names cited in Section 4 identify the specific statements used here.

The original records remain available in the repository. The present comparison is defined by its own model identities, data, and protocols.

A.1 Predecessor-cohort systems measurements

A Lean 4 definition (`emender_1p27B`) pins the E88 predecessor counts: twelve layers of 370 heads, each with a 32×32 tile, yielding 22,200 independent recurrent programs per token at batch size 5 and $370 \times (32 \times 32)$ recurrent-state scalars per layer.

Driving the 1.273 B E88 predecessor instance (batch 5, context 2048) through its own training path on a free NVIDIA RTX 6000 Ada (48 GB) and sampling `nvidia-smi` once a second over a sustained post-warmup window, the GPU holds *median 100% utilization (mean 99.8%, minimum 96% across 133 samples)* while drawing 97% of its 300 W power cap. The accelerator stays busy on width-axis multi-programming alone, with no time-axis scan. Sustained throughput is *7,492 tokens/s* (mean over steady 50-step windows; median 7,468); a slower sibling GPU sustained 7,277 tokens/s, so the absolute figure is GPU-dependent at the few-percent level. E97 uses the shared E88-derived Triton engine with split-edit inputs, but the E88 occupancy measurement is not an E97 occupancy measurement.

Occupancy and peak-arithmetic utilization measure different quantities. Model-FLOPs utilization (MFU) — computed with the standard $6N$ convention against the card's dense bf16 peak of 364 TFLOPS — is *15.7%*, a lower bound: $6N$ counts only the weight-matmul FLOPs and excludes the recurrence and gate operations, so the true MFU is slightly higher. High occupancy with modest MFU is the expected profile of a model bound by memory bandwidth and the serial recurrence rather than by arithmetic: the GPU is busy nearly all of the time but converts about 16% of its peak FLOPs into useful work.

What that throughput buys on the width axis is best read against a real time-axis kernel. A chunkwise linear-scan baseline — Gated DeltaNet as implemented in the flash-linear-attention

(FLA) library (1.352 B, the CMA-matched shape of the predecessor cohort), run on the same GPU, context, and time budget — sustains 8,248 tokens/s at MFU 18.4% and the same near-100% occupancy. The E88 predecessor reaches *≈ 91% of that linear-scan kernel's tokens/s* (7,492 / 8,248) *without performing the sequential time-axis scan at all.* Width-axis multi-programming therefore recovers linear-scan-class throughput on the width axis. It does not beat the chunked-scan kernel on raw tokens/s — FLA is about 10% faster here, at a slightly larger parameter count and a different batch — so the standing is parity-class.

A.2 Downstream capability panels: a null, on the predecessor cohort

No downstream capability evaluation of the 150 B E97/GDN2 pair has been run yet (TODO-NEEDS-EVIDENCE). What exists is committed evidence from the predecessor 1.3 B racer cohort (E88, FLA-GDN, M²RNN, Mamba2 — Section A.4), and it provides the reference comparison: at $\approx 1M$-step training extents, on a 2,049-item multi-choice fact panel and a 2,048-item reasoning panel (BBH + ReCLor + FOLIO), the four architectures sit within a few percentage points of one another in narrow observed bands (Table 10), without a paired or seed-level equivalence test.

Panel (predecessor cohort)	*E88*	*FLA-GDN*	*M²RNN*	*Mamba2*
Fact panel, 2,049 items (accuracy)	0.345	0.373	0.358	0.353
Reasoning panel, 2,048 items (accuracy)	0.319	0.350	0.336	0.324

Table 10: *Downstream panels at the predecessor cohort's training extent — a null, reported as a null.* All four 1.3 B-class models are within ≈ 3.5 percentage points on both panels; none has crossed the threshold where these benchmarks differentiate architectures. Measured on the predecessor E88-era checkpoints (steps 0.86 M–1.72 M per model); the 150 B pair's downstream evaluation is pending. Source: `paper/results/qa_reasoning/fact_panel_latest.csv`, `paper/results/qa_reasoning/reasoning_panel_latest.csv`.

The 8 M probes of Section 7 are the controlled instrument for what these panels cannot resolve: at the predecessor cohort's training extent, multi-choice QA does not separate the update rules, and the separation the theory predicts is expected in state tracking, not in fact recall — the Section 7 length-extrapolation results and the 1.3 B expressivity suite measure it directly.

A.3 Lineage of the E63 → E88 → E97 experimental program

The architecture presented in Section 3 is the endpoint of a multi-year ablation program, and this appendix records how its 1.3 B instances — E88 first, E97 in the body of the paper — were arrived at: which design choice was settled at which stage, and what each one fixed. The design history documents the measurements behind the choices in Section 3. The milestone labels (E63, E70–E75, E88, E97) are internal codenames marking stages of that program; they carry no meaning beyond their ordering; E88 appears in the body only as the developmental predecessor of E97. The reference implementation of the 1.3 B stack, including its fused Triton forward and backward recurrence kernels, is released publicly (see the availability statement in Section 11).

- *Where the nonlinearity goes (E63).* Motivated by nonlinear recurrent computation [26], this stage selected *where* the nonlinearity is placed: on the matrix state itself, rather than only on the gates or the output. The Emender's tanh on the state S is inherited from this stage.

- *The fused kernel and the gradient-spike fix (E70–E75).* Established the fused Triton kernel pattern — outer product, decay, and tanh computed as a single kernel — and surfaced the failure mode it had to survive: unbounded outer products spike the gradient. The L^2-normalized key write the Emender uses is the fix.
- *The first stable 1.3 B instance (E88).* Stable at the billion-parameter class under schedule-free AdamW. Its load-bearing parameterization choices, each fixed by ablation: log-space Mamba2 decay (with weight-decay exemption); L^2-normalized q, k; SiLU on input projections before normalization; the numerically stable $\tanh(z) = 2\sigma(2z) - 1$ in the fused kernel; output RMSNorm inside the layer removed (-0.10 nats); short convolutions removed (-0.027 nats); no write gate; SiLU output gate.
- *Why the M²RNN baseline is reshaped (M²RNN-CMA).* The paper-default M²RNN shape (dim = 3072, depth = 10, $H = 759$, $N = 16$) diverged at step 8,400 under the training setup of the predecessor cohort (Section A.4), with gradient norms of $\approx 4.2 \times 10^7$. The CMA-tuned reshape (dim = 1920, depth = 21, $H = 370$, $N = 16$) is stable, and it is the M²RNN-CMA baseline used in the 8 M probes of Section 7 and in Section A.4. That the paper shape diverges while the CMA-tuned shape is stable shows that raw-write need not diverge; addressing geometry is a stability hypothesis under the tested width, depth, and head count, which also changed.
- *The split-edit extension (E97).* E97 extends the E88 cell with the two input-dependent split gates of Section 3 — the key-axis erase/read gate and the value-axis write gate — executed by the same fused Triton engine (the shared E88-derived kernel with the `SPLIT_EDIT` inputs), and pairs each mixer with a bias-free SwiGLU MLP so the 1.3 B comparison is MLP-controlled. E97 specializes exactly to E88 when both gates are one (Lean: `SplitGatedDelta.e97_specializes_to_e88_all_one_gates_direct`), so this is an abstract cell specialization, not a production-domain strict-inclusion proof. The E97 150 B-token run of Section 6 (`emender_E97_1.3B_20260709_084606`) is the first instance of the family trained past 100 B tokens.

A.4 The predecessor 1.3 B cohort (E88 era)

Before the E97/GDN2 pairing of Section 6, the same questions were asked of a three-model cohort at the 1.3 B class on The Pile: E88 (1.273 B, the developmental predecessor instance), M²RNN-CMA (1.307 B, the raw-write comparator), and GDN (1.352 B, the linear baseline of that generation), each with its own CMA-ES search and each trained for roughly 22–24 GPU-days to released `v0.3` checkpoints. That study established two results the body still leans on — the descriptive closeness of held-out losses with a contamination control, and the 1.3 B S_5 fine-tune ordering — and it is preserved here unchanged, both for its own evidentiary weight and so the E97-era claims can be read against it. Section references inside this appendix point to the body of the paper; all E88/M²RNN-CMA/GDN numbers in it belong to that cohort, not to the Section 6 pair.

Setup

We train three pure-recurrent language models at the 1.3 B-class parameter band on The Pile [9] with a 2048-token context window, byte-pair encoding (p50k_base, 50,281-vocab), schedule-free AdamW [30], and bf16 mixed precision. The three models and their CMA-tuned shapes are:

Model	*Params*	*Batch*	*Shape*
Emender	1.273 B	5	dim=1664, depth=12, H=370, N=32
M²RNN-CMA	1.307 B	5	dim=1920, depth=21, H=370, N=16
GDN	1.352 B	4	dim=2688, depth=21, exp=2, H=44

Per-architecture CMA-ES protocol

All three 1.3 B-class architectures (Emender, M²RNN-CMA, Gated DeltaNet) received independent CMA-ES [10] hyperparameter and shape search over the same six knobs: width, depth, head count, state width, output gating, learning rate. Each search ran under matched candidate budget: population 16, fixed wall-clock per candidate, identical fitness rule of mean training nats over a fixed late-training window. When CMA-ES drifted onto a configured-range edge, the range was repositioned and search continued, applied identically across the three architectures. The per-family winner shapes in the table above carry into the Section 7 expressivity probes and Section 4 formal analysis unchanged. The concurrent M²RNN paper (Mishra et al. [5], section 5.2) holds width, depth, optimizer, learning rate, weight decay and gradient clipping uniform across compared architectures and varies only the sequence-mixing block, a fair-by-uniformity protocol that does not give each architecture its own best-tuning.

The per-architecture search was adopted to avoid the undisclosed HPO budgets common in nearby work and the contradictory within-class results that follow from them. It also functioned as a measurement. On the Emender, CMA-ES repeatedly pushed head count up against whatever ceiling the bounds set, and each range repositioning revealed the same pressure on the next iteration. The final $H = 370$ is the interior optimum reached after the bounds were placed far enough out that CMA-ES stopped against open ground. A search configured for cross-architecture fairness thus placed its optimum at high head count without being tuned toward it: throughput comes from many small, independently addressed recurrent programs.

Gradient conditioning is a third recipe property

A fourth run, *M²RNN-paper* (the paper-default shape from [5] re-implemented at 1.3 B with dim=3072, depth=10, H=759, N=16), was attempted under the same training setup and *diverged* at step 8,400 with gradient norm $\approx 4.2 \times 10^7$. The CMA-tuned reshape *M²RNN-CMA* (dim=1920, depth=21, H=370, N=16) of the same update family is stable under the same optimizer and the same data; the divergent paper shape is the stability control and is not in the comparison panel. The two configurations differ in width (3072 vs 1920), depth (10 vs 21), head count (759 vs 370), and addressing geometry. Concentrating many value heads behind few q, k projections is a possible gradient-conditioning mechanism, consistent with the observed stability difference, but these runs do not isolate it. A matched single-factor geometry ablation is needed to distinguish that hypothesis from the other changed dimensions; the update-family comparison alone does not establish the cause.

Under the addressing-bottleneck hypothesis, the relevant geometry may be an *addressing* bottleneck rather than a generic head-count effect. The paper-default shape has high *value-head multiplicity* — many value channels, or heads — but routes many of them through a narrower shared q, k addressing interface; in a pure recurrent stack that may concentrate both credit assignment and gradient flow through the shared address projections. The CMA reshape may reduce that bottleneck by moving toward *address-program multiplicity*: more independently addressed recurrent programs, each with its own small state tile, rather than more value

channels behind the same address path. The stable M²RNN-CMA is therefore a relevant raw-write comparator controlling the broad pure-recurrent systems setting, not uniquely isolating the cause of the paper-default shape's divergence or the write rule from all other changed dimensions.

Configuration	*Role in this paper*	*Geometry*	*Outcome*
M²RNN paper shape	Stability control, not main comparator	High value-head multiplicity with shared q, k addressing	Diverges under the pure-recurrent 1.3 B setup
M²RNN-CMA	Raw-write pure-PNR comparator	CMA moves toward independent address-program multiplicity	Stable; reaches the same held-out bpb band
Emender / E88	Delta-correcting pure-PNR model	Independent address programs by construction	Stable; same held-out bpb band; stronger S_3/S_5 state tracking

Table 11: *Three pure-recurrent configurations of the nonlinear matrix-state family.* The paper-default M²RNN shape is a stability control; M²RNN-CMA is the raw-write comparator once CMA-ES has moved it into the address-program-multiplexed regime; the Emender is the delta-correcting model. Outcomes are the descriptive held-out bpb closeness of this section and the S_3/S_5 separation of Section 7.

Loss-vs-wallclock comparison

After about 22–24 GPU-days, E88 reaches 0.973 bpb on The Pile, GDN 0.973, M²RNN-CMA 0.979; all three are sub-1-bpb. These values are read at each model's released v0.3 checkpoint step (E88 1,542,000; GDN 2,031,000; M²RNN-CMA 1,491,000), which are the canonical checkpoints reported throughout. The corresponding 100K-trailing training losses are 2.644 nats/token (E88, step 1,542,000), 2.644 (GDN, step 2,031,000), and 2.660 (M²RNN-CMA, step 1,491,000). Under the training tokenizer (`p50k_base` BPE) on The Pile, mean bytes per token is 3.92 over a 2000-sample sweep at the training context of 2048 tokens (Section A.5), so bpb = nats/token $\times \log_2 \frac{e}{\text{bytes/token}} \approx$ nats/token $\times$ 0.368. These training-loss curves are a training-dynamics diagnostic, not the basis for the architecture comparison: the reported measurement is the held-out bpb (next subsection), where the three models are descriptively close, and the architectures separate on the Section 7 state-tracking probes rather than on bulk language-model bits-per-byte.

Held-out bits-per-byte: the three are descriptively close

The loss-vs-wallclock curve above plots *training-loss* bpb under the fixed canonical bytes/token = 3.92 normalization. To test whether that within-class ordering survives off the training stream, we separately measure *held-out* bpb: each model's real negative log-likelihood, in nats, over a held-out Pile slice it never trained on, divided by the slice's real UTF-8 byte count — tokenizer-invariant, with no 3.92 constant (method in Section A.5). On the canonical 10 MB held-out slice (context 2048, stride 1024), Emender reaches *0.966* held-out bpb, GDN *0.966*, and M²RNN-CMA *0.961* — all sub-1-bpb, and all at or slightly below their respective train-loss figures (0.973 / 0.973 / 0.979). These held-out numbers are measured at the same released v0.3 checkpoint steps as the train-loss figures above (E88 1,542,000; GDN 2,031,000; M²RNN-CMA 1,491,000), so train-loss and held-out now sit on one canonical checkpoint per model; generalization is healthy with no held-out blow-up.

Held-out bpb and train-loss bpb are different objects — different data *and* different byte-normalization — and are not directly comparable. The train-loss ordering does *not* reproduce held-out. Across five independent held-out Pile slices drawn from random deep offsets across the 1.31 TB corpus, the three models sit inside a 0.006-bpb band of mean bpb, narrower than the

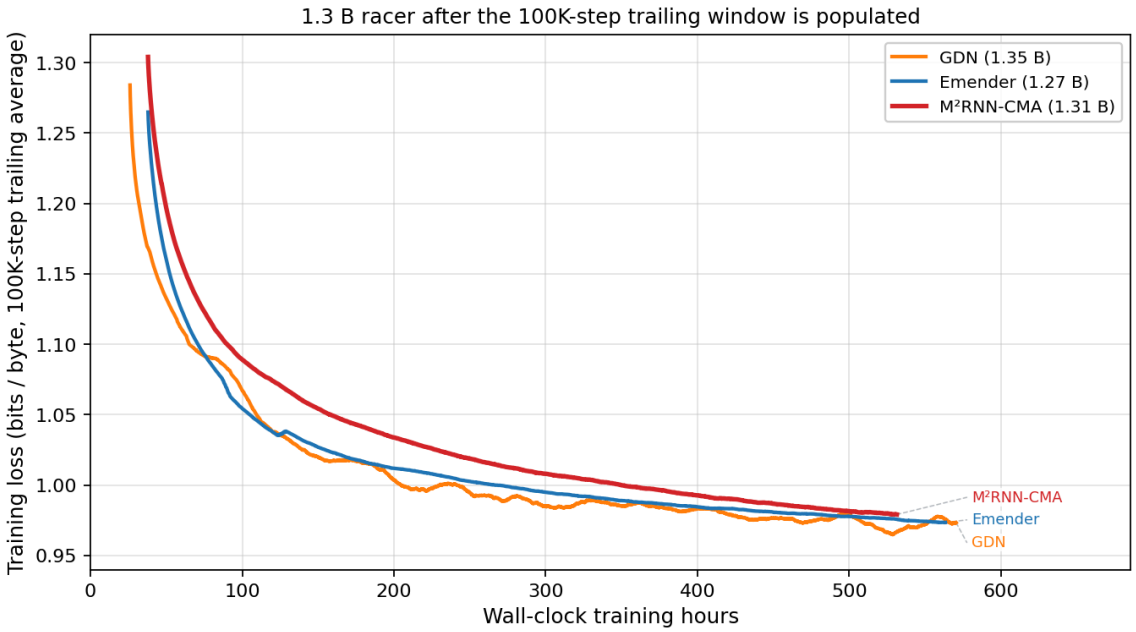


Figure 5: *Training-dynamics diagnostic for the 1.3 B comparison: training-loss-vs-wallclock trajectories on The Pile under matched per-architecture CMA-ES. These curves are a diagnostic of training dynamics, not the basis for the architecture comparison — the reported measurement is the held-out bpb (below). All three trajectories occupy the same sub-1-bpb band; at their released v0.3 checkpoints E88 and GDN are at 0.973 BPB and M²RNN-CMA at 0.979 across the sampled window.* Schedule-free AdamW on The Pile with a 2048-token context. The single panel uses linear wall-clock hours from the start of the run to each model's released v0.3 checkpoint step, but each curve is drawn only after the 100K-step trailing window is fully populated and the curve is inside the plotted BPB range. Curves are 100K-step trailing moving averages of training loss in bits per byte; the nats/token → bits/byte conversion uses the canonical bytes/token = 3.92 for `p50k_base` on The Pile (Section A.5). E88 is at 1.273 B parameters; M²RNN-CMA at 1.307 B; GDN at 1.352 B; the three plotted models have trained about 22.1-23.8 GPU-days at their released v0.3 checkpoints. The plotted trajectory is one realization per architecture. These training-loss trajectories are a diagnostic; the architecture comparison rests on the held-out bpb (below) and the predecessor ablations and CMA-ES sweeps (250+ configs/architecture), not on these curves. The multi-week per-architecture training extent is the standard unit at this scale class. Endpoint values are 100K-step trailing averages read at each model's released v0.3 checkpoint step (E88 step 1,542,000; GDN step 2,031,000; M²RNN-CMA step 1,491,000): E88 0.973 BPB, GDN 0.973 BPB, and M²RNN-CMA 0.979 BPB. E88 and GDN remain in the same sub-1-bpb wallclock band. M²RNN-CMA has higher loss than the other two across the sampled window. The paper-shape M²RNN baseline (not shown) diverged at step 8,400. Color convention used throughout the paper: Emender = blue, GDN = orange, M²RNN-CMA = red. Curves end at each model's released v0.3 checkpoint step — the same checkpoints used for the held-out bpb below; training continued past these points.

largest cross-slice standard deviation (0.025 bpb); the between-model mean gaps are 0.05–0.31× the pooled cross-slice std. On held-out bulk bits-per-byte the three update rules are *descriptively close*, not shown statistically equivalent by a seed-level or paired equivalence test. Which model is lowest is slice-dependent (lowest-bpb counts: Emender 0/5, GDN 1/5, M²RNN-CMA 4/5), so bulk held-out perplexity shows descriptive closeness rather than tested equivalence at this budget. This is the expected reading: the architectures separate on the Section 7 state-tracking probes, not on bulk language-model bits-per-byte. Section A.5 places these numbers in the landscape of open Pile-trained models, out-of-distribution anchors, and classical compressors.

Narrow observed loss bands under matched compute

The narrow observed bands occur under matched compute for each update rule. The three rules — delta-correcting (E88), raw-write (M²RNN-CMA), and linear (GDN) — each received

the *same* per-architecture CMA-ES hyperparameter-and-shape search budget and were then trained to *matched* FLOPs, and they converge to within ≈ 0.005–0.006 bpb of one another on *both* axes: held-out 0.966 / 0.966 / 0.961 and train-loss 0.973 / 0.973 / 0.979. Equal tuning effort and equal compute land three structurally different recurrences in the same narrow loss band.

After each family is tuned to its own best operating point, the converged losses occupy a narrow observed band; this does not establish equal loss across update rules. Where they come apart is the Section 7 state-tracking probes, not the reported bpb bands, and so far that separation shows only on synthetic algebraic state-tracking (S_5/parity/FSM, Section 7) — tasks that *no* model in the cohort solves. The non-synthetic signals — bulk held-out loss, the QA panel, and the reasoning panel (Section 6) — occupy narrow observed bands, without a tested equivalence claim.

The loss figures of this appendix are single-seed per architecture. The five held-out Pile slices (Section A.5) show slice-dependent ordering of the lowest-bpb model and narrow observed bands of between-model means; their cross-slice standard deviations do not measure uncertainty of a paired difference. The Common-Pile contamination control (Section A.5) also shows descriptive closeness on a second distribution. These observations do not establish an architecture-agnostic compute-optimal loss at this matched FLOP budget. The state-tracking results that carry the paper do not rest on these single-seed loss runs: they rest on the 3-seed 8 M expressivity probes (Section 7) and the 1.3 B length-generalization separation (below), where the architectures come apart.

1.3 B fine-tune study on the predecessor cohort

The 8 M probes above have large state-count headroom; an inductive-bias interpretation is plausible but not guaranteed by those counts. The complementary question is whether the separation survives at the 1.3 B scale, in the trained checkpoints themselves. We fine-tune each released v0.3 checkpoint — E88 (delta, 1.273 B), GDN (linear-recurrent, 1.352 B), and M²RNN-CMA (raw-write, 1.307 B) — on the S_3 and S_5 word problems and measure *length generalization*: train on prefixes of length $T \leq 64$, evaluate out to $T = 512$ (8× the longest trained length). The trainable harness is initialized by a strict weight-load of the public v0.3 checkpoint; a full held-out-slice load-sanity check reproduces each model's published readback to $\leq 1.3 \times 10^{-4}$ nats, so the fine-tune is of the released artifact itself.

To keep the comparison about expressivity rather than trainability, each model is first driven *to competence on the solvable tasks*: at the longest trained length $T = 64$, E88 reaches parity 1.000 / S_3 1.000, GDN 0.997 / 0.928, and M²RNN 0.999 / 0.748 (S_3 0.990 at $T = 32$). Under the identical matched recipe (lr 2×10^{-4}, 2500 steps) the raw-write M²RNN under-fits even solvable parity (0.557); reaching solvable competence took a gentler, longer recipe (lr 5×10^{-5}, 12,000 steps) where E88 and GDN saturate the solvable tasks at lr 2×10^{-4} in ≤ 5000 steps. This *effort-to-competence* gap is the matched-FLOP efficiency statement at 1.3 B: raw-write needs more budget to reach the same solvable-task competence the delta update reaches cheaply.

That to-competence procedure removed the *baseline's* trainability confound but left an asymmetry: M²RNN got a gentler, longer recipe to reach competence while E88 never got the same extra budget on S_5, so E88′s S_5 length-extrapolation shortfall (0.162 at $T = 512$) was ambiguous between a real capacity ceiling and mere under-training. The headline run below *fixes the asymmetry*: all three models get the *same* a-priori recipe and the *same*, larger, fixed budget on S_5: M²RNN's exact gentle recipe (lr 5×10^{-5} constant) with the step count *doubled*

to 24,000 (2× M²RNN's to-competence budget), identical length curriculum $T \leq 64$, and *no* S_5-tuning (a single fixed step count, no early-stopping; constant LR so a flat curve cannot be confused with a decayed learning rate). The recipe was chosen and frozen before any S_5 result was seen. Table 12 and Figure 6 report where each model lands.

Model	S_5 *T=64*	S_5 *T=128*	S_5 *T=256*	S_5 *T=512*	S_5 *T=1024*
E88 (delta)	*0.921*	*0.536*	*0.272*	*0.143*	*0.076*
M²RNN (raw-write)	0.658	0.335	0.172	0.090	0.049
GDN (linear)	0.117	0.063	0.035	0.022	0.015
random	0.0083	0.0083	0.0083	0.0083	0.0083

Table 12: *S_5 length-extrapolation at 1.3 B scale, symmetric 24,000-step budget; all three models, one identical a-priori recipe, no S_5-tuning.* Each released `@v0.3` checkpoint is loaded strictly and full-fine-tuned on S_5 alone under a single fixed recipe (lr 5×10^{-5} constant, 24,000 steps = 2× M²RNN's to-competence budget, identical length curriculum $T \leq 64$, seed 42), chosen and frozen before any S_5 result was seen. $T = 64$ is the longest trained length; columns to its right are extrapolation out to 16× ($T = 1024$). Chance is $\frac{1}{120} = 0.0083$. At the trained length E88 (delta) nearly solves the tested prefixes (0.921) while this GDN run under-fits them (0.117, far below competence); raw-write M²RNN reaches 0.658. Under extrapolation all three degrade and E88 > M²RNN > GDN at *every* length, but E88 plateaus below ceiling (0.143 at $T = 512$, flat from $\approx 12,000$ steps) rather than tracking S_5 to arbitrary length. Source: `paper/review/S5_SYMMETRIC_BUDGET.md`; data `paper/review/s5_symmetric_data/{e88,m2rnn,gdn}.json`.

Trained-length competence vs length-extrapolation plateau. Trained-length fit and length extrapolation differ. First, *at the trained length* ($T = 64$) the delta model fits the held-out S_5 prefixes substantially better: E88 reaches 0.921, near-solving a non-solvable task, whereas this linear-recurrent GDN run under-fits it even at the trained length (0.117, far below competence) and raw-write M²RNN reaches 0.658. Thus the delta update nearly solves the tested trained-length prefixes where this GDN run under-fits them; this is not an acquisition impossibility. Second, *under length-extrapolation* (out to $T = 1024$, 16× the trained length) all three degrade; E88 degrades the slowest and stays strictly ahead at *every* length, but it *plateaus below ceiling* (0.143 at $T = 512$, 0.076 at $T = 1024$). This is a measured length-generalization plateau, not a proved capacity limit or expressive impossibility: the $T = 512$ curve is flat from $\approx 12,000$ steps at 2× M²RNN's budget under constant LR (see *The delta update is also length-bounded* below). The accurate statement is that the delta update learns S_5 at the trained length and leads at every length on that S_5 task, with its S_5 length-extrapolation plateauing below ceiling at this width.

The historical GDN run plateaus. GDN reaches 0.117 at trained length, 0.022 at $T = 512$, and 0.015 at $T = 1024$, near chance, with a flat curve by about step 6,000. The same run is stronger on parity and S_3. These are protocol-specific observations, not proof of a non-solvable computation barrier. The GDN2 reversal in Section 7 and open constrained realizability question G3 prevent a families-wide interpretation.

Delta vs raw-write: a budget-robust ordering, not solve-vs-fail. The symmetric run removes the recipe asymmetry directly. E88 received the *same* doubled budget M²RNN got, on the *same* gentle recipe, with *no* S_5-tuning, and still plateaus, so both confounds (M²RNN's trainability deficit and E88′s never-having-had-the-extra-budget) are bought out symmetrically. Under that symmetric budget raw-write M²RNN fits the trained length less well than the delta model (0.658 vs 0.921) and stays below it at every extrapolation length (0.090 vs 0.143 at $T = 512$;

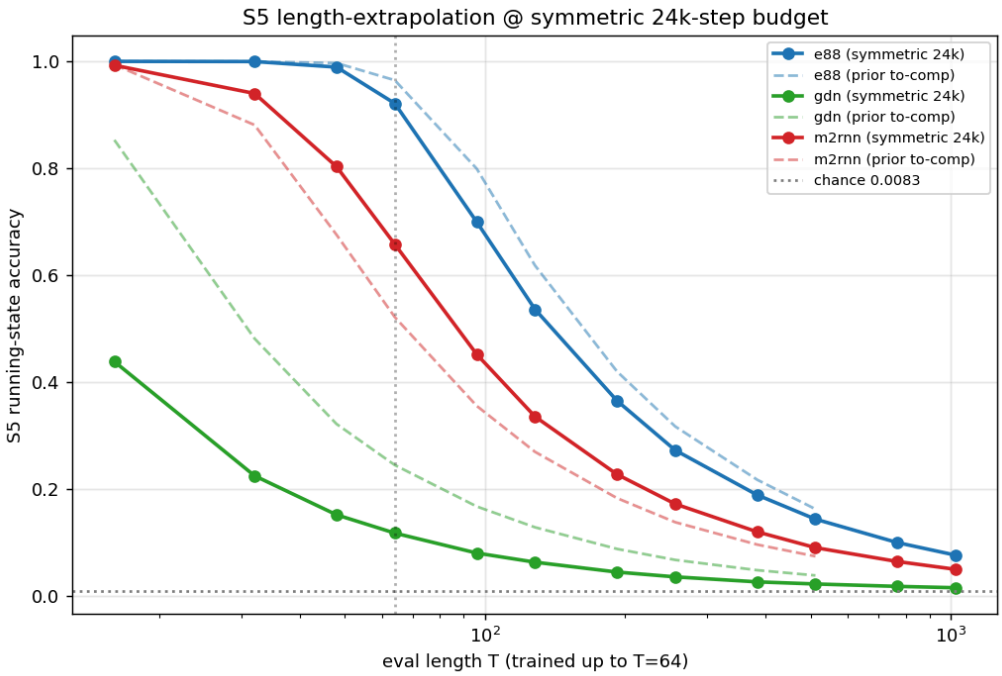


Figure 6: *S_5 length-extrapolation at the symmetric 24,000-step budget.* Running-state accuracy vs sequence length T (log axis) for the three released 1.3 B architectures, each fine-tuned on S_5 under the *same* a-priori recipe (lr 5×10^{-5} constant, 24,000 steps, no S_5-tuning): solid curves are this symmetric run (E88 blue, M²RNN red, GDN green); faint dashed curves are the prior per-model to-competence run, shown for contrast. Trained on $T \leq 64$ (vertical marker), evaluated out to $T = 1024$ (16× the longest trained length); chance is the dotted line ($\frac{1}{120}$). At the trained length the delta model nearly solves S_5 while this GDN run under-fits it; under extrapolation E88 > M²RNN > GDN at *every* length, with E88 plateauing below ceiling rather than tracking S_5 to arbitrary length. Numbers match Table 12. Real data: `paper/review/s5_symmetric_data/{e88,m2rnn,gdn}.json`; figure generated by `scripts/finetune_s5_symmetric.py` / `scripts/analyze_s5_symmetric.py`.

0.049 vs 0.076 at $T = 1024$), tracking the linear GDN's collapse shape far more than the delta model's. The ordering E88 > M²RNN > GDN holds at *every* length out to 16× and is robust to a large budget swing: the matched 2,500-step (lr 2×10^{-4}), the per-model to-competence, and this symmetric 24,000-step (lr 5×10^{-5}) runs all land E88 ≈ 0.14 at $T = 512$ strictly ahead of M²RNN ≈ 0.09. The claim is therefore one of *length-extrapolation efficiency*, not can/can't: at matched (indeed doubled) budget the delta update reaches strictly further with length than raw-write, and both *nonlinear* updates plateau below ceiling as length grows.

The delta update also plateaus under these recipes. E88's $T = 512$ accuracy is flat from about 12,000 steps under constant LR, and the three recipe variants converge near 0.14. This motivates studying length generalization; it does not isolate capacity from optimization. A late-rate extrapolation suggesting only about 0.03 further gain is not a measured doubled-budget result. The supported statement is that neither tested nonlinear arm reaches ceiling at extrapolated length, while E88 remains ahead under the recorded budgets.

Generalization beyond seen sequences. Evaluation words are held out and test-disjoint; the generator alphabet gives exponentially many possible words. Above-chance accuracy out to $T = 1024$ supports learning transferable structure rather than memorizing the enumerated training examples; exact algorithm recovery and finite-context heuristics remain open. The solvable controls are useful diagnostics for the non-solvability difference.

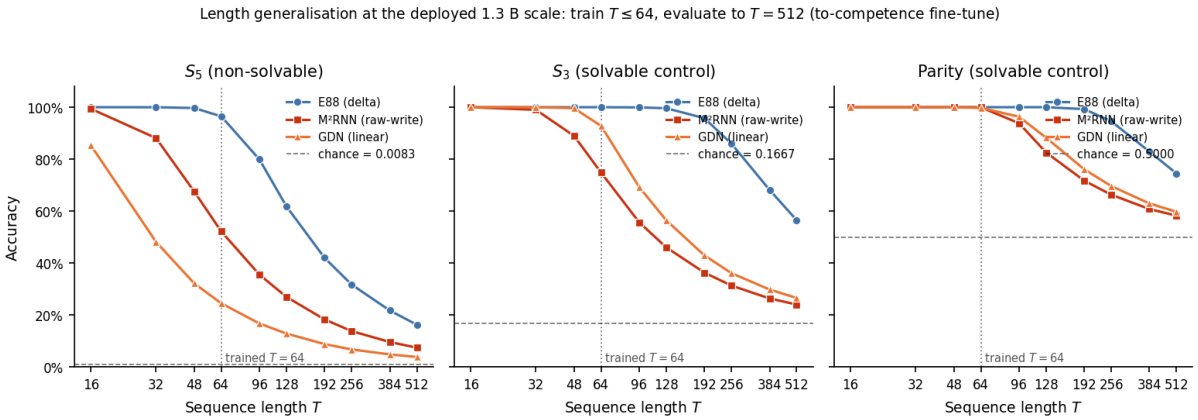


Figure 7: *Solvable controls and the prior to-competence S_5 cross-check at 1.3 B scale.* Accuracy vs sequence length T for the three released architectures on the non-solvable S_5 probe (left) and the solvable S_3 and parity controls (center, right). All three are fine-tuned on $T \leq 64$ (dotted marker) and evaluated out to $T = 512$ — 8× the longest trained length; chance is the dashed line ($\frac{1}{120}$, $\frac{1}{6}$, $\frac{1}{2}$). Colors follow the paper convention (E88 blue, M²RNN-CMA red, GDN orange). The S_5 panel here is the earlier per-model *to-competence* run (the headline symmetric-budget S_5 result is Table 12 / Figure 6); it is retained as a cross-check and reproduces the same E88 (delta) > M²RNN (raw-write) > GDN (linear) ordering at *every* length under a different recipe and budget: E88 degrades the most slowly (0.162 at $T = 512$, far from solving), the linear GDN converges toward the chance floor, and raw-write tracks GDN's collapse shape. The ordering is consistent across both runs, but all three degrade with length and none tracks S_5 to arbitrary length. On the solvable controls all three are competent at the trained length and degrade only with extrapolation, motivating a structure-sensitive learning hypothesis without proving non-solvability causes the gap. Real data: `paper/review/s3_s5_finetune_v03_data_tocomp/`; figure script: `paper/figures/plot_s5_s3_1p3b_lengthgen.py`.

Raw-write's shortfall is partly trainability. M²RNN's S_5 (and S_3) accuracy was still slowly rising at 12,000 steps — its curve is not the flat, converged ceiling GDN's is — so its S_5 shortfall is partly entangled with optimization difficulty: full fine-tuning of the 1.3 B raw-write model is unstable and needed the gentler/longer recipe merely to fit parity. These observations support a matched-FLOP efficiency reading; a pure-expressivity wall for either raw-write or linear GDN remains unestablished: raw-write needs more budget to approach what the delta update reaches cheaply, and within the budgets tested (including the symmetric doubled budget) it does not get there. The observations across the recorded recipes are (a) the clean expressivity contrast is delta E88 (fits the trained prefixes better, leads at every tested length) vs this GDN run (under-fits and plateaus under the tested recipes), and (b) raw-write M²RNN, even at best-effort competence, exhibits the length-extrapolation collapse and stays below the delta update's reach at every length; the delta update too plateaus below ceiling at this width.

The solvable controls hold (S3 and parity). Figure 7 surfaces the two solvable controls alongside S_5. All three architectures are competent at the trained length on both: at $T = 64$, parity is 1.000 / 0.997 / 0.999 and S_3 is 1.000 / 0.928 / 0.748 (E88 / GDN / M²RNN), and they degrade only under extrapolation — E88 still 0.565 on S_3 and 0.745 on parity at $T = 512$. Competence on the solvable tasks makes a structure-sensitive learning explanation plausible, but does not isolate non-solvability from other differences in the task and recipe. (The raw-write M²RNN's *own* S_3 decay at length, falling below the linear GDN past $T = 96$, is the same matched-FLOP efficiency story, not an expressivity wall — its solvable-task curve was still rising; see *Raw-write's shortfall is partly trainability* above.)

QA and reasoning panel at 1.3 B: parity-rate evidence

For capability beyond loss numbers we evaluate the three 1.3 B-class models on a 300-item multi-choice continuation harness sampled from ARC-C/E [50], HellaSwag [51], SciQ [52], OpenBookQA [53], and BoolQ [54]. At the current training budget, E88 reaches 0.367 (random 0.29), GDN 0.380, and M²RNN-CMA 0.367; all three sit within one standard error of one another (SE $\approx$ 6 pp at 50 items per task). On a separate reasoning panel (BIG-Bench Hard [55], ReCLor [56], FOLIO [57]), all three families collapse on multi-step object tracking (the seven-object shuffled-objects tracking task, 0.10–0.13, near-random) and on FOLIO/ReCLor (near-random for all three), with GDN leading on formal fallacies and web-of-lies. E88′s overall reasoning accuracy (0.319) is within one standard error of M²RNN-CMA (0.336). None has crossed the threshold where reasoning benchmarks differentiate, consistent with Section 1′s class-level claim at this training budget. The state-tracking-specific reasoning separation, if it exists, will emerge at longer training horizons; Section 12 states the prediction explicitly.

A.5 Held-out bits-per-byte measurement (predecessor cohort)

This appendix is the evidentiary backbone of the held-out descriptive closeness reported in Section A.4: it documents exactly how the held-out bits-per-byte figures are measured and places them in a landscape of reference models and classical compressors. Every number is measured by the eval harness described below; none is re-fitted. The measurement code, slice manifests, and re-exported checkpoints are public (see the availability statement in Section 11).

Tokenizer-invariant held-out bpb

So that models with different tokenizers are comparable on identical bytes, held-out bpb is computed from each model's real summed negative log-likelihood over the slice and the slice's real UTF-8 byte count:

$$\text{bpb} = (\text{total NLL}_{\text{nats}} / \ln 2) / \text{ total UTF-8 bytes.}$$

Each model uses its *own* tokenizer, and the denominator is the shared byte count, so bytes/token is a *measured* per-tokenizer quantity (4.00 for the GPT-NeoX BPE, 3.82 for our `p50k_base` runs on the canonical slice), whereas the train-loss curve uses the fixed bytes/token = 3.92 constant in Section 6. The held-out and train-loss figures are therefore distinct objects: they differ in normalization as well as in the data scored. Every token is scored once under a sliding window (context 2048, stride 1024) with up to 2047 tokens of left context, the standard fixed-length-model perplexity recipe; out-of-distribution anchors GPT-2 XL and OPT-1.3B run at context 1024 (GPT-2 XL position limit). Pipeline: `scripts/measure_pile_bpb.py` (open models) and `scripts/measure_pile_bpb_elman.py` (our models, in-harness forward).

Algorithm 8 states the scoring order behind the held-out measurement. The denominator counts the shared source bytes once; overlapping context is not rescored. Correct schedule-free weight recovery and the first-block sanity check precede reporting, and a failure is not a numeric measurement.

Algorithm 8 — tokenizer-invariant held-out evaluation.

```
Input: sha-verified slice bytes, model, tokenizer, window/stride
1. Load the bound inference weights; apply required y-mode recovery.
2. Tokenize the slice with this model's tokenizer.
3. Run the first-block sanity gate; stop on failure.
4. Slide the context window; mask already scored target positions.
5. Sum next-token negative log-likelihood on newly scored positions.
```

```
6. Divide total NLL by log(2) and by the slice's UTF-8 byte count.
Return: bits per byte, token/byte counts, slice and weight identities
```

Held-out slice provenance

The canonical held-out slice is a whole-line 10 MB region (9,999,511 bytes, sha256 `3e4241a9…`) taken at byte offset 10^{12} (≈76.5% into the 1.31 TB Pile master file) — a deep offset the 1.3 B runs' sub-one-epoch stream from the file start is least likely to have consumed. The five-slice robustness check of Section 6 adds four further independent, non-overlapping, sha-verified slices at offset fractions 0.137 / 0.341 / 0.523 / 0.911, each ≈10 MB and ≈2.5 M scored tokens. Identical bytes feed every model, so the byte denominator is shared and bpb is comparable across tokenizers. A block-loss sanity gate (mean nats/token on the first 2048-token block must lie in $[1.5, 4.0]$, model train loss ≈2.6) ran before any of our models' bpb was reported; all gated runs passed. The sha-verified slice manifests are released with the measurement code (see the Section 11 availability statement).

Second distribution: Common Pile (contamination control)

Because the open Pile-trained references (Pythia, GPT-Neo) saw the Pile in training, their Pile numbers are effectively in-distribution. As an independent check we re-measure through the identical pipeline on a held-out slice of Common Pile (v0.1 main-mix; 9,999,606 bytes, document-aligned on the `0x1E` record separator, at a random deep offset 65.5% into its 1 TB corpus), a permissively-licensed corpus none of these models trained on. The Pile-trained models score within ≤ 0.003 bpb of their Pile numbers there (Pythia-1.4B +0.0028, GPT-Neo-1.3B +0.0002, Pythia-1B +0.0003), so the Pile result is *not* contamination-inflated. Our three models on the same Common-Pile slice measure Emender 0.981, GDN 0.963, M²RNN-CMA 0.973 — within ≈0.02 bpb of their Pile figures, with ordering GDN < M²RNN-CMA < Emender (again not the train-loss ordering), consistent with a small distribution shift rather than a blow-up. Best classical coder on Common Pile is xz −9 at 2.061 bpb.

Checkpoints and reproducibility

The three held-out checkpoints (Emender step 1,542,000; GDN 2,031,000; M²RNN-CMA 1,491,000) are pinned to persistent storage with full schedule-free optimizer state and sha-verified. Recovering a correct inference forward from these schedule-free [30] runs requires the *y-mode* weight swap (load the optimizer state and call `optimizer.train()`); the saved model weights alone are the eval-extrapolated *x-mode* view and are catastrophic at inference (≈18 nats/token). The public HuggingFace v0.3 release was re-exported with the corrected y-mode weights and verified, by a clean-cache readback from the hub through the bundled modeling code, to reproduce the held-out bpb to within 2×10^{-5} nats (Emender 0.96613, GDN 0.96612, M²RNN-CMA 0.96132). Only the `v0.3` revision carries this corrected re-export; the earlier public revisions (`v0.1`, `v0.2`) were shipped from the same schedule-free *x-mode* pipeline, predate the fix, and are not usable for reproduction — cite `@v0.3` only.

Where the held-out bpb sits — landscape, not a ranking

Tier	*Model / coder*	*Params*	*Held-out bpb*	*Trained on Pile?*
Ours (matched budget)	Emender (E88)	1.273 B	0.966	yes, ≈0.05 epoch
Ours (matched budget)	GDN	1.352 B	0.966	yes, ≈0.05 epoch
Ours (matched budget)	M²RNN-CMA	1.307 B	0.961	yes, ≈0.05 epoch
Open Pile-trained	Pythia-1.4B	1.42 B	0.7157	yes, ≈1 epoch (in-dist.)
Open Pile-trained	GPT-Neo-1.3B	1.32 B	0.7403	yes, ≈1 epoch (in-dist.)
Open Pile-trained	Pythia-1B	1.01 B	0.7423	yes, ≈1 epoch (in-dist.)
Familiar OOD	OPT-1.3B	1.32 B	0.8615	no (OOD)
Familiar OOD	GPT-2 XL	1.56 B	1.0137	no (OOD)
Classical	xz −9 (LZMA)	—	2.190	n/a (model-free)
Classical	gzip −9 (DEFLATE)	—	2.803	n/a (model-free)

Table 13: *Held-out bits-per-byte landscape — context, not an architecture ranking.* All neural numbers are tokenizer-invariant held-out bpb on the canonical 10 MB Pile slice; classical coders are single-stream over the same bytes. The vertical ordering is *dominated by training budget and train/test distribution, not by the update rule*, and must not be read as a quality verdict. Our three models trained on ≈15–16 B tokens (≈0.05 epoch of the Pile); the open Pile-trained references saw ≈10–20× more (≈1 epoch, ≈300 B tokens) *and* are in-distribution on this slice (it is effectively training data for them — possible contamination), so their 0.72–0.74 is closer to a train-loss than a held-out number. The OOD anchors (OPT, GPT-2 XL) are out-of-distribution on the Pile, so their higher bpb is expected and is not a quality knock; GPT-2 XL measures 1.0137 here versus the Pile paper's published 1.0468 [9], a ≈0.03-bpb pipeline check confirming this single slice reads slightly low against the full Pile test set. The architecture-focused comparison is the matched-budget three-way at the top, which Section A.4 reports as descriptively close, not as a statistical equivalence result.